

Contents

第二十一册简介：Transformer 架构详解、升级变种与未来架构演进	9
本书定位	9
为什么需要这本书	9
适合读者	9
学完后应具备的能力	9
阅读建议	10
与其他书的关系	10
第二十一册：Transformer 架构详解、升级变种与未来架构演进	10
本书定位	10
适合读者	10
章节目录	10
第一部分：Transformer 基础结构重读	10
第二部分：Transformer 的优点与核心瓶颈	11
第三部分：现代 LLM 中的 Transformer 改造	11
第四部分：Linear Attention 与高效注意力路线	11
第五部分：状态空间模型 SSM 与 Mamba 路线	11
第六部分：RWKV、RetNet、Hyena 与其他替代架构	11
第七部分：混合架构与未来方向	11
第八部分：主流大模型架构创新时间线	12
与其他书的关系	12
第一批优先扩写章节	12
第一章：Transformer 为什么能成为大模型基础架构	12
0. 本讲资料边界与第二轮精修口径	12
1.1 本章定位	13
1.2 资料来源和可信边界	13
1.3 Transformer 之前的问题：序列建模不只是“读文本”	13
1.4 Self-Attention 的信息路由本质	14
1.5 并行训练：大模型时代的硬件友好性	15
1.6 统一目标：Next-Token Prediction 的威力	15
1.7 Scaling Law 巩固了 Transformer 路线	16
1.8 In-Context Learning：模型把上下文当临时程序	16
1.9 生态优势：基础设施围绕 Transformer 成熟	17
1.10 为什么不是 RNN/LSTM 成为大模型主干	17
1.11 为什么不是 CNN 成为大模型主干	18
1.12 Transformer 的核心优势总结	18
1.13 Transformer 的代价：成功不等于完美	18
1.14 一个小例子：为什么 attention 适合代码补全	19
1.15 Transformer 架构选择审计指标	19
1.15.1 最小可运行 Transformer 架构选择审计 demo	20
1.16 面试题	22
题 1：Transformer 为什么能成为大模型基础架构？	22
题 2：Transformer 相比 RNN/LSTM 的核心优势是什么？	23
题 3：为什么 decoder-only 成为通用 LLM 主流？	23
题 4：Scaling law 对 Transformer 路线有什么影响？	23
题 5：既然 Transformer 这么强，为什么还研究 Mamba、SSM、RWKV、RetNet？	23
1.17 小练习	23
1.18 本章总结	23
第二章：Self-Attention 的信息路由本质	24
0. 本讲资料边界与第二轮精修口径	24
2.1 本章定位	24
2.2 资料来源和可信边界	24

2.3 Self-Attention 解决的核心问题	25
2.4 Q、K、V 的角色直觉	25
2.5 Attention Score 是什么	26
2.6 为什么要除以 $\sqrt{d_k}$	26
2.7 Softmax 权重是不是“重要性”	27
2.8 Self-Attention 的矩阵形状	27
2.9 一个最小 PyTorch 实现	28
2.10 Causal Attention 和 Bidirectional Attention	28
2.11 信息路由图：每层都在重写 token 表示	29
2.12 Attention 和检索的相似与不同	29
2.13 为什么长上下文仍然会失败	30
2.14 Attention 的计算瓶颈	30
2.15 Self-Attention 的常见误区	30
2.16 Self-Attention 信息路由审计指标	31
2.16.1 最小可运行 Attention 路由审计 demo	31
2.17 面试题	34
题 1：Self-Attention 的本质是什么？	34
题 2：为什么 Q、K、V 要分开？	34
题 3：为什么 attention score 要除以 $\sqrt{d_k}$ ？	34
题 4：Attention weight 能解释模型行为吗？	34
题 5：为什么 attention 可以看全局，但长上下文仍会 lost-in-the-middle？	34
题 6：Self-Attention 和 RAG 中的检索有什么区别？	35
2.18 小练习	35
2.19 本章总结	35
第三章：Q、K、V、Attention Head 与表示子空间	35
0. 本讲资料边界与第二轮精修口径	35
3.1 本章定位	36
3.2 资料来源和可信边界	36
3.3 从单头到多头：为什么一个 attention 不够	36
3.4 多头注意力的数学形式	37
3.5 QK Circuit 和 OV Circuit	37
3.6 表示子空间：Head 到底分工在哪里	38
3.7 Head 数和 Head Dim 的 Trade-off	38
3.8 为什么有些 Head 可以剪掉	39
3.9 Head 的可解释性：能看，但不能迷信	39
3.10 为什么 MQA/GQA 要动 K/V Head	40
3.11 Q Head 和 KV Head 的分离直觉	40
3.12 代码：MHA 与 GQA 的形状差异	41
3.13 Head 和 Residual Stream	41
3.14 表示子空间和 Superposition	42
3.15 面试中如何解释 Head 的作用	42
3.16 常见误区	42
3.17 QKV 与多头子空间审计指标	43
3.17.1 最小可运行 QKV / Head 子空间审计 demo	43
3.18 面试题	45
题 1：Multi-head attention 为什么有用？	45
题 2：QK circuit 和 OV circuit 分别是什么？	45
题 3：为什么有些 attention head 可以剪掉？	46
题 4：MHA、MQA、GQA 的核心区别是什么？	46
题 5：为什么减少 K/V head 能加速推理？	46
题 6：Attention head 是否可解释？	46
3.19 小练习	46
3.20 本章总结	46
第四章：MHA、MQA、GQA、MLA 的演进与 Trade-off	47

0. 本讲资料边界与第二轮精修口径	47
4.1 本章定位	47
4.2 资料来源和可信边界	47
4.3 为什么注意力结构开始围绕 KV Cache 演进	48
4.4 MHA: 表达完整, 但 KV Cache 昂贵	48
4.5 MQA: 一个 K/V Head 服务所有 Query Heads	49
4.6 GQA: MHA 和 MQA 之间的折中	49
4.7 MHA、MQA、GQA 的统一视角	50
4.8 MLA: 从减少 KV Head 到压缩 KV 表示	50
4.9 MLA 和 MoE 的关系	51
4.10 FlashAttention、PagedAttention 和 MQA/GQA/MLA 的区别	51
4.11 KV Cache 粗略计算	51
4.12 训练和推理视角不同	52
4.13 什么时候选 MHA、MQA、GQA、MLA	52
4.14 Attention 架构成本审计指标	53
4.14.1 最小可运行 MHA / GQA / MQA / MLA 成本审计 demo	53
4.15 常见误区	55
4.16 面试题	55
题 1: MHA、MQA、GQA 的区别是什么?	55
题 2: 为什么 MQA/GQA 能提升推理速度?	55
题 3: GQA 为什么比 MQA 更常见?	55
题 4: MLA 和 GQA 的区别是什么?	55
题 5: FlashAttention、PagedAttention、GQA 分别优化什么?	55
4.17 小练习	55
4.18 本章总结	56
第五章: FFN、SwiGLU、Gated MLP 与 MoE FFN	56
0. 本讲资料边界与第二轮精修口径	56
5.1 本章定位	56
5.2 资料来源和可信边界	57
5.3 原始 Transformer FFN 是什么	57
5.4 FFN 为什么重要	58
5.5 ReLU、GELU 到 GLU	58
5.6 SwiGLU 是什么	58
5.7 为什么现代 LLM 爱用 SwiGLU	59
5.8 一个 PyTorch 版 SwiGLU MLP	59
5.9 FFN 参数量和计算量	60
5.10 MoE FFN: 把一个 FFN 变成多个专家	60
5.11 为什么 MoE 通常替换 FFN, 而不是 Attention	61
5.12 Router: MoE 的关键模块	61
5.13 GShard、Switch、Mixtral、DeepSeekMoE 的演进	61
5.14 Dense FFN、SwiGLU、MoE FFN 对比	62
5.15 MoE 的工程 Trade-off	62
5.16 FFN / MoE 成本审计指标	63
5.16.1 最小可运行 FFN / MoE 成本审计 demo	63
5.17 FFN 和 Attention 的互补关系	65
5.18 常见误区	65
5.19 面试题	66
题 1: Transformer FFN 的作用是什么?	66
题 2: 为什么现代 LLM 常用 SwiGLU?	66
题 3: SwiGLU 和普通 FFN 参数量怎么比?	66
题 4: 为什么 MoE 通常替换 FFN?	66
题 5: Switch Transformer 和 Mixtral 的 routing 有什么区别?	66
题 6: MoE 的主要工程难点是什么?	66
5.20 小练习	66
5.21 本章总结	67

第六章: Residual、LayerNorm、RMSNorm、Pre-Norm 与训练稳定性	67
0. 本讲资料边界与第二轮精修口径	67
6.1 本章定位	67
6.2 资料来源和可信边界	68
6.3 Residual Connection 的直觉	68
6.4 Residual Stream: 现代 Transformer 的主干	69
6.5 LayerNorm 归一化什么	69
6.6 RMSNorm: 去掉均值中心化	69
6.7 LayerNorm 和 RMSNorm 的代码	70
6.8 Post-LN Transformer	70
6.9 Pre-LN Transformer	71
6.10 为什么 Warmup 重要	71
6.11 DeepNorm: 极深 Transformer 的残差缩放	72
6.12 Admin: 残差依赖和扰动放大	72
6.13 Norm Placement 对表示的影响	72
6.14 大模型中的常见配置	73
6.15 常见训练不稳定现象	73
6.16 训练稳定性审计指标	73
6.16.1 最小可运行 Residual / Norm 稳定性审计 demo	74
6.17 常见误区	76
6.18 面试题	77
题 1: Residual connection 为什么重要?	77
题 2: LayerNorm 和 BatchNorm 有什么区别?	77
题 3: RMSNorm 和 LayerNorm 有什么区别?	77
题 4: Pre-LN 和 Post-LN 的区别是什么?	77
题 5: 为什么 Post-LN 更依赖 warmup?	77
题 6: DeepNorm 解决什么问题?	77
6.19 小练习	78
6.20 本章总结	78
第七章: Position Encoding 总览: Sinusoidal、Learned、Relative、RoPE、ALiBi	78
0. 本讲资料边界与第二轮精修口径	78
7.1 本章定位	78
7.2 资料来源和可信边界	79
7.3 为什么 Self-Attention 本身不知道顺序	79
7.4 绝对位置编码: 把“第几个位置”加到 token 上	80
7.5 Sinusoidal Position Encoding	80
7.6 Sinusoidal 为什么用 sin 和 cos	81
7.7 Learned Absolute Position Embedding	81
7.8 绝对位置编码的核心限制	82
7.9 Relative Position Representation	82
7.10 Relative Position Bias	83
7.11 RoPE: Rotary Position Embedding	83
7.12 RoPE 为什么适合 decoder-only LLM	84
7.13 RoPE 的实现直觉	84
7.14 ALiBi: Attention with Linear Biases	85
7.15 Sinusoidal、Learned、Relative、RoPE、ALiBi 对比	86
注入位置	86
绝对还是相对	86
长度外推	86
工程复杂度	86
现代 LLM 使用情况	86
7.16 位置编码和 KV Cache	86
7.17 位置编码和长上下文扩展	87
7.18 Position Interpolation	87
7.19 YaRN	88

7.20 位置编码审计指标与最小 demo	88
7.21 面向专家: RoPE 的频率、相位和外推问题	90
7.22 面向专家: 位置编码和 attention pattern 的耦合	91
7.23 常见误区	91
误区 1: 有 position embedding 就一定能处理长上下文	91
误区 2: Sinusoidal 可以生成无限位置, 所以天然能无限外推	91
误区 3: Learned position embedding 一定比固定位置编码强	91
误区 4: RoPE 是相对位置编码, 所以没有外推问题	91
误区 5: ALiBi 总是优于 RoPE	91
误区 6: passkey retrieval 通过就说明长上下文理解强	91
7.24 面试高频问题	92
题 1: 为什么 Transformer 需要位置编码?	92
题 2: Sinusoidal position encoding 的公式和直觉是什么?	92
题 3: learned absolute position embedding 有什么问题?	92
题 4: relative position representation 为什么更贴合 attention?	92
题 5: RoPE 的核心机制是什么?	92
题 6: 为什么 RoPE 适合 decoder-only LLM?	92
题 7: ALiBi 和 RoPE 有什么区别?	92
题 8: 为什么 RoPE 模型扩展上下文不能只改 max length?	92
7.25 小练习	92
7.26 本章总结	93

第八章: Causal Mask、Prefix LM、Bidirectional Attention 与注意力模式

93

0. 本讲资料边界与第二轮精修口径	93
8.1 本章定位	93
8.2 资料来源和可信边界	94
8.3 Attention Mask 是什么	94
8.4 三种容易混淆的 mask	95
Attention Mask	95
Padding Mask	95
Loss Mask	95
8.5 Causal Mask	95
8.6 Causal LM 的训练和推理	96
8.7 手写 Causal Mask	96
8.8 Bidirectional Attention	97
8.9 Encoder-Only、Decoder-Only、Encoder-Decoder	98
Encoder-Only	98
Decoder-Only	98
Encoder-Decoder	98
8.10 Encoder-Decoder 的三种 Attention	98
8.11 Prefix LM	99
8.12 Prefix LM 和 Encoder-Decoder 的区别	99
8.13 UniLM: 用 Mask 统一任务	100
8.14 UL2: 不同语言建模范式的混合	100
8.15 Sparse Attention 和 Sliding Window Attention	100
8.16 Attention Pattern 的图视角	101
8.17 Packed Sequence Training 中的 Mask	101
8.18 KV Cache 场景下的 Mask	102
8.19 FlashAttention 和 Mask	102
8.20 Mask 的数值稳定性	103
8.21 Mask 可见性审计指标与最小 demo	103
8.22 面向专家: Mask、目标函数和数据格式的耦合	106
8.23 面向专家: Bidirectional 不是不能生成, Causal 也不是不能理解	106
8.24 常见误区	107
误区 1: attention mask 和 loss mask 是一回事	107
误区 2: Causal LM 训练是一个 token 一个 token 串行训练	107

误区 3: BERT 双向看上下文, 所以一定比 GPT 强	107
误区 4: Prefix LM 等同于普通 causal LM	107
误区 5: 只要 mask 数学上对, 工程上就高效	107
误区 6: packed sequence 只要拼起来就能训练	107
8.25 面试高频问题	107
题 1: 为什么 decoder-only LLM 需要 causal mask?	107
题 2: attention mask、padding mask、loss mask 有什么区别?	107
题 3: Bidirectional attention 为什么适合理解任务?	107
题 4: Encoder-only、decoder-only、encoder-decoder 的 attention mask 有什么区别?	107
题 5: Prefix LM 的 mask 是什么样的?	108
题 6: packed sequence training 中 mask 为什么重要?	108
题 7: 为什么复杂 attention mask 可能影响性能?	108
题 8: KV cache 下 causal mask 有什么变化?	108
8.26 小练习	108
8.27 本章总结	108

第九章: Transformer 的并行性、表达力和 scaling 优势

109

0. 本讲资料边界与第二轮精修口径	109
9.1 本章定位	109
9.2 资料来源和可信边界	109
9.3 Transformer 的并行性优势	110
9.4 并行性不等于计算量少	110
9.5 GPU/TPU 友好性	111
9.6 Self-Attention 的表达力: 动态信息路由	111
9.7 一跳长距离交互	112
9.8 多头注意力: 多个子空间的并行路由	112
9.9 FFN 的表达力: 逐 token 的非线性计算	112
9.10 Residual Stream: 逐层写入和读取的共享工作区	113
9.11 Scaling Law 到底说了什么	113
9.12 Scaling Law 为什么重要	113
9.13 Chinchilla: 从“更大”到“compute-optimal”	114
9.14 Scaling 不只是参数, 还包括数据质量	114
9.15 GPT-3: Scaling 带来的 In-Context Learning	114
9.16 PaLM: 大规模训练和系统能力	115
9.17 Emergent Abilities: 涌现能力怎么理解	115
9.18 Transformer 的表达力和 Scaling 为什么能配合	116
9.19 为什么不是 CNN 或 RNN 成为 LLM 主干	116
9.20 Scaling 的边界	116
9.21 并行与 Scaling 审计指标与最小 demo	117
9.22 面向专家: Scaling Law 的正确用法	119
9.23 面向专家: 表达力、优化和归纳偏置的张力	119
9.24 常见误区	120
误区 1: Transformer 比 RNN 更省计算	120
误区 2: Scaling law 说明只要模型变大就一定更好	120
误区 3: 涌现能力说明大模型能力完全不可预测	120
误区 4: Attention 是 Transformer 表达力的全部来源	120
误区 5: Scaling law 是理论定律	120
误区 6: 更低 loss 一定意味着更安全、更可靠	120
9.25 面试高频问题	120
题 1: Transformer 为什么适合大规模并行训练?	120
题 2: Self-Attention 的表达力来自哪里?	121
题 3: Transformer 中 attention 和 FFN 分别负责什么?	121
题 4: Scaling law 对大模型训练有什么意义?	121
题 5: Chinchilla 的核心启发是什么?	121
题 6: GPT-3 的 few-shot 能力说明了什么?	121
题 7: 如何理解 emergent abilities?	121

题 8: Scaling 的边界是什么?	121
9.26 小练习	121
9.27 本章总结	122
第十章: $O(n^2)$ Attention 的计算与显存瓶颈	122
10.0 本讲资料边界与第二轮精修口径	122
10.1 本章定位	122
10.2 资料来源和可信边界	123
10.3 标准 Attention 的计算流程	123
10.4 为什么是 $O(n^2)$	124
10.5 二次复杂度为什么可怕	124
10.6 Attention 显存主要花在哪里	125
10.7 训练显存不只是参数显存	125
10.8 计算瓶颈和 IO 瓶颈	126
10.9 FlashAttention 解决了什么	126
10.10 FlashAttention 没解决什么	126
10.11 为什么 softmax attention 难以简单线性化	127
10.12 Sparse Attention	127
10.13 Linear Attention 和 Kernel Approximation	127
10.14 $O(n^2)$ 在训练、Prefill、Decode 中不同	128
训练	128
Prefill	128
Decode	128
10.15 Causal Mask 是否把复杂度减半	128
10.16 长上下文训练为什么难	129
10.17 Attention 瓶颈和 FFN 瓶颈的关系	129
10.18 Memory-Efficient Attention 和 Activation Checkpointing	130
10.19 FlashAttention、PagedAttention、GQA 不是一回事	130
10.20 面向专家: 为什么 IO-aware 很关键	130
10.21 面向专家: 为什么近似 Attention 难替代 Full Attention	131
10.22 Attention 成本审计指标与最小 demo	131
10.23 常见误区	133
误区 1: FlashAttention 把 $O(n^2)$ 变成 $O(n)$	133
误区 2: $O(n^2)$ 只是训练问题	133
误区 3: Causal mask 解决了二次复杂度	133
误区 4: 参数显存够就能训练长上下文	133
误区 5: Linear attention 一定比 full attention 好	133
误区 6: PagedAttention 是一种模型 attention 结构	133
10.24 面试高频问题	133
题 1: 为什么标准 self-attention 是 $O(n^2)$?	133
题 2: Attention 的显存瓶颈在哪里?	133
题 3: FlashAttention 解决了什么?	133
题 4: FlashAttention 是否把复杂度从 $O(n^2)$ 降到 $O(n)$?	133
题 5: 训练、prefill、decode 的 attention 成本有什么区别?	134
题 6: Sparse attention 和 linear attention 的区别是什么?	134
题 7: 为什么理论复杂度低的方法不一定真实更快?	134
题 8: FlashAttention、PagedAttention、GQA 分别优化什么?	134
10.25 小练习	134
10.26 本章总结	134
第十一章: KV Cache、长上下文推理和显存增长问题	135
11.0 本讲资料边界与第二轮精修口径	135
11.1 本章定位	135
11.2 资料来源和可信边界	136
11.3 自回归推理为什么需要 KV Cache	136
11.4 KV Cache 缓存什么	136

11.5 有无 KV Cache 的复杂度对比	137
没有 KV Cache	137
有 KV Cache	137
11.6 Prefill 和 Decode	137
Prefill	137
Decode	138
11.7 KV Cache 显存公式	138
11.8 KV Cache 粗略估算	138
11.9 KV Cache 是线性增长，但仍然可怕	139
11.10 Decode 为什么常受内存带宽限制	139
11.11 MHA、MQA、GQA 对 KV Cache 的影响	139
11.12 MLA: 从减少 KV Heads 到压缩 KV 表示	140
11.13 PagedAttention 解决什么	140
11.14 Continuous Batching	141
11.15 Prefix Sharing	141
11.16 Sliding Window Cache	141
11.17 Cache Quantization	142
11.18 长上下文推理的延迟拆解	142
11.19 吞吐、延迟、显存的三角关系	142
11.20 面向专家: KV Cache 和 Tensor Parallel	143
11.21 面向专家: 为什么 KV Cache 影响架构设计	143
11.22 KV Cache 成本审计指标与最小 demo	143
11.23 常见误区	145
误区 1: KV cache 会减少模型权重显存	145
误区 2: KV cache 越大，模型效果一定越好	145
误区 3: FlashAttention 能解决 KV cache 过大	145
误区 4: PagedAttention 是 GQA 的替代品	145
误区 5: 长上下文只影响 prefill	146
误区 6: MQA/GQA 只是减少参数	146
11.24 面试高频问题	146
题 1: KV cache 是什么? 为什么需要它?	146
题 2: KV cache 显存如何估算?	146
题 3: Prefill 和 decode 的区别是什么?	146
题 4: 为什么长上下文推理会被 KV cache 卡住?	146
题 5: MQA/GQA 为什么能提升推理速度?	146
题 6: PagedAttention 解决什么问题?	146
题 7: FlashAttention 和 PagedAttention 有什么区别?	146
题 8: MLA 和 GQA 的区别是什么?	146
11.25 小练习	147
11.26 本章总结	147

第十二章: In-Context Learning 与显式 token 检索能力	147
12.0 本讲资料边界与第二轮精修口径	147
12.1 本章定位	148
12.2 资料来源和可信边界	148
12.3 什么是 In-Context Learning	149
12.4 Zero-shot、One-shot、Few-shot	149
12.5 ICL 和 Fine-tuning 的区别	150
12.6 ICL 和 Prompt Engineering 的关系	150
12.7 ICL 和 RAG 的区别	151
12.8 为什么 Attention 对 ICL 很关键	151
12.9 显式 Token 检索能力是什么意思	152
12.10 示例到底提供了什么	152
12.11 Induction Head 的直觉	153
12.12 ICL 作为隐式优化器	153
12.13 上下文是临时程序	154

12.14 为什么 ICL 受示例顺序影响	154
12.15 为什么 ICL 受干扰信息影响	154
12.16 Lost in the Middle 与显式检索失败	155
12.17 ICL、RAG 和 Long Context 的组合	155
12.18 为什么 ICL 是替代架构的重要考题	156
12.19 面向专家：ICL 的多机制解释	156
12.20 面向专家：显式检索和隐式知识的分工	156
12.21 ICL / Token Retrieval 审计指标与最小 demo	157
12.22 常见误区	160
误区 1：ICL 就是模型在推理时 fine-tuning	160
误区 2：Few-shot 示例越多越好	160
误区 3：示例标签错了，ICL 就一定完全失效	160
误区 4：长上下文模型一定能利用所有 token	160
误区 5：RAG 检索到正确文档就一定回答正确	160
误区 6：ICL 是纯 prompt 技巧，和架构无关	160
12.23 面试高频问题	160
题 1：什么是 In-Context Learning?	160
题 2：ICL 和 fine-tuning 有什么区别?	160
题 3：为什么 attention 对 ICL 很重要?	160
题 4：Few-shot 示例提供了什么信息?	160
题 5：ICL 和 RAG 有什么关系?	160
题 6：什么是显式 token 检索能力?	161
题 7：为什么长上下文不等于强 ICL?	161
题 8：为什么 ICL 是替代架构的重要评估项?	161
12.24 小练习	161
12.25 本章总结	161

第二十一册简介：Transformer 架构详解、升级变种与未来架构演进

本书定位

第二十一册专门补齐 Transformer 架构深度、优缺点分析、现代 LLM 架构变体、Transformer 替代路线、主流大模型创新时间线和未来架构演进。

它覆盖原始 Transformer、现代 decoder-only LLM 改造、Linear Attention、SSM/S4/Mamba、RWKV、RetNet、Hyena、MoE、MLA、混合架构、主流模型演进和未来方向。

为什么需要这本书

第二册已经覆盖 Transformer 架构进阶，但仍属于总览。本书把“架构”作为独立专题，帮助读者真正理解为什么 Transformer 成为主流、瓶颈在哪里、Mamba/SSM 等路线想替代什么，GPT、LLaMA、Mistral、Mixtral、Gemini、Qwen、DeepSeek 等主流模型如何一步步互相借鉴和升级，以及未来架构更可能如何演进。

适合读者

1. 已理解基础 Transformer，但想深入架构 trade-off 的读者。
2. 准备模型架构深挖、论文讨论和开放研究题的候选人。
3. 想理解 Mamba、RWKV、RetNet、Hyena、Linear Attention 等路线的人。
4. 想按时间线理解主流大模型架构创新点和技术继承关系的人。

学完后应具备的能力

1. 能讲清 Transformer 的结构优势和 $O(n^2)$ 瓶颈。
2. 能解释 RoPE、RMSNorm、SwiGLU、MQA/GQA、MLA、MoE 等现代改造。
3. 能比较 Linear Attention、SSM、Mamba、RWKV、RetNet、Hyena 等替代路线。

4. 能判断新架构论文的贡献、局限和工程可落地性。
5. 能按时间线讲清 GPT、LLaMA、Mistral、Mixtral、Qwen、DeepSeek 等模型的关键架构创新。
6. 能回答 “Transformer 会不会被替代” 这类开放题。

阅读建议

建议学完第一册 Transformer 主线和第二册架构进阶后阅读。本书适合配合论文精读、主流模型技术报告和架构开放题训练。

与其他书的关系

第二十一册是第二册架构部分的专题扩展，也为第十册论文研究、第十六册 reasoning 架构和第十五册多模态架构提供支撑。

第二十一册：Transformer 架构详解、升级变种与未来架构演进

本书定位

本书是 “大模型算法岗系统面试二十四部曲” 的第二十一册，专门补齐 Transformer 架构深度、优缺点分析、现代 LLM 架构变体、Transformer 替代路线、主流大模型创新时间线和未来架构演进。

第二册第 11-25 讲已经覆盖 Transformer 架构进阶，本书在此基础上做专题化、系统化和论文化扩展，重点回答：

1. Transformer 为什么成为大模型主流架构。
2. Transformer 的核心优点、结构瓶颈和工程代价是什么。
3. 现代 LLM 相比原始 Transformer 做了哪些关键改造。
4. Mamba、SSM、RWKV、RetNet、Hyena、Linear Attention 等路线到底想替代什么。
5. 未来架构更可能是纯替代，还是 Attention、SSM、卷积、MoE、Memory、Retrieval 的混合系统。
6. 主流大模型从 GPT、LLaMA、Mistral、Mixtral 到 Qwen、Gemini、DeepSeek 是如何逐步升级和互相借鉴的。
7. 面试中如何评价一个新架构，而不是只复述论文宣传。

适合读者

1. 已经学完基础 Transformer，但想深入理解架构 trade-off 的读者。
2. 想准备大模型算法岗架构深挖和论文追问的候选人。
3. 想理解 Mamba、SSM、RWKV、RetNet、Hyena 等后 Transformer 路线的工程师。
4. 想从研究视角判断 “Transformer 会不会被替代” 的学习者。

章节目录

第一部分：Transformer 基础结构重读

1. Transformer 为什么能成为大模型基础架构
2. Self-Attention 的信息路由本质
3. Q、K、V、Attention Head 与表示子空间
4. MHA、MQA、GQA、MLA 的演进与 trade-off
5. FFN、SwiGLU、Gated MLP 与 MoE FFN
6. Residual、LayerNorm、RMSNorm、Pre-Norm 与训练稳定性
7. Position Encoding 总览：Sinusoidal、Learned、Relative、RoPE、ALiBi
8. Causal Mask、Prefix LM、Bidirectional Attention 与注意力模式

第二部分：Transformer 的优点与核心瓶颈

9. Transformer 的并行性、表达力和 scaling 优势
10. $O(n^2)$ Attention 的计算与显存瓶颈
11. KV Cache、长上下文推理和显存增长问题
12. In-Context Learning 与显式 token 检索能力
13. 长距离依赖、精确检索与 lost-in-the-middle
14. Transformer 的归纳偏置不足：顺序、局部性和状态记忆
15. Transformer 在流式、在线和低延迟场景的局限

第三部分：现代 LLM 中的 Transformer 改造

16. Decoder-only 为什么成为通用 LLM 主流
17. LLaMA、GPT、Mistral 风格架构对比
18. RoPE、NTK Scaling、YaRN 与长上下文位置外推
19. Sliding Window Attention、Global Attention 与稀疏注意力
20. FlashAttention、PagedAttention 与 attention kernel 优化
21. MoE Transformer: Switch、GShard、Mixtral 与 DeepSeek-MoE
22. Dense Transformer 与 MoE Transformer 的训练和服务 trade-off
23. 小模型、边缘模型与高效 Transformer 改造

第四部分：Linear Attention 与高效注意力路线

24. Linear Attention 的基本思想和核技巧
25. Performer、Linear Transformer 与注意力近似
26. 稀疏 Attention、局部 Attention、块稀疏 Attention
27. Longformer、BigBird、Reformer 等长序列路线
28. 高效 Attention 为什么没有完全替代标准 Attention

第五部分：状态空间模型 SSM 与 Mamba 路线

29. 状态空间模型 SSM 的数学直觉
30. SSM、RNN、CNN、Attention 的信息流对比
31. S4: Structured State Space 的核心思想
32. Mamba: Selective State Space 与 Selective Scan
33. Mamba 的训练、推理和硬件效率优势
34. Mamba 的局限：精确检索、ICL、规模验证与生态
35. Mamba-2 与后续 SSM 改进方向
36. SSM 会不会替代 Transformer：证据、争议与面试回答

第六部分：RWKV、RetNet、Hyena 与其他替代架构

37. RWKV: RNN 式状态与 Transformer 式训练的结合
38. RetNet: Retention 机制与并行/递推统一
39. Hyena: 隐式长卷积与长序列建模
40. Mega、Gated State Space、H3 等早期混合路线
41. 卷积序列模型在 LLM 中的复兴可能性
42. 替代架构横向对比：复杂度、表达力、硬件和生态

第七部分：混合架构与未来方向

43. Attention + SSM 混合架构为什么现实
44. Attention + Convolution + SSM 的模块分工
45. Retrieval-Augmented Architecture: 把外部检索变成架构组件
46. Memory-Augmented Transformer 与长期记忆架构
47. MoE + SSM + Attention 的未来组合可能

48. 多模态架构中的 Transformer 替代与混合路线
49. Agentic Model Architecture: 模型、工具、记忆、检索和执行器
50. 未来大模型架构会如何演进: 面试开放题

第八部分: 主流大模型架构创新时间线

51. GPT 系列: decoder-only、scaling 与 in-context learning
52. Chinchilla、PaLM 与 compute-optimal training
53. InstructGPT、ChatGPT 与 RLHF 对架构使用方式的改变
54. LLaMA 系列: RoPE、RMSNorm、SwiGLU 与开源 LLM 基线
55. Mistral/Mixtral: GQA、Sliding Window Attention 与开源 MoE
56. Gemini、Claude、GPT-4V: 多模态、长上下文与系统能力
57. Qwen、LLaMA 3、Gemma: 开源训练工程成熟化
58. DeepSeek-V2/V3/R1: MLA、DeepSeekMoE、MTP 与 reasoning RL
59. 主流模型创新点横向对比: attention、FFN、MoE、position、training recipe
60. 从时间线看下一代架构: MLA、MoE、SSM、Memory、Retrieval 与 Agentic System

本部分入口: [主流大模型架构创新时间线.md](#)

与其他书的关系

1. 第一册讲 Transformer 最小闭环, 本书讲架构深挖。
2. 第二册第 11-25 讲是进阶总览, 本书是完整专题版。
3. 第六册讲推理部署, 本书补架构对 serving 的影响。
4. 第十册讲论文复现, 本书提供架构论文主线。
5. 第十六册讲 reasoning, 本书补 reasoning model 的架构基础。
6. 第二十四册讲推理框架, 本书补 MLA、GQA、MoE 等架构设计对 serving engine 的影响。

第一批优先扩写章节

1. 第 1 章: Transformer 为什么能成为大模型基础架构。
2. 第 10 章: $O(n^2)$ Attention 的计算与显存瓶颈。
3. 第 18 章: RoPE、NTK Scaling、YaRN 与长上下文位置外推。
4. 第 29 章: 状态空间模型 SSM 的数学直觉。
5. 第 32 章: Mamba: Selective State Space 与 Selective Scan。
6. 第 36 章: SSM 会不会替代 Transformer。
7. 第 43 章: Attention + SSM 混合架构为什么现实。
8. 第 50 章: 未来大模型架构会如何演进。
9. 主流大模型架构创新时间线: 从 GPT、LLaMA、Mistral、Mixtral 到 Qwen、DeepSeek。

第一章: Transformer 为什么能成为大模型基础架构

0. 本讲资料边界与第二轮精修口径

本章第二轮精修以公开论文、技术报告和前序章节为资料边界: Transformer 原论文、GPT-3、Scaling Laws、Chinchilla、LLaMA、FlashAttention、Mamba / Mamba-2, 以及本项目第一册、第二册、第六册、第十三册、第十四册和第二十册中关于 self-attention、decoder-only、next-token prediction、KV Cache、分布式训练、推理部署和 Agent context window 的内容。

本章只解释“为什么 Transformer 在当前大模型生态里成为基础架构”, 不把某个闭源模型的内部结构、训练数据、隐藏 scaling recipe、routing 策略或系统优化细节写成确定事实; 也不把 toy 指标写成真实模型 benchmark。后续章节会专门讨论 MQA/GQA/MLA、FFN、RMSNorm、RoPE、attention 复杂度、KV Cache、SSM/Mamba、RWKV、RetNet、Hyena 和混合架构。

第二轮补强重点有三点:

1. 把 self-attention、next-token prediction、attention 成本、KV Cache 和架构选择指标改成 GitHub Markdown 更稳定的数学表达。
2. 用一组可解释指标回答“为什么不是 RNN/CNN/SSM 立刻成为通用 LLM 主干”。
3. 补一个 0 依赖 Python demo，用 toy 架构表审计路由路径、训练并行性、内容路由、scaling 证据、生态成熟度、ICL 支持和部署压力。

1.1 本章定位

很多人学 Transformer 时，会先记住几个模块：Self-Attention、Multi-Head Attention、FFN、LayerNorm、Residual、Position Encoding。这样能应付基础题，但还不够回答更高阶的问题：

为什么最后是 Transformer 成为大模型的基础架构，而不是 RNN、LSTM、CNN、纯检索系统或者其他序列模型？

这个问题是架构理解的入口。因为 Transformer 的成功不是某个单点技巧，而是多个因素叠加：并行训练、长距离信息路由、统一自回归目标、硬件友好、scaling law、预训练迁移、上下文学习和生态成熟。

学完本章，你应该能回答：

1. Transformer 诞生前，RNN/CNN 序列模型主要瓶颈是什么。
2. Self-Attention 为什么改变了信息路由方式。
3. Transformer 为什么适合大规模并行训练。
4. Decoder-only Transformer 为什么成为通用 LLM 主流。
5. Scaling law、GPT-3、Chinchilla、LLaMA 等工作如何共同巩固 Transformer 路线。
6. Transformer 成为基础架构的同时，留下了哪些瓶颈。

1.2 资料来源和可信边界

本章主要参考以下公开论文和技术报告：

1. Vaswani et al., 2017, Attention Is All You Need。提出 Transformer，使用 attention 机制替代 recurrence 和 convolution，在机器翻译任务中取得更好质量、更高并行性和更低训练成本。
2. Kaplan et al., 2020, Scaling Laws for Neural Language Models。系统研究语言模型损失随参数、数据和计算量的幂律关系，推动大规模 Transformer 训练路线。
3. Brown et al., 2020, Language Models are Few-Shot Learners。GPT-3 展示了大规模自回归语言模型的 few-shot 和 in-context learning 能力。
4. Hoffmann et al., 2022, Training Compute-Optimal Large Language Models。Chinchilla 工作指出固定计算预算下参数量和训练 token 数需要更合理匹配，很多早期大模型是 undertrained。
5. Touvron et al., 2023, LLaMA: Open and Efficient Foundation Language Models。LLaMA 展示了只使用公开数据也能训练强 foundation model，并把现代 decoder-only LLM 组件推成开源基线。
6. Gu and Dao, 2023/2024, Mamba: Linear-Time Sequence Modeling with Selective State Spaces。Mamba 论文明确指出 Transformer 及其 attention 模块几乎成为 foundation model 默认架构，同时也针对长序列计算低效提出替代方向。

本章重点解释“为什么 Transformer 成为主流”，不是说 Transformer 没有缺点，也不是说它一定永远不可替代。后续章节会专门讲 $O(n^2)$ 瓶颈、SSM/Mamba、RWKV、RetNet、Hyena 和混合架构。

1.3 Transformer 之前的问题：序列建模不只是“读文本”

语言、代码、语音、时间序列、DNA 都可以看作序列。

序列建模要解决的问题是：

给定一串 token，模型如何让每个位置理解其他位置的信息，并预测、生成或分类？

Transformer 之前，主流路线包括：

1. RNN。
2. LSTM/GRU。
3. Seq2Seq + Attention。
4. CNN/Temporal Convolution。

RNN/LSTM 的核心思想是递推：

$$h_t = f(h_{t-1}, x_t)$$

也就是说，第 t 个位置的状态来自上一个状态和当前输入。

这个设计很自然，但有三个问题。

第一，训练难并行。

因为 h_t 依赖 h_{t-1} ，你必须按时间步一步步算。GPU/TPU 喜欢大矩阵并行计算，而 RNN 的时间依赖会限制并行度。

第二，长距离依赖难。

如果第 1000 个 token 要依赖第 10 个 token，信息要经过很多步递推。LSTM/GRU 用门控缓解梯度消失，但不能彻底消除长路径带来的困难。

第三，状态瓶颈明显。

RNN 把过去信息压进一个 hidden state。这个 state 容量有限，很难同时保留大量细节。

CNN 序列模型解决了一部分并行问题。卷积可以并行处理局部窗口，但它的感受野需要堆很多层才能覆盖长距离。扩大 kernel 或 dilation 可以缓解，但仍然不像 attention 那样让任意两个 token 直接交互。

因此，Transformer 出现前的核心矛盾是：

RNN 适合顺序递推，但不适合大规模并行。

CNN 适合并行局部建模，但全局交互不够直接。

Transformer 把这个矛盾换了一种解法：不要逐步传状态，也不要只靠局部卷积，而是让所有 token 直接通过 attention 互相读取信息。

1.4 Self-Attention 的信息路由本质

Self-Attention 的公式通常写成：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\mathrm{top}}}{\sqrt{d_k}}\right)V$$

这不是单纯数学形式，它代表一种信息路由机制。

对每个 token 来说：

1. Query 表示 “我想找什么信息”。
2. Key 表示 “我能被什么查询匹配”。
3. Value 表示 “如果别人关注我，我提供什么内容”。

注意力权重就是：

当前位置应该从其他位置读取多少信息。

Self-Attention 的关键突破是路径长度。

在 RNN 中，两个相距很远的 token 通信需要经过很多步：

$$\text{token}_i \rightarrow h_{i+1} \rightarrow h_{i+2} \rightarrow \dots \rightarrow h_j$$

在 Self-Attention 中，它们可以一跳交互：

$$\text{token}_i \leftrightarrow \text{token}_j$$

这对语言很重要。因为语言中的依赖关系不总是局部的：

1. 主语和谓语可能相隔很远。
2. 代词可能指向前文实体。
3. 代码中的变量定义和使用可能隔很多行。
4. 数学推理需要反复引用条件。
5. 长上下文问答需要从远处检索证据。

Transformer 能成为大模型基础架构，首先就是因为 attention 把 “信息该从哪里来” 变成了可学习的内容路由问题。

1.5 并行训练：大模型时代的硬件友好性

Transformer 原论文标题叫 Attention Is All You Need，但摘要里同样重要的一句话是：Transformer 更可并行化，训练时间更少。

大模型不是只靠好想法，还要靠能训练。

训练大模型需要：

1. 大 batch。
2. 大矩阵乘法。
3. 高 GPU/TPU 利用率。
4. 可分布式扩展。
5. 可稳定堆深和堆宽。

Self-Attention 和 FFN 都可以表达成大矩阵运算。对长度为 n 的序列，训练时所有位置可以同时计算 Q、K、V、attention score 和 FFN。

这和 RNN 的逐步递推不同。

从硬件角度看，Transformer 的主干计算是：

Linear projection
Matrix multiplication
Softmax
MLP / FFN
Residual / normalization

这些操作很适合 GPU/TPU 加速，也方便做 kernel 优化、并行切分和混合精度训练。

所以 Transformer 的成功不是 “算法与硬件无关”。恰恰相反，它非常硬件友好。后来的 FlashAttention、PagedAttention、tensor parallel、sequence parallel、pipeline parallel、ZeRO/FSDP 等系统优化，都围绕 Transformer 训练和推理不断成熟。

一句话总结：

Transformer 不只是效果好，它还足够适合被现代加速器大规模训练。

1.6 统一目标：Next-Token Prediction 的威力

原始 Transformer 是 encoder-decoder，用于机器翻译。

但今天通用大模型主流是 decoder-only Transformer。

为什么？

因为 decoder-only 可以和自回归语言建模自然结合：

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{1:t-1})$$

也就是每一步预测下一个 token。

这个目标有几个巨大优势。

第一，数据来源极广。

网页、书籍、论文、代码、对话、数学题、文档都可以转成 next-token prediction 训练样本。

第二，任务形式统一。

翻译、摘要、问答、代码生成、分类、推理，都可以转成 “给定上下文，继续生成答案”。

第三，训练目标简单。

大规模交叉熵训练足够稳定，不需要为每个任务设计特殊 head。

第四，推理过程自然。

模型一次生成一个 token，能做开放式生成、对话和工具调用。

因此，decoder-only Transformer 和 next-token prediction 形成了一个强耦合组合：

统一架构 + 统一目标 + 海量数据 + 可扩展训练

GPT 系列证明了这条路线的可行性。GPT-3 进一步展示，当模型规模足够大时，可以不更新参数，仅通过 prompt 和 few-shot examples 完成很多任务。这就是 in-context learning 的冲击。

1.7 Scaling Law 巩固了 Transformer 路线

Transformer 成为基础架构，还有一个关键原因：它在大规模下表现出可预测的 scaling 行为。

Kaplan 等人的 scaling laws 工作显示，语言模型 loss 与模型参数量、数据量、训练计算量之间存在幂律关系。尽管后续 Chinchilla 修正了最优数据/参数配比，但核心信息没有变：

只要训练得当，扩大模型、数据和计算，Transformer 语言模型性能会持续改善。

这对产业非常重要。

如果一个架构小模型很好，但放大后不稳定，企业不会轻易投入巨额训练成本。

Transformer 给了研究者和工程团队一种信心：

1. 训练更大模型大概率有效。
2. 扩大数据大概率有效。
3. 增加计算预算大概率有效。
4. loss 下降和下游能力提升有经验规律可循。

Chinchilla 的贡献是进一步说明：不是盲目堆参数，而是要在固定 compute 下平衡参数量和训练 token 数。Chinchilla 用更小参数、更多 token 的模型超过了更大但 undertrained 的模型，说明 scaling 不是“越大越好”，而是“参数、数据、计算要匹配”。

这反过来巩固了 Transformer 路线：大家不是换掉架构，而是在 Transformer 上优化训练配比、数据质量、tokenizer、长上下文、MoE、注意力变体和推理系统。

1.8 In-Context Learning：模型把上下文当临时程序

GPT-3 的重要性在于 few-shot learning。

用户在 prompt 中写几个例子：

English: cat
French: chat

English: dog
French: chien

English: apple
French:

模型不更新参数，却能根据上下文模式继续输出。

这说明 Transformer 不只是压缩训练数据，它还能在上下文窗口内做某种“临时适配”。

为什么 attention 对 in-context learning 重要？

因为模型可以在生成当前 token 时，直接关注 prompt 中的示例、规则、格式、变量和约束。

可以把上下文看成一块临时工作内存：

Prompt examples / instructions / retrieved documents / conversation history

-> Attention routes relevant information

-> Model generates answer

这就是为什么 RAG、long-context、agent memory、tool traces 都围绕 Transformer context window 展开。Transformer 的上下文窗口天然成了模型的运行时输入空间。

当然, in-context learning 不是完美的。它会受上下文长度、位置偏置、lost-in-the-middle、干扰信息、prompt 格式影响。后续章节会展开这些问题。

1.9 生态优势：基础设施围绕 Transformer 成熟

架构能不能成为基础架构，不只取决于论文指标，还取决于生态。

Transformer 已经形成完整生态：

1. 训练框架：Megatron-LM、DeepSpeed、FSDP、Colossal-AI 等。
2. 推理框架：vLLM、TensorRT-LLM、SGLang、TGI 等。
3. Kernel 优化：FlashAttention、fused RMSNorm、fused MLP、paged KV cache。
4. 模型实现：Hugging Face Transformers、Megatron、llama.cpp 等。
5. 架构 recipe：RoPE、RMSNorm、SwiGLU、GQA、MoE、LoRA、QLoRA。
6. 评估工具：MMLU、GSM8K、HumanEval、MT-Bench、HELM 等。
7. 部署经验：KV cache、continuous batching、speculative decoding、quantization。

生态有路径依赖。一个新架构要替代 Transformer，不只要在论文 benchmark 上更好，还要回答：

1. 是否能稳定训练到数百亿、万亿参数规模？
2. 是否有成熟 kernel 和分布式实现？
3. 是否支持长上下文、工具调用、RAG、多模态、MoE？
4. 是否能迁移已有数据、评估和部署体系？
5. 是否在真实业务负载上更便宜？

这也是为什么 Mamba、SSM、RWKV、RetNet、Hyena 等路线很重要，但没有立刻完全替代 Transformer。

1.10 为什么不是 RNN/LSTM 成为大模型主干

面试里经常会问：为什么不是 LSTM？

可以从四个角度回答。

第一，训练并行性。

LSTM 时间步依赖强，不适合像 Transformer 一样在长序列上完全并行训练。

第二，长距离信息路径。

LSTM 虽有门控，但远距离依赖仍需穿过很多递推状态。Attention 可以让任意位置直接交互。

第三，状态容量。

LSTM 把历史压进固定隐状态，难以精确保留大量 token 级细节。Transformer 可以在上下文中保留所有 token 表示，并通过 attention 动态读取。

第四，生态和 scaling 验证。

Transformer 在 GPT、PaLM、LLaMA、Qwen、DeepSeek 等大模型中已经被反复验证。LSTM 路线没有在同等规模上形成同样成熟的 scaling 证据。

这不代表递推思想无价值。Mamba、RWKV、RetNet 等路线本质上都在重新探索“状态”和“线性时间”的价值，只是它们必须补齐传统 RNN 的并行训练、内容选择和长程表达问题。

1.11 为什么不是 CNN 成为大模型主干

CNN 的优势是局部性、并行性和硬件友好。

但语言建模中，纯 CNN 有几个劣势：

1. 全局依赖需要堆很多层或扩大感受野。
2. 动态内容路由不如 attention 直接。
3. 对任意两个远距离 token 的精确交互不够灵活。
4. prompt 中示例、约束、文档证据的检索更适合 attention。

卷积并没有消失。Hyena、H3、长卷积、局部卷积、混合架构都在重新利用卷积的效率和长序列优势。

但从通用 LLM 主干看，attention 的全局动态路由更符合语言、代码和上下文学习需求。

1.12 Transformer 的核心优势总结

可以把 Transformer 成为基础架构的原因压缩成七点。

第一，信息路由灵活。

任意 token 可以直接关注任意 token，适合长距离依赖、代码引用、文档问答和上下文学习。

第二，训练并行性强。

训练时不需要按时间递推，适合 GPU/TPU 大规模矩阵计算。

第三，统一目标简单。

Decoder-only + next-token prediction 可以吃海量无标注文本和代码。

第四，scaling 行为好。

参数、数据、计算扩大后，性能有较稳定的改善趋势。

第五，迁移能力强。

预训练模型可以通过 prompt、SFT、LoRA、RLHF/DPO/RL 等方式适配任务。

第六，推理形态自然。

自回归生成适合聊天、写代码、工具调用、长文生成和多轮交互。

第七，生态成熟。

训练、推理、压缩、评估、部署、开源模型都围绕 Transformer 建立。

1.13 Transformer 的代价：成功不等于完美

Transformer 的主流地位不能掩盖它的瓶颈。

最明显的是 attention 的平方复杂度。

对于长度 n 的序列，标准 attention score 是 $n \times n$ 矩阵。训练长序列时，计算和显存压力随长度快速增长。

$$S = QK^{\top}, \quad S \in \mathbb{R}^{T \times T}, \quad \mathrm{cost}_{\mathrm{attn}} = O(T^2d)$$

推理时，decoder-only 模型还要缓存每层每个历史 token 的 K/V：

$$M_{\mathrm{kv}} \approx 2LTHb$$

其中 L 是层数， T 是已缓存 token 数， H 是 hidden size， b 是每个元素的字节数；前面的 2 来自 K 和 V 两份缓存。真实系统还会受 batch size、KV heads、tensor parallel、量化和分页管理影响。

这就是为什么 MQA、GQA、MLA、PagedAttention、KV cache quantization、sliding window attention、context compression 都变得重要。

另外，Transformer 的归纳偏置也不强：

1. 不天然知道局部性。
2. 不天然有持久状态。
3. 不天然适合无限流式输入。
4. 对位置编码和长度外推敏感。
5. 在长上下文中可能 lost-in-the-middle。
6. 精确算法推理仍需要训练和后训练增强。

因此，后 Transformer 架构研究的核心不是 “Transformer 一无是处”，而是试图补它的短板：

更低复杂度、更强长上下文、更便宜 KV cache、更好状态记忆、更适合流式推理。

1.14 一个小例子：为什么 attention 适合代码补全

看一个简化代码片段：

```
def normalize_scores(scores):
    total = sum(scores)
    return [score / total for score in scores]
```

```
values = [1, 2, 3]
result = normalize_scores(values)
```

如果模型要理解最后一行，它需要关联：

1. `normalize_scores` 的函数定义。
2. `scores` 参数的含义。
3. `total` 的计算。
4. 返回值是列表。
5. `values` 是输入列表。

这些信息分布在不同位置。Self-Attention 允许最后一行的 token 直接关注函数定义、参数、变量和返回语句。

如果是更长的项目文件，注意力和检索工具结合后，可以让模型在上下文中动态读取相关代码片段。这也是 Transformer 很适合代码模型和 coding agent 的原因之一。

1.15 Transformer 架构选择审计指标

如果面试官继续追问 “为什么不是 RNN、CNN 或 Mamba 直接成为通用 LLM 主干”，可以把架构选择拆成一组指标。设候选架构为 a_i ：

$$a_i = (r_i, p_i, c_i, u_i, s_i, e_i, h_i, m_i, \ell_i, k_i, z_i)$$

其中 r_i 是远距离信息路由路径， p_i 是训练并行性， c_i 是内容相关路由能力， u_i 是 next-token prediction 适配度， s_i 是 scaling 证据， e_i 是生态成熟度， h_i 是硬件友好性， m_i 是 in-context learning 支持， ℓ_i 是长上下文效率， k_i 是 KV / state 缓存效率， z_i 是流式状态能力。

可以定义路由得分：

$$R_i = \frac{1}{d_i}$$

其中 d_i 是两个远距离 token 之间的有效通信步数。标准 self-attention 中 $d_i \approx 1$ ，RNN 类递推中 d_i 会随距离增长。

综合基础架构得分可以写成：

$$S_i = w_r R_i + w_p p_i + w_c c_i + w_u u_i + w_s s_i^{\mathrm{scale}} + w_e e_i + w_h h_i + w_m m_i$$

部署压力可以单独看：

$$D_i = 1 - \frac{L_i + K_i + Z_i}{3}$$

这里 L_i 是长上下文效率， K_i 是缓存效率， Z_i 是流式状态能力。Transformer 的 S_i 通常很高，但 D_i 也不低，这正是后续 GQA/MLA、FlashAttention、PagedAttention、sliding window、SSM 和混合架构研究的动机。

上线或主干选择门禁可以写成：

$G_{\mathrm{arch}}=\mathbb{1}[S_i\geq \tau_s \wedge P_i\geq \tau_p \wedge C_i\geq \tau_c \wedge S_i^{\mathrm{scale}}\geq \tau_{\mathrm{scale}}]$

注意，这不是说 Transformer 在每个单项都最优，而是说它在“可并行训练、内容路由、next-token 目标、scaling 证据、生态和上下文学习”这些基础模型核心维度上同时过线。

1.15.1 最小可运行 Transformer 架构选择审计 demo

下面的 demo 不训练模型，也不调用任何深度学习框架，只用 toy 评分表说明：RNN/LSTM 有流式状态优势但并行性和远距离路由弱；CNN/TCN 并行和硬件友好但内容相关路由弱；SSM/Mamba-like 长上下文效率高但大规模通用 LLM 证据和生态仍需积累；Transformer 综合门禁最高，但部署压力也最高；attention + SSM hybrid 有现实潜力，但仍要补 scaling 和生态证据。

```
architectures = [
    {
        "name": "rnn_lstm",
        "route_steps": 16,
        "train_parallel": 0.15,
        "content_routing": 0.35,
        "ntp_alignment": 0.85,
        "scaling_evidence": 0.25,
        "ecosystem": 0.35,
        "hardware_fit": 0.45,
        "icl_support": 0.35,
        "long_context_efficiency": 0.90,
        "kv_efficiency": 1.00,
        "streaming_state": 1.00,
    },
    {
        "name": "cnn_tcn",
        "route_steps": 4,
        "train_parallel": 0.90,
        "content_routing": 0.45,
        "ntp_alignment": 0.80,
        "scaling_evidence": 0.35,
        "ecosystem": 0.45,
        "hardware_fit": 0.85,
        "icl_support": 0.45,
        "long_context_efficiency": 0.75,
        "kv_efficiency": 1.00,
        "streaming_state": 0.50,
    },
    {
        "name": "transformer",
        "route_steps": 1,
        "train_parallel": 0.95,
        "content_routing": 0.95,
        "ntp_alignment": 0.95,
        "scaling_evidence": 0.95,
        "ecosystem": 0.95,
        "hardware_fit": 0.90,
        "icl_support": 0.95,
        "long_context_efficiency": 0.35,
        "kv_efficiency": 0.35,
        "streaming_state": 0.35,
    },
]
```

```

{
    "name": "ssm_mamba_like",
    "route_steps": 8,
    "train_parallel": 0.80,
    "content_routing": 0.75,
    "ntp_alignment": 0.80,
    "scaling_evidence": 0.55,
    "ecosystem": 0.45,
    "hardware_fit": 0.85,
    "icl_support": 0.55,
    "long_context_efficiency": 0.95,
    "kv_efficiency": 1.00,
    "streaming_state": 0.95,
},
{
    "name": "attention_ssm_hybrid",
    "route_steps": 2,
    "train_parallel": 0.90,
    "content_routing": 0.90,
    "ntp_alignment": 0.90,
    "scaling_evidence": 0.70,
    "ecosystem": 0.65,
    "hardware_fit": 0.80,
    "icl_support": 0.85,
    "long_context_efficiency": 0.70,
    "kv_efficiency": 0.65,
    "streaming_state": 0.65,
},
]

weights = {
    "route_score": 0.16,
    "train_parallel": 0.14,
    "content_routing": 0.16,
    "ntp_alignment": 0.12,
    "scaling_evidence": 0.16,
    "ecosystem": 0.12,
    "hardware_fit": 0.07,
    "icl_support": 0.07,
}

def route_score(route_steps):
    return round(1 / route_steps, 3)

def weighted_score(row):
    score = weights["route_score"] * route_score(row["route_steps"])
    for key, weight in weights.items():
        if key != "route_score":
            score += weight * row[key]
    return round(score, 3)

def avg(values):

```

```
return round(sum(values) / len(values), 3)
```

```
ranked = []
tradeoffs = {}
for row in architectures:
    foundation_score = weighted_score(row)
    pressure = {
        "long_context": row["long_context_efficiency"],
        "kv_cache": row["kv_efficiency"],
        "streaming": row["streaming_state"],
    }
    gates = {
        "parallel_ok": row["train_parallel"] >= 0.75,
        "routing_ok": route_score(row["route_steps"]) >= 0.50 and row["content_routing"] >= 0.80,
        "scaling_ok": row["scaling_evidence"] >= 0.70,
        "ecosystem_ok": row["ecosystem"] >= 0.65,
        "icl_ok": row["icl_support"] >= 0.80,
        "score_ok": foundation_score >= 0.80,
    }
    ranked.append((row["name"], foundation_score, all(gates.values())))
    tradeoffs[row["name"]] = {
        "route_score": route_score(row["route_steps"]),
        "foundation_score": foundation_score,
        "deployment_pressure": round(1 - avg(pressure.values()), 3),
        "failed_gates": [name for name, passed in gates.items() if not passed],
    }

ranked.sort(key=lambda item: item[1], reverse=True)
print(f"ranked={ranked}")
print(f"transformer_tradeoff={tradeoffs['transformer']}")
print(f"hybrid_tradeoff={tradeoffs['attention_ssm_hybrid']}")
print(f"weakest={ranked[-1]}")
```

运行结果应类似：

```
ranked=[('transformer', 0.955, True), ('attention_ssm_hybrid', 0.764, False), ('ssm_mamba_like', 0.588, False), ('cnn_to
transformer_tradeoff={'route_score': 1.0, 'foundation_score': 0.955, 'deployment_pressure': 0.65, 'failed_gates': []}
hybrid_tradeoff={'route_score': 0.5, 'foundation_score': 0.764, 'deployment_pressure': 0.333, 'failed_gates': ['score_o
weakest=('rnn_lstm', 0.327, False)
```

面试表达时要强调：这个 demo 的数字是 toy 评分，不是论文 benchmark。它的价值在于把 “Transformer 为什么赢” 讲成多维度门禁：不是某个模块神奇，而是同时满足并行训练、内容路由、统一目标、scaling 证据、生态成熟和上下文学习；它的部署压力也解释了为什么后续章节要继续研究高效 attention、KV cache 和混合架构。

1.16 面试题

题 1：Transformer 为什么能成为大模型基础架构？

参考回答：

我会从四个层面回答。第一是算法层，Self-Attention 让任意 token 直接交互，解决 RNN 长路径和 CNN 局部感受野的问题。第二是 only Transformer 可以用 next-token prediction 统一海量文本和代码数据。第四是 scaling 和生态层，GPT-3、Chinchilla、LLaMA 等工作证明 Transformer 随参数、数据和计算扩大能稳定提升，并形成成熟训练、推理和开源生态。

题 2: Transformer 相比 RNN/LSTM 的核心优势是什么?

参考回答:

RNN/LSTM 按时间递推, 训练并行性差, 远距离依赖需要穿过很多 hidden states。Transformer 用 Self-Attention 让所有位置并行计算, 并让任意两个 token 一跳交互。它更适合大规模 GPU/TPU 训练, 也更适合在上下文中动态检索信息。

题 3: 为什么 decoder-only 成为通用 LLM 主流?

参考回答:

因为 decoder-only 和 next-token prediction 结合得最自然。它用 causal mask 做自回归建模, 可以把网页、书籍、代码、对话等海量数据用于训练。decoder-only 更适理解任务, encoder-decoder 更适合 seq2seq, 但通用生成式 assistant 更需要 decoder-only 的统一生成能力。

题 4: Scaling law 对 Transformer 路线有什么影响?

参考回答:

Scaling law 给了产业和研究很强的可预期性: 在 Transformer 语言模型上, 扩大参数、数据和计算通常能带来 loss 和能力改善。GPT-3 展示了大规模 decoder-only 模型的 few-shot 能力, Chinchilla 则进一步说明参数量和训练 token 数要匹配, 不能只堆参数。这些

题 5: 既然 Transformer 这么强, 为什么还研究 Mamba、SSM、RWKV、RetNet?

参考回答:

因为 Transformer 成功不等于没有瓶颈。标准 attention 有 $O(n^2)$ 训练成本, 长上下文推理还要维护很大的 KV cache; Transformer

1.17 小练习

1. 用一张表比较 RNN、CNN、Transformer 在并行性、长距离依赖、状态表示、硬件友好性上的差异。
2. 手写 Self-Attention 的 Q、K、V 直觉解释, 不使用公式也能讲清楚。
3. 解释为什么 decoder-only + next-token prediction 可以统一翻译、摘要、问答和代码生成。
4. 阅读 Attention Is All You Need 摘要, 找出作者强调的三个关键词: attention、parallelizable、training cost。
5. 阅读 GPT-3 摘要, 解释 few-shot learning 和 fine-tuning 的区别。
6. 阅读 Chinchilla 摘要, 解释为什么 “更大模型” 不一定 compute-optimal。
7. 用 2 分钟回答 “Transformer 会不会被 Mamba 替代”, 要求同时讲 Transformer 优势和瓶颈。
8. 用本章 demo 的维度重新评估一个你最近读到的新架构, 把它的高分项、低分项和仍缺证据的地方写清楚。

1.18 本章总结

本章回答了 “Transformer 为什么能成为大模型基础架构”。

核心结论:

1. Transformer 解决了 RNN 难并行、长路径依赖和 CNN 全局交互不直接的问题。
2. Self-Attention 的本质是内容相关的信息路由, 让任意 token 可以动态读取其他 token。
3. Transformer 的训练计算非常适合 GPU/TPU, 大规模并行训练是它成功的关键条件。
4. Decoder-only Transformer 和 next-token prediction 形成了统一架构与统一训练目标。
5. GPT-3 展示了大规模自回归模型的 few-shot 和 in-context learning 能力。
6. Scaling laws 和 Chinchilla 让大模型训练从经验扩展走向更可预测的参数、数据和计算配比。
7. LLaMA 等开源模型把现代 Transformer recipe 推成生态基线。
8. Transformer 的瓶颈仍然明显, 尤其是 $O(n^2)$ attention、KV cache、长上下文和流式状态问题。
9. 后续 Mamba、SSM、RWKV、RetNet、Hyena 等路线, 都是围绕这些瓶颈展开的替代或混合尝试。

下一章会深入 Self-Attention 的信息路由本质, 重点讲 Q、K、V、attention score、softmax 和多 token 交互到底在学什么。

第二章：Self-Attention 的信息路由本质

0. 本讲资料边界与第二轮精修口径

本章第二轮精修以公开论文、文档和前序章节为资料边界：Transformer 原论文对 scaled dot-product attention 和 multi-head attention 的定义，BERT 对双向 encoder self-attention 的使用，Michel 等人关于 attention head 冗余和可解释性边界的分析，Efficient Transformers survey 对 attention 效率路线的梳理，Lost in the Middle 对长上下文位置鲁棒性的实验结论，Mistral 7B 关于 GQA 和 sliding window attention 的公开技术报告，FlashAttention-2 对长序列 attention kernel 瓶颈的说明，以及 PyTorch scaled_dot_product_attention 的公开 API 语义。

本章只解释 Self-Attention 的信息路由机制、数学形状、mask、解释边界和工程瓶颈；不把 attention weight 写成完整因果解释，不展开具体闭源模型的 head 功能归因，也不把 toy demo 的数值当成真实模型行为。后续章节会继续拆 Q/K/V 子空间、多头、MHA/MQA/GQA/MLA、长上下文、KV Cache 和高效 attention。

第二轮补强重点有三点：

1. 将 Q/K/V、scaled dot-product attention、softmax 权重、输出聚合、形状和复杂度改成稳定 MathJax 表达。
2. 明确 attention weight 的解释边界：它是局部信息混合权重，不是完整因果归因。
3. 补一个 0 依赖 Python demo，审计 attention 路由命中、causal mask 未来泄漏、softmax 行归一化、缩放后的 entropy 变化、lost-in-the-middle 和解释风险。

2.1 本章定位

上一章讲了 Transformer 为什么能成为大模型基础架构。本章进入 Transformer 的核心：Self-Attention。

很多人会背公式：

$$\mathrm{Attention}(Q,K,V)=\mathrm{softmax}\left(\frac{QK^{\mathrm{top}}}{\sqrt{d_k}}\right)V$$

但如果只会背公式，面试官继续追问就容易卡住：

1. Q、K、V 为什么要分开？
2. Attention score 到底表示什么？
3. 为什么要除以 $\sqrt{d_k}$ ？
4. Softmax 后的权重能不能解释成“重要性”？
5. Self-Attention 为什么是一种信息路由机制？
6. 它和 RNN 的状态传递、CNN 的局部卷积有什么根本差别？
7. 为什么 attention 强，但长上下文仍然会 lost-in-the-middle？

本章目标是把 Self-Attention 从“公式模块”讲成“信息流系统”。学完后，你应该能从直觉、数学、代码、工程和面试五个角度解释它。

2.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need。原论文定义 scaled dot-product attention 和 multi-head attention，指出 attention 可以让模型关注不同位置的信息，并替代 recurrence/convolution。
2. Devlin et al., 2018/2019, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding。BERT 展示了 encoder self-attention 如何联合条件化左右上下文，形成深层双向表示。
3. Michel et al., 2019, Are Sixteen Heads Really Better than One?。分析多头注意力中部分 head 可被裁剪，提醒我们不要把每个 attention head 都过度解释成稳定语义专家。
4. Tay et al., 2020/2022, Efficient Transformers: A Survey。系统梳理围绕 attention 计算和显存效率的改造路线。
5. Liu et al., 2023, Lost in the Middle。指出长上下文模型虽然能接收长输入，但对中间位置相关信息的利用不一定鲁棒。
6. Jiang et al., 2023, Mistral 7B。公开强调 GQA 和 Sliding Window Attention，用于提升推理效率和降低长序列成本。

7. Dao, 2023, FlashAttention-2。说明 attention 是长序列扩展的主要瓶颈之一，并从 GPU 并行和 work partitioning 角度优化 attention kernel。

本章会强调一个边界：attention weight 有解释价值，但不能被简单等同于“模型真正推理原因”。Attention 是信息混合权重，不是完整因果解释。

2.3 Self-Attention 解决的核心问题

序列模型的核心问题是：

每个位置应该从哪些其他位置读取信息？读取多少？读到的信息如何汇总？

举一个句子：

The animal didn't cross the street because it was too tired.

这里 it 指代谁？更可能是 animal，不是 street。

模型在处理 it 时，需要从前文中找到相关实体，并结合语义判断。

RNN 的方式是：

把前文压缩到 hidden state，再一步步传过来。

CNN 的方式是：

用局部窗口和多层堆叠扩大感受野。

Self-Attention 的方式是：

当前位置直接向所有可见位置发起查询，根据匹配程度加权读取它们的信息。

这就是信息路由：

1. 谁需要信息。
2. 谁提供信息。
3. 谁和谁匹配。
4. 信息按什么权重流动。
5. 多个来源如何汇总成新的表示。

Self-Attention 不是简单“看所有 token”，而是为每个 token 动态构造一条从其他 token 汇聚信息的路由表。

2.4 Q、K、V 的角色直觉

Q、K、V 来自同一个输入表示 X 的三个线性投影：

$Q=XW_Q, K=XW_K, V=XW_V$

可以用信息检索类比：

1. Query：当前 token 发出的查询。
2. Key：每个 token 暴露出来的索引。
3. Value：每个 token 真正提供的内容。

为什么不直接用同一个向量？

因为“用什么来匹配”和“提供什么内容”不一定是同一件事。

举例：

The animal didn't cross the street because it was too tired.

it 的 Query 可能在寻找：

一个前文实体，能解释 tired 这个状态。

animal 的 Key 可能表示：

我是一个可被代词指代的生物实体。

animal 的 Value 可能包含：

animal 的语义、数、位置、上下文信息。

所以 Q/K/V 分开后，模型可以学到不同子空间：

1. 用 Q/K 做匹配。
2. 用 V 做信息传递。
3. 匹配逻辑和内容表示可以不同。

这是 attention 的一个核心设计。

2.5 Attention Score 是什么

Attention score 通常是点积：

$$s_{ij} = q_i^T k_j$$

它表示位置 i 的 Query 和位置 j 的 Key 的匹配程度。

如果 q_i 和 k_j 方向相近，点积大，说明位置 i 更应该读取位置 j 的信息。

对一个长度为 n 的序列，所有 token 两两计算 score，得到一个 $n \times n$ 矩阵：

$$S = QK^T, \quad S \in \mathbb{R}^{N \times N}$$

第 i 行表示：

第 i 个 token 对所有 token 的关注打分。

第 j 列表示：

第 j 个 token 被其他 token 关注的程度。

这张矩阵就是一张动态信息路由图。

每个 forward pass，每一层，每个 head 都会根据当前输入重新生成不同的路由图。

这和固定卷积核很不一样。卷积的局部连接模式基本固定，而 attention 的连接权重由内容决定。

2.6 为什么要除以 $\sqrt{d_k}$

Scaled dot-product attention 会计算：

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

为什么要除以 $\sqrt{d_k}$ ？

直觉是：如果向量维度 d_k 很大，随机向量点积的方差会变大，score 数值可能很大。

Score 很大时，softmax 会变得非常尖锐：

一个位置接近 1，其他位置接近 0

这会带来两个问题：

1. 梯度变小，训练不稳定。
2. 早期训练时 attention 过早变成硬选择。

除以 $\sqrt{d_k}$ 是为了控制 score 的尺度，让 softmax 落在更合适的梯度区间。

面试中可以这样说：

缩放不是为了改变匹配关系，而是为了控制点积分布的方差，避免维度变大后 softmax 饱和。

2.7 Softmax 权重是不是“重要性”

Softmax 后得到 attention weights:

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{m=1}^N \exp(s_{im})}$$

然后输出:

$$o_i = \sum_{j=1}^N a_{ij} v_j$$

从计算角度, a_{ij} 确实表示位置 j 的 Value 对位置 i 输出的加权贡献。

但要小心: attention weight 不等于完整解释。

原因有几个:

1. Value 向量本身已经是复杂表示, 不是原始 token。
2. 后面还有残差、FFN、LayerNorm、多层堆叠。
3. 多个 head 会并行混合信息。
4. 注意力权重可被参数重排影响。
5. 有些 head 可以裁剪而不明显影响性能。

Michel 等人的多头裁剪研究提醒我们: 不是每个 head 都稳定承担可解释语义角色。某些 head 可能冗余, 某些层更依赖多头。

所以更准确的表述是:

Attention weight 是当前 head 当前层的信息混合权重, 有局部解释价值, 但不能直接等同于模型最终决策原因。

2.8 Self-Attention 的矩阵形状

假设:

$$D = H \cdot D_h$$

其中 B 是 batch size, N 是 sequence length, D 是 hidden size, H 是 head 数, D_h 是 head dim。

输入:

$$X: [B, N, D]$$

投影后:

$$Q: [B, H, N, D_h]$$

$$K: [B, H, N, D_h]$$

$$V: [B, H, N, D_h]$$

Attention score:

$$QK^T: [B, H, N, N]$$

Attention output:

$$\text{Attn @ } V: [B, H, N, D_h]$$

合并 heads 后:

$$\text{Output: } [B, N, D]$$

这里最关键的是 $[N, N]$ 。

它说明每个 head 都构造了一张 token-to-token 的关系图。这也是 attention 强大的来源, 也是 $O(n^2)$ 瓶颈的来源。

2.9 一个最小 PyTorch 实现

下面是一个简化的单头 causal self-attention。

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class SingleHeadCausalSelfAttention(nn.Module):
    def __init__(self, hidden_size: int, head_dim: int):
        super().__init__()
        self.q_proj = nn.Linear(hidden_size, head_dim, bias=False)
        self.k_proj = nn.Linear(hidden_size, head_dim, bias=False)
        self.v_proj = nn.Linear(hidden_size, head_dim, bias=False)
        self.out_proj = nn.Linear(head_dim, hidden_size, bias=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: [batch, seq_len, hidden_size]
        batch_size, seq_len, _ = x.shape

        q = self.q_proj(x) # [batch, seq_len, head_dim]
        k = self.k_proj(x) # [batch, seq_len, head_dim]
        v = self.v_proj(x) # [batch, seq_len, head_dim]

        scores = q @ k.transpose(-2, -1)
        scores = scores / math.sqrt(k.shape[-1])

        causal_mask = torch.triu(
            torch.ones(seq_len, seq_len, device=x.device, dtype=torch.bool),
            diagonal=1,
        )
        scores = scores.masked_fill(causal_mask, float("-inf"))

        weights = F.softmax(scores, dim=-1)
        out = weights @ v
        return self.out_proj(out)
```

这段代码对应的核心链路是：

$x \rightarrow q/k/v \rightarrow \text{score} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow \text{weighted sum of } v \rightarrow \text{output projection}$

如果是 encoder self-attention，就不使用 causal mask，所有 token 可以双向互相看。BERT 就是典型的双向 Transformer encoder。

2.10 Causal Attention 和 Bidirectional Attention

Self-Attention 本身不规定能看哪些位置。能看哪些位置由 mask 决定。

Decoder-only LLM 使用 causal mask：

当前位置只能看自己和过去 token，不能看未来 token。

这适合 next-token prediction：

$p(x_t | x_{1:t-1})$

Encoder-only 模型如 BERT 使用 bidirectional attention：

每个 token 可以同时看左边和右边。

这适理解任务，比如分类、抽取、问答表示学习。

区别可以总结为：

类型	可见范围	典型模型	适合任务
Causal self-attention	只能看过去	GPT、LLaMA、Qwen	生成、对话、代码补全
Bidirectional self-attention	左右都能看	BERT、RoBERTa	理解、分类、抽取
Encoder-decoder attention	decoder 看 encoder 输出	T5、原始 Transformer 翻译	seq2seq

所以“Self-Attention”不等于“GPT 注意力”。GPT 的关键是 causal self-attention。

2.11 信息路由图：每层都在重写 token 表示

Self-Attention 的输出不是最终答案，而是新的 token 表示。

第 l 层输入：

$$X^{\{\ell\}}=[x_1^{\{\ell\}},x_2^{\{\ell\}},\ldots,x_N^{\{\ell\}}]$$

经过 attention 后：

$$\tilde{x}_i^{\{\ell\}}=\sum_{j=1}^Na_{ij}^{\{\ell\}}v_j^{\{\ell\}}$$

这意味着第 i 个 token 的表示被其他 token 的信息重写。

多层堆叠后，一个 token 的表示会逐步聚合越来越复杂的上下文信息。

例如在代码中：

```
result = normalize_scores(values)
```

result 这个 token 的高层表示可能融合了：

1. 函数定义。
2. 参数类型。
3. 返回值含义。
4. 当前作用域。
5. 上下文中的变量命名习惯。

这就是为什么 Transformer 的每一层都不是简单“抽特征”，而是在重新路由和组合上下文信息。

2.12 Attention 和检索的相似与不同

Attention 很像检索：

query -> match keys -> retrieve values

但它和传统向量检索不同。

相似点：

1. 都有 query。
2. 都根据相似度找相关内容。
3. 都返回某种 value 或文档表示。

不同点：

1. Attention 的 K/V 来自当前上下文内部，不是外部索引库。
2. Attention 是可微分的，会端到端训练。
3. Attention 每层每头动态生成路由。
4. Attention 输出是加权混合，而不是简单 top-k 文档。
5. Attention 受上下文长度限制。

RAG 可以看作把外部检索和内部 attention 结合：

external retriever 找文档

-> 文档放入 prompt

-> Transformer attention 在上下文内读取文档

因此，Transformer 的 context window 是一种内部工作记忆，而 RAG 是外部记忆补充。

2.13 为什么长上下文仍然会失败

既然 attention 可以看所有 token，为什么长上下文模型还会漏掉中间信息？

原因是“可见”不等于“会用”。

Lost in the Middle 的研究发现，模型在多文档问答和 key-value retrieval 中，相关信息放在上下文开头或结尾时表现更好，放在中间时性能可能下降。

这说明：

1. Attention 矩阵允许访问中间 token。
2. 但模型训练分布、位置编码、注意力模式和生成策略可能让它更偏向首尾信息。
3. 长上下文中的干扰信息会竞争 attention 权重。
4. 深层表示不一定保留所有细节。

所以长上下文能力要分开看：

能接收多长输入 != 能稳定利用所有位置的信息

这也是为什么后续会有长上下文评估、retrieval-aware training、context compression、memory、global attention、sliding window + global token 等路线。

2.14 Attention 的计算瓶颈

Self-Attention 强在 $N \times N$ 路由图，也贵在 $N \times N$ 路由图。

训练时，score 矩阵大小是：

[batch, heads, seq_len, seq_len]

当 seq_len 翻倍，attention score 的元素数量约变成 4 倍。

FlashAttention-2 摘要里明确指出，attention layer 是扩展到更长序列时的主要瓶颈，运行时间和内存随序列长度平方增长。

高效 attention 路线通常做几类事情：

1. 不近似 attention，但优化 IO 和 kernel，例如 FlashAttention。
2. 改 attention pattern，例如 sliding window、block sparse、global tokens。
3. 近似 attention，例如 low-rank、kernel attention、linear attention。
4. 减少 KV cache，例如 MQA、GQA、MLA。
5. 用其他序列模型替代部分 attention，例如 SSM、Mamba、Hyena。

Mistral 7B 使用 GQA 和 Sliding Window Attention，就是在保持 Transformer 主干的同时降低推理成本和长序列成本。

2.15 Self-Attention 的常见误区

误区一：Attention 权重就是因果解释。

更准确：attention weight 是局部信息混合权重，不是完整因果解释。

误区二：Attention 能看全局，所以长上下文一定可靠。

更准确：可见范围只是上限，模型是否能稳定利用，还取决于训练、位置、干扰和评估。

误区三：多头越多越好。

更准确：多头提供多个子空间和路由模式，但部分 head 可能冗余，head 数需要和模型规模、任务、推理成本一起考虑。

误区四：Self-Attention 只是在做相似度检索。

更准确：它像可微分检索，但 Q/K/V、残差、FFN、多层堆叠让它远比一次相似度搜索复杂。

误区五：Attention 的 $O(n^2)$ 只是训练问题。

更准确：训练有 score 矩阵成本，推理还有 KV cache 成本。长上下文 serving 时 KV cache 往往是核心瓶颈。

2.16 Self-Attention 信息路由审计指标

为了把 Self-Attention 从公式讲成可检查的信息路由系统，可以把一个 attention routing case 抽象成：

$$r_i = (q_i, K_i, V_i, M_i, y_i, z_i, e_i)$$

其中 q_i 是当前 Query， K_i, V_i 是候选 token 的 Key / Value 集合， M_i 是可见性 mask， y_i 是期望路由目标， z_i 是缩放、softmax 和输出聚合后的记录， e_i 是人工或任务层面的证据标签。

路由命中率可以写成：

$$C_{\{\mathrm{route}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\arg\max_j a_{ij} = y_i]$$

causal mask 的未来泄漏率：

$$R_{\{\mathrm{future}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\left[\sum_{j \in F_i} a_{ij}\right] > \epsilon]$$

softmax 行归一化检查：

$$C_{\{\mathrm{row}\}} = \mathbb{1}[\forall i, \left|\sum_{j=1}^N a_{ij} - 1\right| \leq \epsilon]$$

缩放带来的稳定性可以用 entropy 变化粗略观察：

$$\Delta H = \frac{1}{N} \sum_{i=1}^N \left(H(A_i^{\{\mathrm{scaled}\}}) - H(A_i^{\{\mathrm{raw}\}}) \right)$$

解释风险率用于提醒不要把 attention weight 直接当成最终因果解释：

$$R_{\{\mathrm{interp}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\neg \mathrm{supported_explanation}(r_i)]$$

一个简化门禁可以写成：

$$G_{\{\mathrm{routing}\}} = \mathbb{1}[C_{\{\mathrm{route}\}} \geq \tau_r \wedge R_{\{\mathrm{future}\}} = 0 \wedge C_{\{\mathrm{row}\}} = 1 \wedge \Delta H \geq \tau_H]$$

这组指标的重点不是替代真实模型评估，而是帮助面试表达更严谨：attention 要同时检查路由是否命中、mask 是否正确、softmax 是否归一化、缩放是否缓解过尖分布，以及解释边界是否被夸大。

2.16.1 最小可运行 Attention 路由审计 demo

下面的 demo 不依赖 PyTorch，只用 Python 标准库模拟 4 个 routing case：代词指代、causal mask 阻断未来 token、lost-in-the-middle 路由失败，以及 attention weight 不足以作为因果解释的样本。

```
import math
```

```
cases = [
    {
        "id": "pronoun_coref_scaled",
        "query": [1.2, 0.2, 0.0],
        "keys": {
            "animal": [1.1, 0.1, 0.0],
            "street": [0.1, 1.0, 0.0],
            "it": [1.0, 0.0, 0.2],
            "tired": [0.9, 0.2, 0.1],
        },
    },
]
```

```

    "expected": "animal",
    "visible": ["animal", "street", "it", "tired"],
    "mask_future": False,
    "interpretation_support": True,
},
{
    "id": "causal_mask_blocks_future",
    "query": [0.0, 1.0, 0.1],
    "keys": {
        "x0": [0.1, 0.9, 0.0],
        "x1": [0.0, 1.1, 0.0],
        "future_answer": [0.0, 1.4, 0.2],
        "future_noise": [1.0, 0.1, 0.0],
    },
    "expected": "x1",
    "visible": ["x0", "x1"],
    "mask_future": True,
    "interpretation_support": True,
},
{
    "id": "lost_middle_route_miss",
    "query": [1.0, 0.0, 0.0],
    "keys": {
        "doc_start": [0.9, 0.0, 0.0],
        "doc_middle_answer": [0.75, 0.0, 0.0],
        "doc_end": [0.95, 0.0, 0.0],
    },
    "expected": "doc_middle_answer",
    "visible": ["doc_start", "doc_middle_answer", "doc_end"],
    "mask_future": False,
    "interpretation_support": True,
},
{
    "id": "attention_weight_not_causal_explanation",
    "query": [0.4, 0.8, 0.0],
    "keys": {
        "format_token": [0.2, 1.0, 0.0],
        "evidence_token": [1.0, 0.2, 0.0],
        "distractor": [0.3, 0.7, 0.0],
    },
    "expected": "evidence_token",
    "visible": ["format_token", "evidence_token", "distractor"],
    "mask_future": False,
    "interpretation_support": False,
},
]

```

```

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

```

```

def softmax(scores):
    m = max(scores)
    exps = [math.exp(s - m) for s in scores]

```



```

total = sum(exps)
return [x / total for x in exps]

def entropy(probs):
    return -sum(p * math.log(p + 1e-12) for p in probs)

def attend(case, scaled):
    labels = list(case["keys"])
    d_k = len(case["query"])
    scores = []
    for label in labels:
        score = dot(case["query"], case["keys"][label])
        if scaled:
            score /= math.sqrt(d_k)
        if label not in case["visible"]:
            score = -1e9
        scores.append(score)
    weights = softmax(scores)
    return labels, weights

records = {}
for case in cases:
    labels, raw_weights = attend(case, scaled=False)
    _, scaled_weights = attend(case, scaled=True)
    top_label = labels[max(range(len(labels)), key=lambda i: scaled_weights[i])]
    future_mass = sum(
        weight for label, weight in zip(labels, scaled_weights)
        if label not in case["visible"]
    )
    records[case["id"]] = {
        "top": top_label,
        "expected": case["expected"],
        "route_hit": top_label == case["expected"],
        "future_mass": round(future_mass, 6),
        "raw_entropy": round(entropy(raw_weights), 3),
        "scaled_entropy": round(entropy(scaled_weights), 3),
        "row_sum": round(sum(scaled_weights), 6),
        "interpretation_support": case["interpretation_support"],
    }

metrics = {
    "route_hit_rate": round(sum(r["route_hit"] for r in records.values()) / len(records), 3),
    "future_leak_rate": round(sum(r["future_mass"] > 1e-6 for r in records.values()) / len(records), 3),
    "row_sum_ok": all(abs(r["row_sum"] - 1.0) < 1e-6 for r in records.values()),
    "avg_entropy_gain": round(
        sum(r["scaled_entropy"] - r["raw_entropy"] for r in records.values()) / len(records), 3
    ),
    "interpretation_risk_rate": round(
        sum(not r["interpretation_support"] for r in records.values()) / len(records), 3
    ),
}
failed_cases = [case_id for case_id, r in records.items() if not r["route_hit"] or r["future_mass"] > 1e-6]

```

```

failed_gates = []
if metrics["route_hit_rate"] < 0.75:
    failed_gates.append("route_hit")
if metrics["future_leak_rate"] > 0.0:
    failed_gates.append("causal_mask")
if not metrics["row_sum_ok"]:
    failed_gates.append("softmax_row_sum")
if metrics["avg_entropy_gain"] <= 0.0:
    failed_gates.append("scale_stability")
if metrics["interpretation_risk_rate"] > 0.0:
    failed_gates.append("interpretation_boundary")

```

```

print(f"records={records}")
print(f"metrics={metrics}")
print(f"failed_cases={failed_cases}")
print(f"failed_gates={failed_gates}")
print(f"attention_routing_gate_pass={not failed_gates}")

```

运行结果应类似：

```

records={'pronoun_coref_scaled': {'top': 'animal', 'expected': 'animal', 'route_hit': True, 'future_mass': 0, 'raw_entropy': 0.012},
metrics={'route_hit_rate': 0.5, 'future_leak_rate': 0.0, 'row_sum_ok': True, 'avg_entropy_gain': 0.012, 'interpretation_risk_rate': 0.0},
failed_cases=['lost_middle_route_miss', 'attention_weight_not_causal_explanation']
failed_gates=['route_hit', 'interpretation_boundary']
attention_routing_gate_pass=False

```

这段 demo 的重点是：mask 和 softmax 可以完全正确，但路由目标仍可能失败；attention map 可以提供线索，但不能自动变成因果解释。面试中如果能讲清楚这两点，就比单纯背公式更接近工程真实问题。

2.17 面试题

题 1：Self-Attention 的本质是什么？

参考回答：

Self-Attention 的本质是内容相关的信息路由。每个 token 通过 Query 表示自己想找什么信息，其他 token 通过 Key 表示自己能被找到。

题 2：为什么 Q、K、V 要分开？

参考回答：

因为匹配逻辑和传递内容不一定相同。Q/K 用于判断当前位置应该关注哪些位置，V 表示被关注位置实际提供的信息。把它们分开后，

题 3：为什么 attention score 要除以 $\sqrt{d_k}$ ？

参考回答：

点积的方差会随维度增大而增大。如果不缩放，QK 的数值可能过大，softmax 会进入饱和区，变得过于尖锐，梯度也会变小。除以 $\sqrt{d_k}$ 可以缓解这个问题。

题 4：Attention weight 能解释模型行为吗？

参考回答：

只能有限解释。Attention weight 表示某一层某个 head 中，当前 token 对其他 token 的 Value 加权混合程度，有局部解释价值。但全局解释能力有限。

题 5：为什么 attention 可以看全局，但长上下文仍会 lost-in-the-middle？

参考回答：

因为可见不等于会稳定使用。标准 attention 让 token 理论上能访问上下文中所有位置，但模型训练分布、位置编码、注意力竞争、梯度消失等因素会导致长上下文信息丢失。

题 6: Self-Attention 和 RAG 中的检索有什么区别?

参考回答:

Self-Attention 是上下文内部的可微分检索, Q/K/V 都来自当前输入, 输出是对 Value 的加权混合。RAG 的检索通常是外部检索, 从

2.18 小练习

1. 手写一个单头 self-attention 的形状推导, 从 $[B, N, D]$ 到 $[B, N, D]$ 。
2. 用自己的话解释 Q、K、V, 不允许只说“查询、键、值”。
3. 构造一句包含代词指代的英文句子, 说明 attention 如何帮助代词找到 antecedent。
4. 解释为什么 QK^T 是 $N \times N$, 以及这个矩阵为什么既强大又昂贵。
5. 阅读 BERT 摘要, 解释 bidirectional attention 和 causal attention 的任务差异。
6. 阅读 Lost in the Middle 摘要, 解释“能放进上下文”和“能用好上下文”的区别。
7. 用 PyTorch 写一个 multi-head causal self-attention, 并打印每个中间张量形状。
8. 用本章的 0 依赖 demo 增加一个样本, 要求 mask 正确但 route miss, 并解释它对应真实长上下文中的什么问题。

2.19 本章总结

本章从信息路由角度解释了 Self-Attention。

核心结论:

1. Self-Attention 的本质是让每个 token 动态选择从哪些 token 读取信息。
2. Q 表示查询, K 表示匹配索引, V 表示被传递内容。
3. QK^T 形成 token-to-token 的动态路由图。
4. 除以 $\sqrt{d_k}$ 是为了控制点积分布尺度, 避免 softmax 饱和。
5. Attention weight 有局部解释价值, 但不是完整因果解释。
6. Causal attention 适合自回归生成, bidirectional attention 适合理解任务。
7. Self-Attention 像上下文内部的可微分检索, 但不同于外部 RAG 检索。
8. Attention 的全局可见性不保证长上下文鲁棒利用, lost-in-the-middle 是重要现象。
9. Attention 的 $N \times N$ 路由图同时带来强表达力和 $O(n^2)$ 计算/显存瓶颈。

下一章会进一步拆解 Q、K、V、Attention Head 与表示子空间, 重点讲多头为什么有用、head 之间是否真的学到不同功能, 以及 head 维度如何影响表达和效率。

第三章: Q、K、V、Attention Head 与表示子空间

0. 本讲资料边界与第二轮精修口径

本章第二轮精修以公开论文、公开技术报告和前序章节为资料边界: Transformer 原论文对 multi-head attention 的定义, BERT 及后续 attention head 分析对 head pattern 的观察, Michel 等人关于 head pruning 和冗余的实验结论, Abnar 和 Zuidema 对跨层 attention flow 的解释边界提醒, Transformer Circuits 对 QK / OV circuit 与 residual stream 的机制化拆解, Shazeer 的 MQA 论文和 Ainslie 等人的 GQA 论文对 K/V head 压缩和推理带宽瓶颈的解释, 以及近期公开研究对 attention head 角色跨随机重训稳定性的谨慎讨论。

本章只解释 Q/K/V、attention head、表示子空间、QK / OV 拆解、head 冗余、MHA / MQA / GQA 的形状和成本直觉; 不把某个可视化 attention map 写成完整因果解释, 不把 toy demo 的数值当成真实模型 benchmark, 也不推断闭源模型的内部 head 分工。

第二轮补强重点有三点:

1. 将 Q/K/V、multi-head attention、QK circuit、OV circuit、head dim 和 KV cache 成本改成稳定 MathJax 表达。
2. 明确 head 可解释性的边界: head pattern 是线索, 必须结合 ablation、activation patching、QK / OV 分解和任务行为变化。

3. 补一个 0 依赖 Python demo, 审计 QK 路由命中、OV 写入目标、head 冗余、解释风险, 以及 MHA / GQA / MQA 的 KV cache 粗略成本。

3.1 本章定位

上一章从信息路由角度解释了 Self-Attention。本章继续深入: Q、K、V 和 attention head 到底在表示什么? 为什么 Transformer 不只做一个 attention, 而要做 multi-head attention? 每个 head 是否真的学到了不同功能? 为什么有些 head 可以剪掉? 为什么 MQA、GQA 又要减少 K/V head?

这些问题是 Transformer 架构面试的高频追问。只会说 “多头可以关注不同位置” 还不够, 面试官可能继续问:

1. 多头为什么比单头表达力强?
2. QK 和 OV 可以如何分开理解?
3. head_dim 变大会怎样? head 数变多会怎样?
4. attention head 是否可解释?
5. 如果很多 head 冗余, 为什么还要多头?
6. MQA/GQA 为什么减少 K/V head, 而不是减少 Q head?

本章会把 attention head 看成 “在不同表示子空间中运行的信息路由器”。这比 “多个头看不同位置” 更准确。

3.2 资料来源和可信边界

本章主要参考以下公开资料:

1. Vaswani et al., 2017, Attention Is All You Need。原论文提出 multi-head attention, 说明多头允许模型在不同表示子空间、不同位置上联合关注信息。
2. Devlin et al., 2018/2019, BERT。BERT 使用多层多头双向 Transformer, 证明多头 self-attention 可形成强语言理解表示。
3. Clark et al., 2019, What Does BERT Look At?. 分析 BERT attention head, 发现一些 head 关注 delimiter、固定位置偏移、全句信息, 也有 head 与句法关系、共指关系相关。
4. Michel et al., 2019, Are Sixteen Heads Really Better than One?. 发现训练后的模型中大量 attention head 可在测试时剪掉而不显著影响性能, 说明多头存在冗余和层间差异。
5. Abnar and Zuidema, 2020, Quantifying Attention Flow in Transformers。指出跨层信息不断混合, raw attention weights 作为解释并不可靠, 提出 attention rollout/flow 等后验方法。
6. Shazeer, 2019, Fast Transformer Decoding: One Write-Head is All You Need。提出 MQA, 指出增量解码时加载大量 K/V tensor 带来内存带宽瓶颈, 共享 K/V 可加速推理。
7. Ainslie et al., 2023, GQA。提出 grouped-query attention, 在 MHA 和 MQA 之间折中, 用较少 K/V heads 获得接近 MHA 的质量和接近 MQA 的速度。
8. Transformer Circuits, A Mathematical Framework for Transformer Circuits。用 QK circuit 和 OV circuit 拆解 attention head 的 “读哪里” 和 “写什么” 两部分, 为理解 head 功能提供机制解释视角。
9. 近期关于 attention head role stability 的公开研究提醒: 即使两个模型结构和数据相同, 随机初始化和训练过程也可能让某些 head 功能位置发生变化, 因此不能把 “第几层第几个 head” 过度解释成稳定命名模块。

需要注意: attention head 的可解释性研究很有启发, 但不能把某个 head 的可视化图当成完整因果解释。本章会区分 “局部机制解释” “统计相关模式” 和 “最终模型行为”。

3.3 从单头到多头: 为什么一个 attention 不够

单头 attention 做的事情是:

在一个表示子空间里, 计算每个 token 应该从哪些 token 读取信息, 并把它们的 Value 加权混合。

但自然语言和代码中的关系不是单一类型。

同一个 token 可能同时需要:

1. 找前一个 token 的局部搭配。

2. 找主语和谓语关系。
3. 找代词指代实体。
4. 找括号、引号、代码块边界。
5. 找函数定义和调用。
6. 找长上下文中的证据句。
7. 找格式模板中的对应字段。

如果只有一个 attention head，它只能生成一张 attention map。

多头 attention 的直觉是：

让模型在多个不同子空间里同时做信息路由，再把结果拼接起来。

原论文中的表达是，多头允许模型联合关注不同位置、不同表示子空间的信息。

所以多头不是简单并行重复，而是把 hidden representation 切成多个 head，每个 head 用自己的 W_Q 、 W_K 、 W_V 学一个不同的信息路由和内容变换。

3.4 多头注意力的数学形式

输入张量可以写成：

$$X \in \mathbb{R}^{B \times N \times D}$$

其中 B 是 batch size， N 是序列长度， D 是 hidden size。第 h 个 head 的投影是：

$$Q_h = XW_h^Q, \quad K_h = XW_h^K, \quad V_h = XW_h^V$$

并且：

$$Q_h, K_h, V_h \in \mathbb{R}^{B \times N \times d_h}$$

第 h 个 head 的注意力权重和输出可以写成：

$$A_h = \text{softmax}\left(\frac{Q_h K_h^{\text{top}}}{\sqrt{d_h}} + M\right)$$

$$O_h = A_h V_h$$

其中 M 是可选 mask，例如 causal mask。所有 head 拼接后再通过输出投影混回主 hidden space：

$$\text{MHA}(X) = \text{Concat}(O_1, \dots, O_H)W^O$$

如果不考虑一些特殊变体，通常有：

$$D = H \cdot d_h$$

这说明 multi-head attention 至少有三层含义：

1. 每个 head 有自己的 $Q/K/V$ 投影。
2. 每个 head 有自己的 attention map。
3. 所有 head 输出再通过 W_O 混合回主 hidden space。

如果只看 attention weights，会漏掉 Value 和 Output projection 的影响。一个 head 不只是“看哪里”，还包括“把看到的信息写成什么”。

3.5 QK Circuit 和 OV Circuit

一个 attention head 可以拆成两部分理解。

第一部分是 QK circuit：

QK：决定从哪里读。

它由 W_Q 和 W_K 决定，影响 attention score：

$$s_{ij}^{(h)} = (x_i W_h^Q)(x_j W_h^K)^{\text{top}}$$

第二部分是 OV circuit：

OV: 决定读到的信息如何写入 residual stream。

它由 W_V 和 W_O 决定:

$$c_{ij}^{(h)} = a_{ij}^{(h)} x_j W_h^V W_h^O$$

其中 a_{ij} 是第 h 个 head 中位置 i 对位置 j 的 attention weight, W_h^O 可以理解为输出投影中对应第 h 个 head 的那一块。

这个拆法非常重要。

很多初学者只看 attention map, 以为 head 的作用就是 “关注某个 token”。但真正的 head 功能包括:

1. 通过 QK 选择源 token。
2. 通过 OV 把源 token 的某些特征写到目标位置。

举例: 一个 head 可能在 QK 上关注前一个 token, 但 OV 写入的是 “前一个 token 的词性或括号状态”。另一个 head 也关注前一个 token, 但写入的是完全不同的信息。

所以只看 attention weight 不足以完整理解 head。

3.6 表示子空间: Head 到底分工在哪里

为什么说 head 工作在不同表示子空间?

因为每个 head 的 Q/K/V 投影矩阵不同。它们把同一个 x 投影到不同低维空间:

$x \rightarrow Q_h/K_h/V_h$ in head h subspace

这些子空间可以学不同的匹配规则:

1. 位置相关规则。
2. 语法依赖规则。
3. 共指规则。
4. 分隔符规则。
5. 局部 n -gram 规则。
6. 长程引用规则。
7. 格式复制规则。

Clark 等人对 BERT 的分析发现, 一些 attention heads 会关注 delimiter tokens、特定位置偏移, 或者全句信息; 也有一些 head 与句法关系、共指关系相关。

这说明 head 确实可能形成某些可观察模式。

但要小心:

head 有模式, 不等于 head 是稳定、唯一、不可替代的语义模块。

不同模型、不同层、不同随机种子、不同训练阶段, head 的分工都可能变化。

3.7 Head 数和 Head Dim 的 Trade-off

假设 hidden size 固定为 D , head 数是 H , 每个 head 维度是:

$$d_h = \frac{D}{H}$$

增加 head 数会带来:

1. 更多独立 attention maps。
2. 更多不同子空间路由。
3. 每个 head 的维度更小。

减少 head 数会带来:

1. 每个 head 维度更大。
2. attention maps 更少。

3. 可能减少某些并行路由模式。

所以 head 数不是越多越好。

如果 head 太少，模型可能缺少足够的并行关系建模能力。

如果 head 太多，每个 head 维度太小，单个 head 的表达能力可能受限，而且推理时 K/V cache 和内存访问成本也会上升。

现代 LLM 中，head 数、hidden size、KV head 数、tensor parallel 切分方式都会一起设计。它不是纯理论超参数，而是架构和系统共同约束的结果。

3.8 为什么有些 Head 可以剪掉

Michel 等人的研究发现，训练好的多头注意力模型中，很多 head 可以在测试时移除而不显著影响性能。有些层甚至可以减少到单个 head。

这似乎和“多头很重要”矛盾。

其实不矛盾。

原因可能包括：

1. 多头在训练时提供优化路径，但训练后部分 head 冗余。
2. 不同 head 之间功能重叠。
3. 某些任务只需要部分关系。
4. 残差连接和 FFN 可以补偿部分 head 删除。
5. 重要 head 分布不均，有些层更依赖多头。

这给我们的启发是：

Multi-head attention 是一种容量和优化机制，不代表每个 head 都同等重要。

面试中如果被问“多头是否都必要”，不要简单回答“必要”。更好的回答是：多头整体提高表达和优化能力，但具体训练后可能存在冗余，可用于剪枝、压缩和解释分析。

3.9 Head 的可解释性：能看，但不能迷信

Attention head 可视化很直观。你可以画出 attention map，看某个 head 是否关注：

1. 前一个 token。
2. 标点符号。
3. [CLS] 或分隔符。
4. 主谓关系。
5. 代词和先行词。
6. 括号匹配。

这很有帮助，但有三个风险。

第一，raw attention 只是单层单 head 的权重。

第二，跨层后信息会混合。Abnar 和 Zuidema 指出，Transformer 中不同 token 的信息跨层不断混合，raw attention weights 作为解释探针并不可靠。

第三，最终行为还受 OV、FFN、残差、LayerNorm、logit lens 等影响。

因此，解释 head 时更严谨的做法是结合：

1. Attention pattern。
2. Head ablation。
3. Activation patching。
4. OV/QK 分解。
5. Input gradient 或 token ablation。
6. 任务行为变化。

简单说：

Attention map 是线索，不是判决书。

3.10 为什么 MQA/GQA 要动 K/V Head

标准 MHA 中，每个 Q head 都有自己的 K/V head：

$\text{num_q_heads} = \text{num_kv_heads} = H$

推理时，decoder-only 模型需要为每层每个历史 token 缓存 K/V。

如果 batch size 是 B，层数是 L，上下文长度是 T，KV head 数是 H_{kv} ，head dim 是 d_h ，每个元素占 b 字节，那么 KV cache 粗略显存可以写成：

$M_{\text{kv}} \approx 2BLTH_{\text{kv}}d_hb$

前面的 2 来自同时缓存 Key 和 Value。

这就是长上下文推理的核心成本之一。

MQA 的做法是：

多个 Q heads 共享一组 K/V。

也就是：

$\text{num_q_heads} = H$
 $\text{num_kv_heads} = 1$

这样 KV cache 和内存带宽大幅下降，增量解码更快。

问题是 MQA 可能带来质量下降，因为所有 query heads 共享同一套 K/V 表示，表达能力受限。

GQA 是折中：

$\text{num_q_heads} = H$
 $1 < \text{num_kv_heads} < H$

每组 Q heads 共享一个 K/V head。GQA 试图获得接近 MHA 的质量和接近 MQA 的速度。

这解释了为什么现代 LLM 经常从 MHA 走向 GQA：

不是因为多头不重要，而是因为推理阶段 K/V cache 太贵。

3.11 Q Head 和 KV Head 的分离直觉

为什么减少 K/V head，而不是减少 Q head？

因为 Q head 决定“当前 token 用多少种不同方式发起查询”。保留多个 Q heads，模型仍然可以从不同查询子空间发问。

K/V head 决定“历史 token 暴露多少套索引和内容表示”。减少 K/V heads 可以压缩缓存。

GQA 的直觉是：

让多个 query 子空间共享一部分历史索引和内容表示，在表达能力和缓存成本之间折中。

可以类比为：

很多人可以问不同问题，但共享同一个简化档案库。

MHA 是每类问题都有独立档案库。MQA 是所有问题共享一个档案库。GQA 是几组问题共享几套档案库。

3.12 代码：MHA 与 GQA 的形状差异

标准 MHA:

Q: [B, H, N, d_h]
K: [B, H, N, d_h]
V: [B, H, N, d_h]

GQA:

Q: [B, H_q, N, d_h]
K: [B, H_kv, N, d_h]
V: [B, H_kv, N, d_h]

其中:

$H_q > H_{\mathrm{kv}}, \text{且 } H_q \bmod H_{\mathrm{kv}} = 0$

计算 attention 时, 需要把 K/V 按组映射到 Q heads。

简化代码:

```
import torch
```

```
def repeat_kv(x: torch.Tensor, n_rep: int) -> torch.Tensor:
    # x: [batch, num_kv_heads, seq_len, head_dim]
    if n_rep == 1:
        return x
    bsz, num_kv_heads, seq_len, head_dim = x.shape
    x = x[:, :, None, :, :]
    x = x.expand(bsz, num_kv_heads, n_rep, seq_len, head_dim)
    return x.reshape(bsz, num_kv_heads * n_rep, seq_len, head_dim)
```

如果:

```
num_q_heads = 32
num_kv_heads = 8
```

那么:

```
n_rep = 32 / 8 = 4
```

每个 K/V head 会服务 4 个 Q heads。

这个例子说明: MHA/MQA/GQA 的差异不是 attention 公式变了, 而是 Q heads 和 K/V heads 的数量关系变了。

3.13 Head 和 Residual Stream

Transformer Circuits 的一个重要视角是 residual stream。

可以把每层的模块看成在 residual stream 上读写信息:

$$r_{\ell+1} = r_{\ell} + \sum_{h=1}^H O_h(r_{\ell})$$

这里省略了 normalization、输出投影、FFN 和具体层结构, 只强调 attention heads 会从 residual stream 读取信息并把变换后的内容写回。

一个 head 做两件事:

1. 从 residual stream 中读 Q/K/V。
2. 通过 OV 写回 residual stream。

这使得不同 head、不同层可以组合成复杂电路。例如:

1. 早层 head 复制前文 token 信息。
2. 中层 head 组织语法和实体关系。
3. 后层 head 把相关信息送到预测位置。

这种说法是理解机制的有力工具，但不要过度简化成“第几层第几个 head 就固定负责某个任务”。真实大模型有大量 superposition、冗余和分布式表示。

3.14 表示子空间和 Superposition

所谓表示子空间，不是说每个 head 都有一个干净、人工可命名的语义空间。

更现实的情况是：

1. 模型隐藏维度有限。
2. 任务特征很多。
3. 不同特征可能叠加在同一组神经元或子空间中。
4. Head 之间可能共享和重叠功能。
5. 表示会随上下文动态变化。

这就是 mechanistic interpretability 中常说的 superposition 问题。

因此，“head 子空间”是一个有用抽象，但不是严格的人工分区。

面试中可以这样表达：

Multi-head attention 给模型提供了多个低维投影和路由通道，每个 head 可以学习不同类型的匹配和信息写入。但这些功能是分布式的。

3.15 面试中如何解释 Head 的作用

一个比较完整的回答可以分四层：

第一层，公式层：

每个 head 有独立 W_Q 、 W_K 、 W_V ，计算自己的 attention map 和 value mixture。

第二层，表达层：

多个 head 让模型在不同表示子空间中并行建模不同关系。

第三层，机制层：

QK circuit 决定读哪里，OV circuit 决定写什么。

第四层，工程层：

head 数影响表达能力、KV cache、内存带宽和推理吞吐。MHA、MQA、GQA 是质量和推理成本之间的 trade-off。

这样回答比“多个头关注不同位置”更有深度。

3.16 常见误区

误区一：每个 head 都有清晰语义。

更准确：部分 head 可观察到语法、位置、delimiter 等模式，但很多 head 冗余或功能重叠。

误区二：剪掉 head 不影响性能，所以多头没用。

更准确：多头可能对训练、优化和鲁棒性有帮助，训练后存在冗余不等于训练时没用。

误区三：MHA 一定比 GQA 好。

更准确：MHA 表达更完整，但 GQA 在推理 KV cache 和速度上更优，现代 LLM 常需要系统 trade-off。

误区四：Q/K/V 分开只是实现细节。

更准确：Q/K/V 分开是匹配和内容传递解耦的关键设计。

误区五：Attention head 只负责读信息。

更准确：head 既通过 QK 决定读哪里，也通过 OV 决定写什么。

3.17 QKV 与多头子空间审计指标

把一个 head 只解释成“看哪里”容易过度简化。第二轮精修后，本章建议用下面这组指标把 QK 路由、OV 写入、head 冗余和 KV cache 成本一起讲清楚。

一个 head 审计样本可以写成：

$$e_i = (x_i, h_i, t_i, f_i, a_i, o_i, z_i)$$

其中 x_i 是输入上下文， h_i 是 head 编号， t_i 是期望被读取的 token， f_i 是期望写入的目标特征， a_i 是 attention weights， o_i 是该 head 的输出向量， z_i 是人工或任务层面的验证结果。

QK 路由命中率：

$$C_{\{\mathrm{qk}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\arg\max_j a_{ij} = t_i]$$

OV 写入支持率：

$$C_{\{\mathrm{ov}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\mathrm{top_feature}(o_i) = f_i]$$

head 冗余率可以粗略定义为：

$$R_{\{\mathrm{red}\}} = 1 - \frac{|\{\mathrm{unique}(\mathrm{signature}(h))\}|}{H}$$

其中 $\mathrm{signature}(h)$ 可以是一个简化的 head 功能签名，例如“主要读取哪个 token + 主要写入哪个特征”。真实模型里不能只靠这个定义做剪枝，但它适合解释“为什么有些 head 看起来重复”。

GQA 的 KV head 压缩比可以写成：

$$\rho_{\{\mathrm{kv}\}} = \frac{H_q}{H_{\{\mathrm{kv}\}}}$$

KV cache 节省率：

$$S_{\{\mathrm{kv}\}} = 1 - \frac{H_{\{\mathrm{kv}\}}}{H_q}$$

一个简化审计门禁可以写成：

$$G_{\{\mathrm{head}\}} = \mathbb{1}[C_{\{\mathrm{qk}\}} \geq \tau_q \land C_{\{\mathrm{ov}\}} \geq \tau_o \land R_{\{\mathrm{red}\}} \leq \tau_r \land \rho_{\{\mathrm{kv}\}} \geq \tau_{\rho} \land S_{\{\mathrm{kv}\}} \geq \tau_s]$$

这组公式的重点是面试表达：多头不是只有 attention map，还要同时看 QK 是否读对、OV 是否写对、head 是否冗余，以及减少 K/V head 是否真的带来缓存收益。

3.17.1 最小可运行 QKV / Head 子空间审计 demo

下面的 demo 不依赖 PyTorch，只用 Python 标准库模拟 5 个 attention heads。它展示三件事：QK 路由命中不等于 OV 一定写对；两个 head 可能形成相似功能签名；从 MHA 到 GQA / MQA 时 KV cache 成本如何按 `num_kv_heads` 下降。

```
import math
```

```
tokens = ["prev_token", "subject", "delimiter", "evidence", "distractor"]
feature_names = ["local_copy", "entity_state", "format_boundary", "evidence_fact"]
```

```
key_bank = {
    "prev_token": [1.0, 0.1, 0.0, 0.0],
    "subject": [0.1, 1.0, 0.0, 0.0],
    "delimiter": [0.0, 0.1, 1.0, 0.0],
    "evidence": [0.1, 0.4, 0.0, 1.0],
    "distractor": [0.7, 0.7, 0.0, 0.1],
}
```

```

heads = [
    {"name": "previous_token_head", "query": [1.0, 0.0, 0.0, 0.0], "target": "prev_token", "feature": "local_copy"},
    {"name": "subject_coref_head", "query": [0.0, 1.0, 0.0, 0.0], "target": "subject", "feature": "entity_state"},
    {"name": "delimiter_head", "query": [0.0, 0.0, 1.0, 0.0], "target": "delimiter", "feature": "format_boundary"},
    {"name": "delimiter_copy_head", "query": [0.0, 0.0, 0.9, 0.0], "target": "delimiter", "feature": "format_boundary"},
    {"name": "misleading_attention_head", "query": [0.7, 0.7, 0.0, 0.0], "target": "evidence", "feature": "evidence_t"}
]

value_features = {
    "prev_token": [1.0, 0.1, 0.0, 0.0],
    "subject": [0.0, 1.0, 0.1, 0.0],
    "delimiter": [0.0, 0.0, 1.0, 0.0],
    "evidence": [0.0, 0.2, 0.0, 1.0],
    "distractor": [0.3, 0.3, 0.0, 0.1],
}

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def softmax(scores):
    m = max(scores)
    exps = [math.exp(s - m) for s in scores]
    total = sum(exps)
    return [x / total for x in exps]

def weighted_sum(weights, rows):
    return [sum(w * row[i] for w, row in zip(weights, rows)) for i in range(len(rows[0]))]

records = {}
for head in heads:
    scores = [dot(head["query"], key_bank[token]) / math.sqrt(len(head["query"])) for token in tokens]
    weights = softmax(scores)
    top_token = tokens[max(range(len(tokens)), key=lambda i: weights[i])]
    output = weighted_sum(weights, [value_features[token] for token in tokens])
    top_feature = feature_names[max(range(len(feature_names)), key=lambda i: output[i])]
    signature = (top_token, top_feature)
    records[head["name"]] = {
        "top_token": top_token,
        "target": head["target"],
        "qk_hit": top_token == head["target"],
        "top_feature": top_feature,
        "expected_feature": head["feature"],
        "ov_hit": top_feature == head["feature"],
        "signature": signature,
    }

unique_signatures = {tuple(record["signature"]) for record in records.values()}
metrics = {
    "qk_route_hit": round(sum(r["qk_hit"] for r in records.values()) / len(records), 3),
    "ov_write_hit": round(sum(r["ov_hit"] for r in records.values()) / len(records), 3),
    "head_redundancy": round(1 - len(unique_signatures) / len(records), 3),
}

```

```

    "interpretation_risk": round(
        sum(r["qk_hit"] and not r["ov_hit"] or not r["qk_hit"] for r in records.values()) / len(records), 3
    ),
}

```

```

def kv_cache_mib(batch, layers, seq_len, kv_heads, head_dim, bytes_per_elem):
    bytes_used = 2 * batch * layers * seq_len * kv_heads * head_dim * bytes_per_elem
    return round(bytes_used / 1024**2, 1)

```

```

kv_cache = {
    "mha_32kv": kv_cache_mib(1, 32, 4096, 32, 128, 2),
    "gqa_8kv": kv_cache_mib(1, 32, 4096, 8, 128, 2),
    "mqa_1kv": kv_cache_mib(1, 32, 4096, 1, 128, 2),
}

```

```

failed_gates = []
if metrics["qk_route_hit"] < 0.8:
    failed_gates.append("qk_route")
if metrics["ov_write_hit"] < 0.8:
    failed_gates.append("ov_write")
if metrics["head_redundancy"] > 0.25:
    failed_gates.append("head_redundancy")
if metrics["interpretation_risk"] > 0.2:
    failed_gates.append("interpretation_boundary")
if kv_cache["gqa_8kv"] >= kv_cache["mha_32kv"]:
    failed_gates.append("kv_cache_saving")

```

```

print(f"records={records}")
print(f"metrics={metrics}")
print(f"kv_cache_mib={kv_cache}")
print(f"failed_gates={failed_gates}")
print(f"qkv_head_gate_pass={not failed_gates}")

```

运行结果应类似：

```

records={'previous_token_head': {'top_token': 'prev_token', 'target': 'prev_token', 'qk_hit': True, 'top_feature': 'local_token'}}
metrics={'qk_route_hit': 0.8, 'ov_write_hit': 0.8, 'head_redundancy': 0.2, 'interpretation_risk': 0.2}
kv_cache_mib={'mha_32kv': 2048.0, 'gqa_8kv': 512.0, 'mqa_1kv': 64.0}
failed_gates=[]
qkv_head_gate_pass=True

```

这段 demo 的重点不是模拟真实 attention head，而是把面试里的四个关键点连起来：QK 决定读哪里，OV 决定写什么，多头可能存在冗余，GQA / MQA 的缓存收益来自减少 H_kv。

3.18 面试题

题 1：Multi-head attention 为什么有用？

参考回答：

Multi-head attention 让模型在多个表示子空间中并行做信息路由。每个 head 有独立的 Q/K/V 投影，可以学习不同的匹配规则和内部表示。

题 2：QK circuit 和 OV circuit 分别是什么？

参考回答：

QK circuit 决定 attention head 从哪里读信息，也就是 Query 和 Key 的匹配产生 attention pattern。OV circuit 决定读到的信息如何写进输出。

题 3：为什么有些 attention head 可以剪掉？

参考回答：

训练后的多头注意力里可能存在冗余。不同 head 的功能可能重叠，某些任务只依赖部分 head，残差和 FFN 也能补偿一部分删除带来的影响。

题 4：MHA、MQA、GQA 的核心区别是什么？

参考回答：

MHA 是每个 query head 都有自己的 key/value head，表达能力强但 KV cache 大。MQA 是多个 query heads 共享一组 key/value，大幅减少 KV cache 大小。

题 5：为什么减少 K/V head 能加速推理？

参考回答：

Decoder-only 增量推理时，每生成一个 token 都要读取历史 token 的 K/V cache。KV cache 大小和层数、序列长度、KV head 数、head dim 成正比。

题 6：Attention head 是否可解释？

参考回答：

可以有限解释，但不能迷信。可视化 attention map 能发现一些 head 关注 delimiter、固定位置、语法关系或共指关系。但 raw attention 权重本身很难解释。

3.19 小练习

1. 手写 MHA 中 Q、K、V 的张量形状，并解释每一维含义。
2. 用自己的话解释 “QK 决定读哪里，OV 决定写什么”。
3. 画一个 4-head attention 的示意图，给每个 head 假设一种可能功能。
4. 阅读 What Does BERT Look At? 摘要，列出 BERT head 可能学到的几类 pattern。
5. 阅读 Are Sixteen Heads Really Better than One? 摘要，解释 head pruning 说明了什么、没有说明什么。
6. 给出 MHA、MQA、GQA 的 KV cache 大小对比公式。
7. 用 PyTorch 实现一个 repeat_kv 函数，并解释它在 GQA 中的作用。
8. 运行本章 QKV / Head 子空间审计 demo，解释为什么 misleading_attention_head 同时暴露 QK 路由和 OV 写入风险。

3.20 本章总结

本章深入讲了 Q、K、V、attention head 和表示子空间。

核心结论：

1. Q/K/V 分开是为了把匹配逻辑和内容传递解耦。
2. Multi-head attention 让模型在多个表示子空间中并行做信息路由。
3. 一个 head 不只是 attention map，还包括 QK circuit 和 OV circuit。
4. QK 决定读哪里，OV 决定写什么。
5. Attention head 可能学到语法、位置、共指、delimiter 等 pattern，但也可能冗余。
6. Head 可解释性需要谨慎，raw attention 不能直接等同于因果解释。
7. Head 数和 head dim 是表达能力、优化、KV cache 和推理吞吐之间的 trade-off。
8. MQA/GQA 减少 K/V heads，是为了解决增量推理中的 KV cache 和内存带宽瓶颈。
9. 现代 LLM 的 attention 设计已经从纯模型结构问题，变成模型架构和 serving 系统共同优化问题。

下一章会系统比较 MHA、MQA、GQA、MLA 的演进与 trade-off，重点讲为什么现代大模型越来越重视 KV cache 压缩和推理效率。

第四章：MHA、MQA、GQA、MLA 的演进与 Trade-off

0. 本讲资料边界与第二轮精修口径

本章第二轮精修以公开论文、技术报告和前序章节为资料边界：Transformer 原论文对 MHA 的定义，Shazeer 的 MQA 论文对增量解码内存带宽瓶颈的说明，Ainslie 等人的 GQA 论文对 MHA / MQA 折中和 uptraining 的讨论，Mistral 7B 技术报告中 GQA 与 Sliding Window Attention 的公开用法，DeepSeek-V2 / V3 技术报告中 MLA 对 KV cache 的低秩联合压缩设计，vLLM / PagedAttention 论文对 serving 侧 KV cache 分页管理的说明，以及 FlashAttention-2 对 attention kernel IO 和并行划分的优化边界。

本章只讨论公开可验证的 attention 架构与 serving 成本关系；不推断闭源模型的未公开实现，不把 DeepSeek-V2 报告中的具体节省比例机械推广到所有模型，也不把 toy demo 的选型分数当成真实 benchmark。

第二轮补强重点有三点：

1. 将 KV cache 显存、KV head 比例、GQA 特例、MLA latent cache 和架构门禁改成稳定 MathJax 表达。
2. 明确 FlashAttention、PagedAttention、MQA / GQA / MLA 分别位于 kernel、serving memory manager 和 model architecture 三个层面。
3. 补一个 0 依赖 Python demo，审计 MHA / GQA / MQA / MLA 的 cache MiB、相对节省率、quality gate、maturity gate 和 serving 选型。

4.1 本章定位

前两章已经讲清楚 Self-Attention、Q/K/V、attention head 和表示子空间。本章进入现代大模型注意力结构的工程演进：MHA、MQA、GQA、MLA。

如果只从训练论文看，Multi-Head Attention 已经足够优雅。但到了大模型推理系统里，一个更现实的问题出现了：

生成每个 token 时，模型都要读取历史 token 的 K/V cache。

上下文越长、batch 越大、层数越多、KV head 越多，KV cache 越大，内存带宽和显存压力越高。于是注意力结构的优化目标从“表达能力更强”逐渐变成：

表达能力、KV cache、吞吐、延迟、显存、长上下文之间如何折中？

学完本章，你应该能回答：

1. MHA、MQA、GQA、MLA 分别是什么。
2. 为什么 MHA 表达强但推理贵。
3. 为什么 MQA 能显著降低 KV cache，但可能损失质量。
4. 为什么 GQA 成为现代 LLM 常用折中方案。
5. MLA 为什么进一步把 K/V cache 压缩成 latent cache。
6. FlashAttention、PagedAttention 和 MQA/GQA/MLA 分别优化什么层面。
7. 面试中如何系统解释 attention 架构演进。

4.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need。提出 Multi-Head Attention，让模型在多个表示子空间联合关注信息。
2. Shazeer, 2019, Fast Transformer Decoding: One Write-Head is All You Need。提出 Multi-Query Attention，指出增量解码时加载大量 K/V tensor 带来内存带宽瓶颈，共享 K/V 可显著加速。
3. Ainslie et al., 2023, GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints。提出 Grouped-Query Attention，在 MHA 和 MQA 之间折中，质量接近 MHA、速度接近 MQA。
4. Jiang et al., 2023, Mistral 7B。公开说明 Mistral 7B 使用 GQA 加速推理，并结合 Sliding Window Attention 降低长序列成本。

5. DeepSeek-AI, 2024, DeepSeek-V2。提出 Multi-head Latent Attention, 通过把 KV cache 显著压缩到 latent vector 中提升推理效率。
6. DeepSeek-AI, 2024/2025, DeepSeek-V3 Technical Report。继续采用 MLA 和 DeepSeekMoE, 说明 MLA 已被大规模 MoE 模型验证。
7. Kwon et al., 2023, PagedAttention/vLLM。从 serving 系统角度解决 KV cache 动态增长、碎片和重复的问题。
8. Dao, 2023, FlashAttention-2。从 GPU kernel 和 IO 角度优化 attention 计算, 强调 attention 是长序列扩展主要瓶颈。

需要注意: 不同报告对 MLA 的实现细节公开程度不同, 本章只基于公开论文和技术报告解释核心思想, 不臆测闭源实现。

4.3 为什么注意力结构开始围绕 KV Cache 演进

训练时, Transformer 可以一次并行处理整个序列。

推理时, decoder-only LLM 是逐 token 生成:

prefix tokens $\rightarrow$ generate token $t \rightarrow$ append token $t \rightarrow$ generate token $t+1$

为了避免每一步重新计算所有历史 token 的 K/V, 系统会缓存每层每个历史 token 的 K/V, 这就是 KV cache。

对一个 decoder-only 模型, 如果 batch size 是 B , 层数是 L , 上下文长度是 T , KV head 数是 H_{kv} , 每个 head 的维度是 d_h , 每个元素占 b 字节, KV cache 粗略显存可以写成:

$$M_{\{\mathrm{kv}\}} \approx 2BLTH_{\{\mathrm{kv}\}}d_hb$$

前面的 2 来自 K 和 V 两份缓存。

这说明几个事实:

1. 上下文长度翻倍, KV cache 约翻倍。
2. batch size 翻倍, KV cache 约翻倍。
3. KV head 数越多, cache 越大。
4. 长上下文 serving 时, 显存经常被 KV cache 吃掉, 而不只是模型权重。
5. 解码每个新 token 时, 需要反复读取历史 K/V, 内存带宽会成为瓶颈。

因此, 现代注意力结构演进的主线之一就是:

如何少存 K/V, 少读 K/V, 同时尽量不损失模型质量?

4.4 MHA: 表达完整, 但 KV Cache 昂贵

标准 Multi-Head Attention 中:

$$H_q = H_{\{\mathrm{kv}\}} = H$$

每个 query head 有自己对应的 key head 和 value head。

张量形状通常是:

Q: $[B, H, N, d_h]$

K: $[B, H, N, d_h]$

V: $[B, H, N, d_h]$

MHA 的优势:

1. 每个 head 有独立 Q/K/V 子空间。
2. 表达能力完整。
3. 训练和理论最标准。
4. 原始 Transformer、BERT、早期 GPT 类模型都大量使用。

MHA 的代价:

1. 每个 head 都要缓存 K/V。

2. 长上下文时 cache 显存高。
3. 解码时内存带宽压力大。
4. batch 做大时 KV cache 更容易成为瓶颈。

如果模型有 32 个 heads, MHA 就有 32 个 K heads 和 32 个 V heads。对于长上下文和高并发 serving, 这很贵。所以 MHA 是“表达优先”的设计, 而不是“推理成本优先”的设计。

4.5 MQA: 一个 K/V Head 服务所有 Query Heads

Multi-Query Attention 的核心思想很直接:

保留多个 query heads, 但让所有 query heads 共享同一组 K/V。

也就是:

$$H_q = H, \quad H_{\mathrm{kv}} = 1$$

形状变成:

Q: [B, H, N, d_h]

K: [B, 1, N, d_h]

V: [B, 1, N, d_h]

这样 KV cache 直接从 H 份降到 1 份。

MQA 的优势:

1. KV cache 大幅减少。
2. 解码时内存带宽需求降低。
3. 增量推理速度明显提升。
4. 长上下文和大 batch 更友好。

MQA 的代价:

1. 所有 Q heads 共享同一套 K/V 表示。
2. 表达能力可能下降。
3. 某些任务或模型规模下质量可能受影响。
4. 从已有 MHA checkpoint 直接改成 MQA 需要额外处理或再训练。

Shazeer 的论文标题 “One Write-Head is All You Need” 很形象: 推理慢的关键不是 query 数, 而是重复读取大量 K/V。把 K/V 合并, 可以显著降低带宽压力。

4.6 GQA: MHA 和 MQA 之间的折中

Grouped-Query Attention 是更实用的折中。

它让一组 query heads 共享一个 K/V head:

$$H_q = H, \quad 1 < H_{\mathrm{kv}} < H$$

例如:

num_q_heads = 32

num_kv_heads = 8

那么每个 K/V head 服务 4 个 query heads。

GQA 的直觉:

不要像 MHA 那样每个 query head 都有独立 K/V, 也不要像 MQA 那样所有 query heads 只共享一套 K/V, 而是分组共享。

GQA 的优势:

1. KV cache 明显小于 MHA。
2. 表达能力通常好于 MQA。

3. 推理速度接近 MQA。
4. 可以从 MHA checkpoint uptraining 得到。
5. 已被 Mistral 7B 等公开模型采用。

GQA 的代价：

1. 仍然比 MQA cache 大。
2. 质量和速度依赖 `num_kv_heads` 选择。
3. 实现中需要处理 Q heads 到 KV heads 的映射。

GQA 是现代 LLM 很常见的工程选择，因为它把质量损失和推理收益平衡得更好。

4.7 MHA、MQA、GQA 的统一视角

可以把三者看成同一个轴上的不同点：

结构	Q heads	KV heads	KV cache	表达能力	推理效率
MHA	H	H	最大	最完整	最低
GQA	H	G	中等	接近 MHA	接近 MQA
MQA	H	1	最小	可能下降	最高

其中：

$$1 \leq H_{\mathrm{kv}} \leq H_q$$

所以：

$$\mathrm{MHA}: H_{\mathrm{kv}}=H_q, \quad \mathrm{MQA}: H_{\mathrm{kv}}=1$$

这也是为什么 GQA 可以叫 generalized multi-query attention。

面试中可以这样概括：

MHA、MQA、GQA 的核心差异不是 query heads，而是 KV heads 数量。减少 KV heads 可以降低 KV cache 和内存带宽，但会牺牲一部分

4.8 MLA：从减少 KV Head 到压缩 KV 表示

DeepSeek-V2 提出的 Multi-head Latent Attention 进一步推进了这个方向。

GQA/MQA 的思路是：

减少 KV head 数。

MLA 的思路更进一步：

不要直接缓存完整 K/V，而是把 K/V 相关信息压缩成 latent cache。

DeepSeek-V2 报告明确说，MLA 通过把 KV cache 显著压缩到 latent vector 中，保证高效推理，并报告 KV cache 减少 93.3%、最大生成吞吐提升 5.76 倍。

从直觉上看，MLA 想解决的问题是：

1. MHA cache 太大。
2. MQA/GQA 靠共享 K/V 减 cache，但可能限制表达。
3. 能否保留多头 attention 的表达，同时用更紧凑的 latent 表示存历史信息？

可以抽象为：

$$c_t^{\mathrm{kv}} = x_t W_d^{\mathrm{kv}}$$

$$k_t = c_t^{\mathrm{kv}} W_u^k, \quad v_t = c_t^{\mathrm{kv}} W_u^v$$

MLA 的价值在于把注意力结构和 serving 成本更深地绑定起来。它不是单纯换一个 head 数，而是在 K/V 表示本身上做压缩设计。

4.9 MLA 和 MoE 的关系

DeepSeek-V2/V3 都是 MoE 模型，同时使用 MLA。

这不是偶然。

MoE 的特点是：

- 1. 总参数量很大。
- 2. 每个 token 只激活部分专家。
- 3. 训练计算可以相对经济。
- 4. serving 时仍然面临显存、通信、batch、KV cache 等压力。

如果一个 MoE 模型支持长上下文，KV cache 仍然会随层数、长度、batch 增长。即使每 token 激活参数较少，KV cache 也不会自动消失。

因此 MLA 对 MoE 很重要：

MoE 降低每 token 计算成本，MLA 降低长上下文推理时的 KV cache 成本。

这说明现代大模型架构优化通常不是单点优化，而是组合拳：

MoE + MLA + 高效并行 + FP8/量化 + serving 系统优化

4.10 FlashAttention、PagedAttention 和 MQA/GQA/MLA 的区别

这些名字容易混在一起，但它们优化层面不同。

FlashAttention 优化的是 attention kernel。

它不改变 attention 数学结果，而是利用 GPU memory hierarchy，减少 HBM 读写，提高计算效率。FlashAttention-2 进一步优化并行和 work partitioning。

PagedAttention 优化的是 KV cache 管理。

它借鉴操作系统分页思想，减少 KV cache 的内存碎片和重复，提升 serving batch 能力。

MQA/GQA/MLA 优化的是模型架构中的 K/V 表示。

它们改变 KV heads 或 KV cache 表示，从源头减少需要存和读的 K/V。

可以总结为：

方法	优化层面	是否改变模型结构	主要目标
FlashAttention	kernel/IO	否	更快更省显存地算 attention
PagedAttention	serving memory manager	否	更高效管理 KV cache
MQA	attention 架构	是	用 1 个 KV head 降低 cache
GQA	attention 架构	是	分组 KV head，平衡质量和效率
MLA	attention 架构	是	用 latent cache 压缩 K/V 表示

面试中一定要区分：FlashAttention 不是 MQA，PagedAttention 也不是 GQA。

4.11 KV Cache 粗略计算

假设：

L = 32
B = 16
T = 8192
d_h = 128
b = 2

如果 MHA 有 32 个 KV heads:

$$M_{\mathrm{mha}} \approx 2 \times 16 \times 32 \times 8192 \times 32 \times 128 \times 2$$

这里第一个 2 是 K/V 两份，最后的 2 是 bf16 bytes。

如果 GQA 改成 8 个 KV heads，cache 约变成 MHA 的:

$$\frac{8}{32} = \frac{1}{4}$$

如果 MQA 改成 1 个 KV head，cache 约变成 MHA 的:

$$\frac{1}{32}$$

这就是为什么 KV head 数对 serving 成本影响巨大。

当然，实际系统还要考虑 padding、碎片、block size、并行切分、cache quantization、prefix sharing、paged allocation 等因素。

4.12 训练和推理视角不同

从训练视角看:

1. MHA 表达最自然。
2. 并行计算完整序列，KV cache 不是同一个问题。
3. 训练关注 FLOPs、通信、激活显存、稳定性。

从推理视角看:

1. 逐 token decode，无法像训练那样完全并行序列维。
2. KV cache 成为长期状态。
3. 内存带宽、显存容量、batch packing、prefix sharing 非常关键。
4. MQA/GQA/MLA 的收益更明显。

这也是为什么很多架构创新越来越“推理优先”。

过去常见思路是:

先训练一个好模型，再想办法部署。

现在更常见的是:

训练前就考虑 serving 成本，把推理约束写进架构设计。

4.13 什么时候选 MHA、MQA、GQA、MLA

如果是小模型、短上下文、研究实验:

1. MHA 简单直接。
2. 对质量最保守。
3. 实现和分析最容易。

如果是追求极致解码速度、能接受一定质量折中:

1. MQA 很有吸引力。
2. KV cache 最省。
3. 适合带宽瓶颈明显的场景。

如果是通用 LLM serving:

1. GQA 往往是更稳的折中。
2. 现代开源模型常用。
3. 兼顾质量、cache、吞吐。

如果是超大模型、长上下文、MoE、高并发 serving:

1. MLA 这类 latent cache 方案更值得关注。

2. 它从表示层面进一步压缩 cache。
3. 实现复杂度和复现难度也更高。

工程选型永远不是只看论文效果，而是看：

质量目标 + 上下文长度 + batch 并发 + 显存预算 + 延迟要求 + 实现复杂度

4.14 Attention 架构成本审计指标

为了把 MHA / MQA / GQA / MLA 的 trade-off 讲清楚，可以把候选注意力结构抽象成：

$a_i = (H_q, H_{\text{kv}}, d_h, d_c, d_r, q_i, m_i, c_i)$

其中 H_q 是 query head 数， H_{kv} 是 KV head 数， d_h 是 head dim， d_c 是 latent cache dim， d_r 是额外位置相关缓存维度， q_i 是质量代理分， m_i 是工程成熟度， c_i 是实现复杂度。

普通 MHA / MQA / GQA 的 KV cache 可以写成：

$M_{\text{kv}}(a_i) = 2BLTH_{\text{kv}}d_h$

MLA 这类 latent cache 的粗略缓存可以抽象为：

$M_{\text{mla}}(a_i) \approx BLT(d_c + d_r)b$

其中 d_c 是压缩 latent 维度， d_r 可以表示需要额外保留的位置相关 Key 部分。这个公式只是教学抽象，具体实现要以公开技术报告和源码为准。

相对 MHA 的节省率：

$S_{\text{save}}(a_i) = 1 - \frac{M(a_i)}{M_{\text{mha}}}$

Query head 到 KV head 的共享比例：

$\rho_{\text{qkv}} = \frac{H_q}{H_{\text{kv}}}$

一个简化上线门禁可以写成：

$G_{\text{attn}} = \mathbb{1}[q_i \geq \tau_q \text{ and } S_{\text{save}}(a_i) \geq \tau_s \text{ and } m_i \geq \tau_m]$

这组指标不替代真实 ablation 和 serving 压测，但能让面试表达从 “GQA 更快” 升级为 “它用多少 KV head 换来多少 cache 节省、可能损失多少质量、实现成熟度是否足够”。

4.14.1 最小可运行 MHA / GQA / MQA / MLA 成本审计 demo

下面的 demo 不依赖深度学习框架，只用标准库估算四种 attention 结构在同一 serving 场景下的 KV cache MiB、相对 MHA 节省率和简化选型 gate。

```
configs = [
    {"name": "mha_32kv", "q_heads": 32, "kv_heads": 32, "head_dim": 128, "quality": 1.00, "maturity": 0.95, "complexity": 1.0},
    {"name": "gqa_8kv", "q_heads": 32, "kv_heads": 8, "head_dim": 128, "quality": 0.98, "maturity": 0.90, "complexity": 0.9},
    {"name": "mqa_1kv", "q_heads": 32, "kv_heads": 1, "head_dim": 128, "quality": 0.92, "maturity": 0.82, "complexity": 0.8},
    {"name": "mla_latent", "q_heads": 32, "kv_heads": None, "head_dim": 128, "latent_dim": 512, "rope_dim": 64, "quality": 0.95, "maturity": 0.95, "complexity": 0.95}
]
```

```
batch = 16
layers = 32
seq_len = 8192
bytes_per_elem = 2
```

```
def cache_mib(cfg):
    if cfg["kv_heads"] is None:
        elems = batch * layers * seq_len * (cfg["latent_dim"] + cfg["rope_dim"])
    else:
```

```

        elems = 2 * batch * layers * seq_len * cfg["kv_heads"] * cfg["head_dim"]
    return round(elems * bytes_per_elem / 1024**2, 1)

```

```

mha_cache = cache_mib(configs[0])
records = {}
for cfg in configs:
    cache = cache_mib(cfg)
    reduction = round(1 - cache / mha_cache, 3)
    if cfg["kv_heads"] is None:
        qkv_ratio = None
    else:
        qkv_ratio = round(cfg["q_heads"] / cfg["kv_heads"], 3)
    cache_score = min(reduction / 0.95, 1.0)
    score = (
        0.55 * cfg["quality"]
        + 0.25 * cache_score
        + 0.20 * cfg["maturity"]
        - 0.05 * cfg["complexity"]
    )
    failed = []
    if cfg["quality"] < 0.95:
        failed.append("quality")
        score -= 0.08
    if reduction < 0.50:
        failed.append("cache_saving")
    if cfg["maturity"] < 0.70:
        failed.append("maturity")
        score -= 0.04
    records[cfg["name"]] = {
        "cache_mib": cache,
        "reduction_vs_mha": reduction,
        "qkv_ratio": qkv_ratio,
        "serving_score": round(score, 3),
        "failed_gates": failed,
    }

```

```

ranked = sorted(records.items(), key=lambda item: item[1]["serving_score"], reverse=True)
ready = [name for name, record in ranked if not record["failed_gates"]]

```

```

print(f"records={records}")
print(f"ranked=[{name for name, _ in ranked}]")
print(f"ready={ready}")
print(f"best_ready={ready[0] if ready else None}")

```

运行结果应类似：

```

records={'mha_32kv': {'cache_mib': 65536.0, 'reduction_vs_mha': 0.0, 'qkv_ratio': 1.0, 'serving_score': 0.735, 'failed_gates': []},
'gqa_8kv': {'cache_mib': 16384.0, 'reduction_vs_mha': 0.5, 'qkv_ratio': 1.0, 'serving_score': 0.735, 'failed_gates': []},
'mla_latent': {'cache_mib': 0.0, 'reduction_vs_mha': 0.5, 'qkv_ratio': 1.0, 'serving_score': 0.735, 'failed_gates': []},
'mqa_1kv': {'cache_mib': 0.0, 'reduction_vs_mha': 0.5, 'qkv_ratio': 1.0, 'serving_score': 0.735, 'failed_gates': []}}
ranked=['gqa_8kv', 'mla_latent', 'mqa_1kv', 'mha_32kv']
ready=['gqa_8kv']
best_ready=gqa_8kv

```

这段 demo 的重点是：MHA 质量最稳但 cache 太大，MQA cache 最小但可能有质量门禁，MLA cache 很省但实现成熟度和复现成本要单独评估，GQA 常成为通用 serving 的稳健折中。

4.15 常见误区

误区一：MQA/GQA 只是为了减少参数。

更准确：主要是减少 KV cache 和内存带宽，不是参数量优化。

误区二：GQA 一定比 MHA 差很多。

更准确：合适的 GQA 可以质量接近 MHA，同时显著提高推理效率。

误区三：FlashAttention 能解决 KV cache 过大。

更准确：FlashAttention 优化 attention 计算 IO，不从架构上减少历史 K/V cache 总量。

误区四：PagedAttention 是一种 attention 结构。

更准确：PagedAttention 是 serving 系统中的 KV cache 管理算法。

误区五：MLA 只是 GQA 的另一种名字。

更准确：GQA 减少 KV head 数，MLA 压缩 KV 表示到 latent cache，层级不同。

4.16 面试题

题 1：MHA、MQA、GQA 的区别是什么？

参考回答：

MHA 中每个 query head 都有独立 K/V head，表达能力强但 KV cache 大。MQA 保留多个 query heads，但所有 query heads 共享一个 K/V head。

题 2：为什么 MQA/GQA 能提升推理速度？

参考回答：

Decoder-only 增量解码时，每生成一个 token 都要读取历史 token 的 K/V cache。KV cache 大小和 KV head 数成正比。MQA/GQA 减少 KV head 数，从而减少 KV cache 大小。

题 3：GQA 为什么比 MQA 更常见？

参考回答：

MQA 只有一个 K/V head，cache 最省，但所有 query heads 共享同一套 K/V，可能带来质量损失。GQA 保留多个 K/V groups，让一组 query heads 共享一个 K/V head，在质量和效率之间取得平衡。

题 4：MLA 和 GQA 的区别是什么？

参考回答：

GQA 的核心是减少 KV head 数，让多个 query heads 分组共享 K/V head。MLA 的核心是把 K/V 相关信息压缩成 latent cache，再在 attention 计算时解压。DeepSeek-V2 报告显示 MLA 能显著降低 KV cache 并提升吞吐。

题 5：FlashAttention、PagedAttention、GQA 分别优化什么？

参考回答：

FlashAttention 优化 attention kernel 和 GPU IO，不改变数学结果。PagedAttention 优化 serving 系统中的 KV cache 管理，减少内存碎片。

4.17 小练习

1. 推导 MHA、MQA、GQA 的 KV cache 大小比例。
2. 假设 num_q_heads=40、num_kv_heads=8，说明每个 KV head 服务几个 Q heads。
3. 解释为什么 KV cache 是长上下文 serving 的核心瓶颈之一。
4. 阅读 MQA 论文摘要，找出“memory-bandwidth cost”这句话对应的工程问题。
5. 阅读 GQA 论文摘要，解释为什么它是 MHA 和 MQA 的折中。
6. 阅读 DeepSeek-V2 摘要，解释 MLA、MoE、KV cache 和吞吐之间的关系。

7. 用一张表区分 FlashAttention、PagedAttention、MQA、GQA、MLA。
8. 运行本章成本审计 demo，解释为什么 gqa_8kv 在 toy gate 中比 mqa_1kv 和 mla_latent 更适合作为默认上线方案。

4.18 本章总结

本章系统比较了 MHA、MQA、GQA、MLA 的演进与 trade-off。

核心结论：

1. MHA 表达能力完整，但每个 head 都有 K/V，推理 KV cache 昂贵。
2. MQA 让所有 query heads 共享一个 K/V head，显著降低 cache 和内存带宽，但可能损失质量。
3. GQA 让一组 query heads 共享一个 K/V head，是 MHA 和 MQA 之间的实用折中。
4. MLA 进一步把 K/V 信息压缩到 latent cache，是面向长上下文和高效推理的更激进设计。
5. KV cache 是 decoder-only LLM serving 的关键瓶颈之一，影响显存、batch、吞吐和延迟。
6. FlashAttention 优化 kernel，PagedAttention 优化 cache 管理，MQA/GQA/MLA 优化模型结构，三者层面不同但可以组合。
7. 现代注意力设计已经不只是模型表达问题，而是模型架构、推理系统和硬件效率共同优化的问题。

下一章会进入 FFN、SwiGLU、Gated MLP 与 MoE FFN，重点讲 Transformer 中另一个被低估的主力模块：前馈网络。

第五章：FFN、SwiGLU、Gated MLP 与 MoE FFN

0. 本讲资料边界与第二轮精修口径

本章第二轮精修以公开论文和技术报告为资料边界：Transformer 原论文对 position-wise FFN 的定义，Shazeer 的 GLU Variants Improve Transformer 对 ReGLU / GEGLU / SwiGLU 的对比，PaLM 和 LLaMA 公开资料中对 SwiGLU / gated MLP 的采用，GShard、Switch Transformer、Mixtral of Experts、DeepSeekMoE 和 DeepSeek-V2 对 sparse MoE、top-k routing、专家负载、active parameters 与系统复杂度的公开描述。

本章只讨论公开可验证的 FFN / gated MLP / MoE FFN 通用机制；不把某个闭源模型的 expert 数、hidden size、routing recipe 或负载均衡技巧写成行业标准，也不把 toy demo 的门禁结果当成真实 benchmark。

第二轮补强重点有三点：

1. 将 FFN、SwiGLU、MoE routing、参数量、active parameters、expert capacity 和 load balance 写成 GitHub Markdown 更稳定的 MathJax 公式。
2. 明确 dense FFN、SwiGLU/Gated MLP 和 MoE FFN 分别解决“非线性通道变换”“通道门控表达”和“稀疏激活扩容”三个层面的问题。
3. 补一个 0 依赖 Python demo，审计 dense 4D FFN、SwiGLU 近似等参配置、MoE top-2 的总参数 / 激活参数、capacity overflow 和路由负载均衡。

5.1 本章定位

很多人学 Transformer 时，会把注意力机制当成唯一主角，把 FFN 当成“后面跟着的两层 MLP”。这是一个常见误区。

在现代大模型中，FFN/MLP 层通常占据大量参数和计算。Attention 负责 token 之间的信息路由，FFN 则负责对每个 token 的表示做非线性变换、特征组合和容量扩展。可以粗略理解为：

Attention: token 之间交换信息

FFN/MLP: 每个 token 内部加工信息

本章讲 Transformer block 中另一个核心模块：FFN。从原始 Transformer 的 ReLU FFN，到 GELU，再到现代 LLM 常用的 SwiGLU/Gated MLP，再到 MoE FFN。

学完本章，你应该能回答：

1. Transformer FFN 为什么是 position-wise 的。

2. FFN 在 Transformer 中到底起什么作用。
3. 为什么现代 LLM 常用 SwiGLU，而不是普通 ReLU/GELU FFN。
4. Gated MLP 的门控直觉是什么。
5. 为什么 MoE 通常替换 FFN，而不是替换 attention。
6. Switch、GShard、Mixtral、DeepSeekMoE 这些 MoE FFN 的核心 trade-off 是什么。
7. 面试中如何解释 FFN、SwiGLU 和 MoE 的关系。

5.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need。原始 Transformer 使用 position-wise feed-forward networks，每个位置独立应用相同的两层全连接网络和 ReLU。
2. Shazeer, 2020, GLU Variants Improve Transformer。系统测试 GLU 变体，发现部分 gated FFN 变体相对 ReLU/GELU 能提升 Transformer 质量，SwiGLU 后来成为现代 LLM 常用组件。
3. Chowdhery et al., 2022, PaLM。PaLM 是大规模 decoder-only Transformer，公开使用 SwiGLU 等现代结构并展示 scaling 效果。
4. Touvron et al., 2023, LLaMA。LLaMA 使用 RMSNorm、SwiGLU、RoPE 等组件，成为开源 LLM 架构基线。
5. Lepikhin et al., 2020, GShard。使用 sparsely-gated MoE 和自动 sharding，把 Transformer 扩展到 600B+ 参数级别。
6. Fedus et al., 2021/2022, Switch Transformers。简化 MoE routing，用每个 token 选择一个 expert 的方式扩展到 trillion-parameter 模型，并讨论通信、稳定性和低精度训练。
7. Jiang et al., 2024, Mixtral of Experts。Mixtral 8x7B 使用 sparse MoE，每层有 8 个 FFN experts，每个 token 选 2 个专家，推理激活参数少于总参数。
8. DeepSeek-AI, 2024, DeepSeek-V2。DeepSeekMoE 通过 sparse computation 经济地训练强模型，并和 MLA 一起优化训练和推理效率。

需要注意：不同模型对 FFN hidden size、gate 维度、activation、expert 数、top-k routing、负载均衡的细节可能不同。本章重点讲通用机制和 trade-off，不把某个模型的私有配置泛化成标准。

5.3 原始 Transformer FFN 是什么

原始 Transformer 每个 block 中有两个主要子层：

Multi-Head Attention

Position-wise Feed-Forward Network

原论文中的 FFN 形式是：

$$\mathrm{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

其中 x 是单个 token 位置的 hidden vector， W_1 把 hidden size 从 d 扩展到中间维度 m ， W_2 再投回 d ， σ 在原始 Transformer 中是 ReLU。也就是两层线性层，中间加非线性激活。

它叫 position-wise，是因为同一个 FFN 独立作用在每个 token 位置上：

$$y_i = \mathrm{FFN}(x_i), \quad i=1, \dots, T$$

不同位置之间不在 FFN 中直接交互。token 之间的信息交换主要由 attention 完成。

这有一个重要设计分工：

Attention: mix across sequence dimension

FFN: transform across hidden/channel dimension

从张量形状看：

X : [batch, seq_len, hidden_size]

FFN: hidden_size $\rightarrow$ intermediate_size $\rightarrow$ hidden_size

Output: [batch, seq_len, hidden_size]

每个 token 使用同一组 FFN 参数，所以 FFN 像一个作用在 token 表示上的共享非线性函数。

5.4 FFN 为什么重要

如果只有 attention，没有 FFN，模型主要是在不同 token 的 Value 之间做线性加权组合。虽然 attention 权重来自非线性 softmax，但每层表达能力仍然受限。

FFN 提供了几个关键能力。

第一，非线性变换。

FFN 引入 ReLU/GELU/SwiGLU 等非线性，使模型能学习更复杂的特征组合。

第二，通道维度扩展。

典型 FFN 会把 hidden size 扩大到 4 倍左右，再投影回来：

$D \rightarrow 4D \rightarrow D$

这给每个 token 提供更大的中间特征空间。

第三，知识和模式存储。

很多研究和工程直觉都认为 FFN/MLP 层承担了大量事实、模式和特征转换能力。Attention 负责把相关信息带到当前位置，FFN 负责把这些信息加工成下一层可用表示。

第四，参数容量主体。

在很多 Transformer 配置中，FFN 参数量远大于 attention 投影参数量。因此扩展模型容量时，FFN 是非常关键的部分。

一句话：

Attention 决定 token 从哪里拿信息，FFN 决定拿到信息后怎么加工。

5.5 ReLU、GELU 到 GLU

原始 Transformer 使用 ReLU。

后来很多语言模型使用 GELU。GELU 相比 ReLU 更平滑，在 BERT/GPT 系列中很常见。

普通 FFN 可以写成：

$$\mathrm{FFN}(x) = \phi(xW_1)W_2$$

GLU 类结构则引入门控：

$$u = xW_u, \quad g = \sigma(\mathrm{gate})(xW_g), \quad y = (u \odot g)W_d$$

其中 u 是候选特征， g 是 gate branch， $\odot$ 是逐元素乘法， W_d 是 down projection。

直觉上，GLU 把 FFN 分成两条支路：

1. value branch：产生候选特征。
2. gate branch：决定哪些特征通过。

也就是说，不是所有中间特征都同等进入下一层，而是由 gate 控制信息流。

这和 attention 的“加权读取”有点类似，但发生在通道维度，而不是 token 维度。

5.6 SwiGLU 是什么

Shazeer 的 GLU Variants Improve Transformer 测试了多种 GLU 变体，例如 ReGLU、GEGLU、SwiGLU 等。

SwiGLU 可以理解为：

$$g = \mathrm{SiLU}(xW_g), \quad u = xW_u$$

然后把 gate branch 和 up branch 逐元素相乘，再通过 down projection 回到 hidden size:

$y = (g \odot u) W_d$

Swish 通常是:

$\mathrm{SiLU}(z) = z \cdot \frac{1}{1 + \exp(-z)}$

现代 LLM 中常见的 MLP 形式是:

```
gate = SiLU(W_gate x)
up = W_up x
hidden = gate * up
out = W_down hidden
```

这里的 SiLU 和 Swish 基本是同类表达。

SwiGLU 的直觉:

1. $W_{up} x$ 产生候选特征。
2. $\mathrm{SiLU}(W_{gate} x)$ 产生平滑门控。
3. 两者逐元素相乘，选择性激活特征。
4. W_{down} 把中间特征投回 hidden space。

这比普通 FFN 多了一条 projection，但通常可以通过调整 intermediate size 控制参数量。

5.7 为什么现代 LLM 爱用 SwiGLU

LLaMA、PaLM 等模型把 SwiGLU/Gated FFN 推成了现代 LLM 的常见 recipe。

原因可以从三层理解。

第一，表达能力。

门控结构让模型能对不同 token、不同上下文动态选择通道特征。它比简单的 ReLU/GELU 单支路更灵活。

第二，训练效果。

GLU 变体实验显示部分 gated FFN 能提升 Transformer 质量。实践中 SwiGLU 成为大模型常用选择。

第三，工程可控。

SwiGLU 仍然是矩阵乘法 + elementwise activation + elementwise multiply，非常适合 GPU/TPU。它不像 MoE 那样引入复杂 routing 和通信。

所以 SwiGLU 是一个很好的工程折中:

比普通 FFN 表达强很多，但实现复杂度远低于 MoE。

5.8 一个 PyTorch 版 SwiGLU MLP

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
class SwiGLUMLP(nn.Module):
    def __init__(self, hidden_size: int, intermediate_size: int):
        super().__init__()
        self.gate_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
        self.up_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
        self.down_proj = nn.Linear(intermediate_size, hidden_size, bias=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```

gate = F.silu(self.gate_proj(x))
up = self.up_proj(x)
return self.down_proj(gate * up)

```

输入输出形状:

```

x: [batch, seq_len, hidden_size]
gate/up: [batch, seq_len, intermediate_size]
out: [batch, seq_len, hidden_size]

```

注意, FFN/MLP 对每个 token 独立作用, 不直接混合序列维度。

5.9 FFN 参数量和计算量

普通 FFN:

hidden_size -> intermediate_size -> hidden_size

参数量约为:

$P_{\{\mathrm{dense}\}} \approx 2dm$

如果 $\text{intermediate_size} = 4 * \text{hidden_size}$, 参数量约是:

$m=4d, \quad P_{\{\mathrm{dense}\}} \approx 8d^2$

其中 d 是 hidden size, m 是 intermediate size。这里忽略 bias, 因为大模型中 bias 相比矩阵参数通常很小, 很多现代实现也直接不使用 FFN bias。

SwiGLU MLP 有三组矩阵:

gate_proj, up_proj, down_proj

参数量约为:

$P_{\{\mathrm{swiglu}\}} \approx 3dm$

为了让参数量接近普通 4D FFN, 实际常把 SwiGLU intermediate size 设得比 $4d$ 小。近似等参条件是:

$3dm \approx 8d^2, \quad m \approx \frac{8}{3}d$

面试中要能说明:

SwiGLU 多一条 gate projection, 所以不能简单在相同 intermediate size 下和普通 FFN 比参数; 通常会调小 hidden dim 以控制总参

5.10 MoE FFN: 把一个 FFN 变成多个专家

MoE 的核心思想是条件计算。

普通 dense FFN 是:

每个 token 都经过同一个 FFN。

MoE FFN 是:

每个 token 由 router 选择少数几个 experts, 只经过被选中的 FFN。

每个 expert 通常就是一个 FFN/MLP。

如果有 8 个 experts, 每个 token 只选 2 个:

total FFN capacity = 8 experts

active FFN compute per token = 2 experts

这带来一个巨大优势:

总参数量可以很大, 但每个 token 的激活计算相对较小。

这就是 sparse activation。

5.11 为什么 MoE 通常替换 FFN，而不是 Attention

MoE 最常见的位置是 FFN 层，而不是 attention 层。

原因包括：

1. FFN 本来就是 token-wise 独立计算，适合按 token 路由到不同专家。
2. FFN 参数量大，替换成专家后能显著扩展模型容量。
3. Attention 负责 token 间交互，如果按专家拆得太复杂，会影响全局信息路由。
4. FFN expert 可以相对独立并行部署，更适合 expert parallelism。
5. MoE 的主要目标是增加参数容量，而 FFN 是容量主体。

所以 MoE FFN 可以理解为：

把 Transformer block 中最占容量的 dense MLP，替换成多个稀疏激活的 MLP experts。

5.12 Router：MoE 的关键模块

MoE 需要一个 router 决定每个 token 去哪个 expert。

简化形式：

$$r = xW_r, \quad p = \mathrm{softmax}(r), \quad \mathcal{E}_k = \mathrm{TopK}(p, k)$$

对选中的 expert，常见输出组合可以写成：

$$\alpha_e = \frac{p_e}{\sum_{j \in \mathcal{E}_k} p_j}, \quad y = \sum_{e \in \mathcal{E}_k} \alpha_e f_e(x)$$

然后 token 只送到 top-k experts。

Switch Transformer 使用更简单的 top-1 routing：每个 token 只选一个 expert。Mixtral 使用 top-2 routing：每个 token 选两个 experts，并组合输出。

Top-1 routing 优点：

1. 计算更省。
2. 通信更简单。
3. 路由实现更直接。

Top-2 routing 优点：

1. 表达更强。
2. 对单个 expert 选择错误更鲁棒。
3. 输出可由多个专家组合。

Router 的难点是负载均衡。如果很多 token 都去同一个 expert，就会出现：

1. 热点 expert 过载。
2. 其他 expert 闲置。
3. batch 内通信不均衡。
4. 训练不稳定。

所以 MoE 常配合 load balancing loss、capacity factor、token dropping 或 auxiliary-loss-free balancing 等设计。

5.13 GShard、Switch、Mixtral、DeepSeekMoE 的演进

GShard 展示了用 sparsely-gated MoE 和自动 sharding 把 Transformer 扩展到 600B+ 参数级别。它的重点是：

1. 条件计算。
2. 自动分片。
3. 大规模 TPU 训练。
4. 多语言翻译质量提升。

Switch Transformer 进一步简化 MoE，把 routing 简化为 top-1。它强调：

- 1. 简化通信和计算。
- 2. 降低训练复杂度。
- 3. 稳定低精度训练。
- 4. 用 sparse activation 扩展到 trillion parameters。

Mixtral 8x7B 把 MoE 带入开源 LLM 主流视野。公开摘要指出：每层有 8 个 feedforward blocks，每个 token 选 2 个 experts；每个 token 可访问 47B 参数，但推理只激活约 13B 参数。

DeepSeek-V2 的 DeepSeekMoE 则强调经济训练和高效推理。它和 MLA 组合，说明现代架构会同时优化：

FFN 容量 -> MoE

KV cache -> MLA

5.14 Dense FFN、SwiGLU、MoE FFN 对比

结构	核心思想	优点	代价
ReLU/GELU FFN	两层 MLP + 激活	简单稳定	表达较普通
SwiGLU/Gated MLP	候选特征乘门控	表达更强，现代 LLM 常用	多一条 projection
MoE FFN	多个 experts，按 token 选择少数激活	总容量大，激活计算低	routing、通信、负载均衡复杂

可以把它们看成逐步增强：

普通 FFN：一个共享专家

SwiGLU：一个带门控的共享专家

MoE FFN：多个带或不带门控的专家，由 router 选择

5.15 MoE 的工程 Trade-off

MoE 的优势很诱人：更多参数、更少激活计算。

但工程代价也很明显。

第一，通信成本。

不同 token 要发到不同 expert，专家可能分布在不同 GPU/TPU 上，需要 all-to-all 通信。

第二，负载均衡。

Router 可能把大量 token 分到少数 expert，造成热点。

第三，训练稳定性。

MoE 比 dense 模型更容易出现 routing collapse、expert under-utilization、loss spike。

第四，serving 复杂度。

推理时要管理 expert placement、batch dispatch、token combine、cache 和延迟。

第五，评估解释困难。

不同 token 激活不同 expert，模型行为更分散。

所以 MoE 不是免费午餐。

它适合有强系统能力、追求大容量和成本效率的团队。对小团队或小模型，dense SwiGLU 往往更简单可靠。

5.16 FFN / MoE 成本审计指标

把 dense FFN、SwiGLU 和 MoE 放到同一张审计表里时，至少要区分三件事：总参数、每 token 激活参数、路由负载。

dense FFN 每个 token 都使用同一个 FFN：

$$P_{\{\mathrm{active}\}} = P_{\{\mathrm{total}\}} \approx P_{\{\mathrm{dense}\}}$$

MoE FFN 有 E 个 experts，但每个 token 只激活 top-k 个 experts：

$$P_{\{\mathrm{total}\}} \approx E P_{\{\mathrm{expert}\}} + P_{\{\mathrm{router}\}}$$

$$P_{\{\mathrm{active}\}} \approx k P_{\{\mathrm{expert}\}} + P_{\{\mathrm{router}\}}$$

其中 P_{router} 通常远小于 expert 参数，但在严谨估算中要说明是否计入。

路由容量要看 token 分布。设一个 batch 内有 N 个 token，每个 token 选 k 个 experts，平均每个 expert 收到：

$$\bar{n} = \frac{Nk}{E}$$

如果 capacity factor 是 γ ，单个 expert 的容量可粗略写成：

$$C_e = \left\lceil \gamma \bar{n} \right\rceil$$

若第 e 个 expert 实际收到 n_e 个 token，则 overflow token 数为：

$$O = \sum_{e=1}^E \max(0, n_e - C_e)$$

负载不均衡可以用最大负载和平均负载的比值粗略衡量：

$$R_{\{\mathrm{load}\}} = \frac{\max_e n_e}{\bar{n}}$$

一个面试级 FFN / MoE 架构门禁可以写成：

$$G_{\{\mathrm{ffn}\}} = \mathbb{1}_{\{O=0, R_{\{\mathrm{load}\}} \leq \tau_r, U_{\{\mathrm{expert}\}} \geq \tau_u\}}$$

其中 U_{expert} 是至少收到一个 token 的 expert 占比。这个门禁强调：MoE 不是只要 active parameters 小就好，还要看 router 是否均衡、capacity 是否溢出、通信是否可控、质量是否过线。

5.16.1 最小可运行 FFN / MoE 成本审计 demo

下面的 demo 不依赖 PyTorch，只用标准库估算 dense 4D FFN、SwiGLU 近似等参配置和 top-2 MoE 的参数量，并模拟 balanced routing 与 collapsed routing 的差异。

```
from math import ceil, exp
```

```
def dense_ffn_params(hidden_size: int, intermediate_size: int) -> int:
    return 2 * hidden_size * intermediate_size
```

```
def swiglu_params(hidden_size: int, intermediate_size: int) -> int:
    return 3 * hidden_size * intermediate_size
```

```
def softmax(logits):
    max_logit = max(logits)
    weights = [exp(x - max_logit) for x in logits]
    total = sum(weights)
    return [x / total for x in weights]
```

```
def topk_indices(values, k):
    return sorted(range(len(values)), key=lambda i: (-values[i], i))[:k]
```

```

def make_logits(top_pair, num_experts):
    logits = [-2.0] * num_experts
    logits[top_pair[0]] = 3.0
    logits[top_pair[1]] = 2.0
    return logits

def audit_routing(top_pairs, num_experts, top_k, capacity_factor):
    counts = [0] * num_experts
    for pair in top_pairs:
        probs = softmax(make_logits(pair, num_experts))
        for expert_id in topk_indices(probs, top_k):
            counts[expert_id] += 1

    token_count = len(top_pairs)
    avg_load = token_count * top_k / num_experts
    capacity = ceil(capacity_factor * avg_load)
    overflow = sum(max(0, count - capacity) for count in counts)
    load_ratio = max(counts) / avg_load if avg_load else 0.0
    utilized = sum(1 for count in counts if count > 0)

    return {
        "counts": counts,
        "capacity": capacity,
        "overflow": overflow,
        "load_ratio": round(load_ratio, 3),
        "utilized_experts": utilized,
        "gate_pass": overflow == 0 and load_ratio <= 1.5 and utilized >= int(0.75 * num_experts),
    }

hidden_size = 4096
dense_intermediate = 4 * hidden_size
swiglu_intermediate = 11008
num_experts = 8
top_k = 2
capacity_factor = 1.25

dense_params = dense_ffn_params(hidden_size, dense_intermediate)
expert_params = swiglu_params(hidden_size, swiglu_intermediate)
router_params = hidden_size * num_experts
moe_total_params = num_experts * expert_params + router_params
moe_active_params = top_k * expert_params + router_params

balanced_pairs = [(0, 1), (2, 3), (4, 5), (6, 7)] * 3
collapsed_pairs = [(0, 1)] * 10 + [(2, 3)] * 2

balanced_audit = audit_routing(balanced_pairs, num_experts, top_k, capacity_factor)
collapsed_audit = audit_routing(collapsed_pairs, num_experts, top_k, capacity_factor)

print("dense_4d_params=", dense_params)
print("swiglu_params=", expert_params)
print("swiglu_to_dense_ratio=", round(expert_params / dense_params, 4))

```



```

print("moe_total_params=", moe_total_params)
print("moe_active_params=", moe_active_params)
print("active_to_total_ratio=", f"{moe_active_params / moe_total_params:.4f}")
print("balanced_audit=", balanced_audit)
print("collapsed_audit=", collapsed_audit)
print("best_route=", "balanced" if balanced_audit["gate_pass"] else "blocked")

```

预期输出：

```

dense_4d_params= 134217728
swiglu_params= 135266304
swiglu_to_dense_ratio= 1.0078
moe_total_params= 1082163200
moe_active_params= 270565376
active_to_total_ratio= 0.2500
balanced_audit= {'counts': [3, 3, 3, 3, 3, 3, 3, 3], 'capacity': 4, 'overflow': 0, 'load_ratio': 1.0, 'utilized_experts': 1}
collapsed_audit= {'counts': [10, 10, 2, 2, 0, 0, 0, 0], 'capacity': 4, 'overflow': 12, 'load_ratio': 3.333, 'utilized_experts': 2}
best_route= balanced

```

这个 demo 要读出三点：

1. SwiGLU 多一条 projection，所以 11008 这个中间维度让 4096 -> 11008 -> 4096 的 gated MLP 参数量接近普通 4d dense FFN。
2. MoE 总参数可以约为单个 expert 的 E 倍，但 top-2 推理每 token 只激活约 2/E 的 expert 参数。
3. 如果 router collapse 到少数 experts，即使 active parameters 看起来低，也会出现 capacity overflow、expert 利用不足和系统热点。

5.17 FFN 和 Attention 的互补关系

一个 Transformer block 可以这样理解：

Attention：收集上下文信息

FFN：加工当前 token 表示

Residual：保留和累积信息

Norm：稳定数值分布

如果 attention 找到了相关证据，但 FFN 不够强，模型可能无法把证据转换成正确推理。

如果 FFN 很强，但 attention 没把相关信息带来，模型只能加工局部或错误信息。

所以现代架构演进不是只优化 attention，也会优化 FFN：

1. SwiGLU 提升 dense FFN 表达。
2. MoE 提升参数容量。
3. 更好的 routing 提升专家利用率。
4. 更好的并行系统降低 MoE 通信代价。

5.18 常见误区

误区一：FFN 不重要，Transformer 主要靠 attention。

更准确：attention 负责信息路由，FFN 负责非线性变换和容量扩展，二者都核心。

误区二：SwiGLU 只是换了激活函数。

更准确：SwiGLU 是 gated MLP，包含 gate/up/down 三个投影，门控机制改变了通道特征选择方式。

误区三：MoE 就是让模型更大，所以一定更好。

更准确：MoE 增加总参数但只激活部分专家，质量依赖 routing、负载均衡、数据、训练稳定性和系统实现。

误区四：MoE 推理一定更快。

更准确：激活 FLOPs 可能低，但 all-to-all 通信、expert dispatch 和 batch 不均衡可能带来延迟。

误区五：Mixtral 8x7B 每次都用了 47B 参数计算。

更准确：每个 token 可以访问较大总参数空间，但推理只激活少数 experts，对应较少 active parameters。

5.19 面试题

题 1：Transformer FFN 的作用是什么？

参考回答：

Attention 负责 token 之间的信息交换，FFN 负责对每个 token 的 hidden representation 做非线性通道变换。它通常把 hidden size

题 2：为什么现代 LLM 常用 SwiGLU？

参考回答：

SwiGLU 是 gated MLP。它用一条 up projection 产生候选特征，用一条 gate projection 经过 SiLU/Swish 产生门控，两者逐元素相乘

题 3：SwiGLU 和普通 FFN 参数量怎么比？

参考回答：

普通 FFN 通常有 up 和 down 两个矩阵，参数约为 $2 * \text{hidden_size} * \text{intermediate_size}$ 。SwiGLU 有 gate、up、down 三个矩阵，参数

题 4：为什么 MoE 通常替换 FFN？

参考回答：

因为 FFN 是 token-wise 独立计算，参数量大，适合按 token 路由到不同专家。把 FFN 换成多个 experts 后，每个 token 只激活少数

题 5：Switch Transformer 和 Mixtral 的 routing 有什么区别？

参考回答：

Switch Transformer 强调简化 MoE routing，每个 token 通常选择一个 expert，即 top-1 routing，降低通信和计算复杂度。Mixtral 2 routing，表达更强但计算和通信也更复杂。两者都是 sparse MoE，但 routing 策略不同。

题 6：MoE 的主要工程难点是什么？

参考回答：

主要难点包括 all-to-all 通信、负载均衡、routing collapse、expert 利用不均、训练稳定性、低精度训练、serving 时的 exper

5.20 小练习

1. 写出普通 FFN、GEGLU、SwiGLU 的公式，并比较差异。
2. 用 PyTorch 实现一个 SwiGLU MLP，并打印中间张量形状。
3. 计算 hidden size 为 4096 时，普通 4D FFN 的参数量约是多少。
4. 计算同样 hidden size 下，SwiGLU 如果 intermediate size 为 8D/3，参数量约是多少。
5. 画一个 MoE FFN 的 routing 流程图：token -> router -> top-k experts -> combine。
6. 阅读 Switch Transformer 摘要，解释为什么它强调 simple and efficient sparsity。
7. 阅读 Mixtral 摘要，解释“47B total / 13B active”这类说法的含义。
8. 讨论为什么 MoE 适合大规模团队，但不一定适合所有小模型项目。
9. 运行本章 FFN / MoE 成本审计 demo，解释为什么 collapsed routing 会导致 overflow 和 expert 利用不足。

5.21 本章总结

本章系统讲了 FFN、SwiGLU、Gated MLP 和 MoE FFN。

核心结论：

1. FFN 是 Transformer block 中和 attention 同等重要的模块。
2. Attention 负责 token 间信息路由，FFN 负责 token 内通道变换和非线性加工。
3. 原始 Transformer 使用 position-wise ReLU FFN，后来很多模型使用 GELU，再后来现代 LLM 大量采用 SwiGLU。
4. SwiGLU 是 gated MLP，通过 gate/up/down 三个投影增强通道特征选择能力。
5. SwiGLU 参数量比较时要注意 intermediate size，不能只看激活函数名。
6. MoE FFN 把一个 dense FFN 变成多个 experts，由 router 为每个 token 选择少数 experts。
7. MoE 的核心价值是扩大总参数容量，同时保持每 token 激活计算相对较低。
8. MoE 的核心代价是 routing、通信、负载均衡、训练稳定性和 serving 复杂度。
9. 现代大模型架构演进中，attention 优化和 FFN/MoE 优化同样关键。

下一章会进入 Residual、LayerNorm、RMSNorm、Pre-Norm 与训练稳定性，重点讲为什么规范化和残差结构决定了深层 Transformer 能否稳定训练。

第六章：Residual、LayerNorm、RMSNorm、Pre-Norm 与训练稳定性

0. 本讲资料边界与第二轮精修口径

本章第二轮精修以公开论文和技术报告为资料边界：ResNet 对 residual learning 的优化动机，Layer Normalization 对单样本 hidden 维统计的定义，Transformer 原论文中的 residual connection + LayerNorm，RMSNorm 对 re-centering / re-scaling 的拆分，Xiong 等人对 Post-LN / Pre-LN 和 warmup 关系的分析，Admin 对 residual branch 依赖和扰动放大的讨论，DeepNet / DeepNorm 对极深 Transformer 残差缩放与初始化的设计，以及 LLaMA 等公开模型对 RMSNorm / Pre-Norm 风格结构的采用。

本章只讨论公开可验证的 residual、normalization placement、warmup 和训练稳定性机制；不把某个模型的私有初始化、学习率 schedule、loss spike 处理或混合精度 recipe 写成通用标准，也不把 toy demo 的门禁结果当成真实训练 benchmark。

第二轮补强重点有三点：

1. 将 Residual、LayerNorm、RMSNorm、Pre-LN、Post-LN、warmup、residual scaling 和稳定性门禁改成稳定 MathJax 表达。
2. 明确 norm placement 不是孤立技巧，而是和残差主路径、初始化、学习率、optimizer、precision、数据 outlier 共同决定训练稳定性。
3. 补一个 0 依赖 Python demo，审计 residual stream 尺度、update ratio、mean drift 和 warmup 下的 early update pressure。

6.1 本章定位

前面几章讲了 attention 和 FFN。现在进入一个更容易被低估、但对大模型训练极其关键的主题：残差连接和归一化。

如果没有 Residual、LayerNorm/RMSNorm、合适的 norm placement、初始化和学习率策略，深层 Transformer 很容易训练不稳定：loss spike、梯度爆炸、早期发散、需要很长 warmup、深层模型无法收敛。

本章要回答的问题是：

1. Residual connection 为什么能让深层网络更容易训练。
2. LayerNorm 到底归一化什么，为什么适合 Transformer。
3. RMSNorm 为什么在现代 LLM 中常见。
4. Post-LN 和 Pre-LN Transformer 有什么区别。
5. 为什么 Post-LN 往往更依赖 warmup，而 Pre-LN 更稳定。
6. DeepNorm、Admin 等方法想解决什么问题。

7. 大模型训练中 norm、residual、初始化、学习率之间有什么关系。

这章的核心观点是：

Transformer 能堆深，不只是因为 attention 强，还因为 residual path 和 normalization 让信息与梯度能稳定流动。

6.2 资料来源和可信边界

本章主要参考以下公开资料：

1. He et al., 2015, Deep Residual Learning for Image Recognition。提出 residual learning，说明残差结构让非常深的网络更容易优化。
2. Ba et al., 2016, Layer Normalization。提出 LayerNorm，对单个样本的层内神经元活动做归一化，训练和测试行为一致，适合序列模型。
3. Vaswani et al., 2017, Attention Is All You Need。原始 Transformer 使用 residual connection + LayerNorm。
4. Zhang and Sennrich, 2019, Root Mean Square Layer Normalization。提出 RMSNorm，去掉 re-centering，仅保留 re-scaling，计算更简单，并能取得接近 LayerNorm 的效果。
5. Xiong et al., 2020, On Layer Normalization in the Transformer Architecture。分析 Post-LN 和 Pre-LN，说明 LayerNorm 位置影响初始化时梯度行为，Pre-LN 有助于减少 warmup 依赖。
6. Liu et al., 2020, Understanding the Difficulty of Training Transformers。分析 Transformer 训练困难，指出残差分支依赖和扰动放大问题，并提出 Admin 初始化。
7. Wang et al., 2022, DeepNet: Scaling Transformers to 1,000 Layers。提出 DeepNorm 和配套初始化，尝试结合 Post-LN 性能和 Pre-LN 稳定性，把 Transformer 扩展到极深层。
8. Touvron et al., 2023, LLaMA。LLaMA 等现代 LLM 使用 RMSNorm、Pre-Norm 风格结构，成为开源 LLM 常见 recipe。

需要注意：不同模型的 norm placement、residual scaling、初始化、optimizer、learning rate warmup、precision、parallelism 会互相影响。不能孤立地说某个 norm “一定最好”。

6.3 Residual Connection 的直觉

残差连接来自 ResNet 的核心思想。

普通深层网络每层学习：

$$y=F(x)$$

残差网络让层学习：

$$y=x+F(x)$$

也就是说，模型不用从零学习完整映射，只需要学习输入上的增量修正。

这有几个好处。

第一，信息直通。

即使 $F(x)$ 初始很差， x 仍然可以直接传到下一层。

第二，梯度更容易传播。

反向传播时，梯度可以沿 identity path 流动，不必完全穿过复杂非线性模块。

第三，深层网络更容易接近恒等映射。

如果某些层暂时不需要复杂变换，可以让 $F(x)$ 接近 0。

Transformer 中每个子层通常都有残差：

$$h'=h+\mathrm{Attention}(h)$$

$$h_{\mathrm{out}}=h'+\mathrm{FFN}(h')$$

这让 Transformer 可以逐层在 residual stream 上累积信息。

6.4 Residual Stream: 现代 Transformer 的主干

可以把 Transformer 中的隐藏状态看作 residual stream。

每个 block 做的事情是：

```
residual stream
-> attention reads/writes information
-> residual stream
-> FFN processes channel features
-> residual stream
```

Attention 和 FFN 都不是完全替换 hidden state，而是在 residual stream 上写入增量。

这解释了为什么 Transformer 中的残差特别重要：

1. 各层可以不断向同一个信息流写入特征。
2. 早期层的信息有机会保留到后期层。
3. 梯度有较短路径回到浅层。
4. 多个模块像在共同编辑一个表示空间。

但残差也带来问题：如果每层写入的增量尺度不受控，表示会随着层数变深而漂移或爆炸。

这就是 normalization 和 residual scaling 的作用。

6.5 LayerNorm 归一化什么

LayerNorm 对每个样本、每个 token 的 hidden dimension 做归一化。

假设一个 token 表示是：

x : [hidden_size]

LayerNorm 计算：

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

$$\mathrm{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i$$

它和 BatchNorm 的关键区别是：

1. BatchNorm 使用 batch 维度统计。
2. LayerNorm 使用单个样本内部 hidden 维度统计。
3. LayerNorm 训练和推理计算一致。
4. LayerNorm 不依赖 batch size。

这对 NLP/LLM 很重要，因为：

1. 序列长度可变。
2. batch size 可能很小或动态变化。
3. 自回归推理时一次可能只处理一个 token。
4. 分布式训练中 batch 统计同步成本高。

所以 Transformer 使用 LayerNorm 而不是 BatchNorm 是非常自然的选择。

6.6 RMSNorm: 去掉均值中心化

RMSNorm 是 LayerNorm 的简化变体。

LayerNorm 做两件事：

re-centering: 减去均值

re-scaling: 除以标准差

RMSNorm 只做 re-scaling:

$$\mathrm{rms}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

$$\mathrm{RMSNorm}(x)_i = w_i \frac{x_i}{\mathrm{rms}(x)}$$

它不减均值，也通常没有 bias。

RMSNorm 的直觉：

1. 控制向量尺度。
2. 保留方向信息。
3. 计算更简单。
4. 对大模型训练足够有效。

RMSNorm 论文认为 LayerNorm 中的 re-centering invariance 可能不是必需的，而 re-scaling invariance 更关键。现代 LLM 如 LLaMA 使用 RMSNorm，使它成为开源 LLM 常见组件。

6.7 LayerNorm 和 RMSNorm 的代码

简化 LayerNorm:

```
import torch
import torch.nn as nn
```

```
class SimpleLayerNorm(nn.Module):
    def __init__(self, hidden_size: int, eps: float = 1e-5):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.bias = nn.Parameter(torch.zeros(hidden_size))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        mean = x.mean(dim=-1, keepdim=True)
        var = (x - mean).pow(2).mean(dim=-1, keepdim=True)
        x = (x - mean) / torch.sqrt(var + self.eps)
        return self.weight * x + self.bias
```

简化 RMSNorm:

```
class SimpleRMSNorm(nn.Module):
    def __init__(self, hidden_size: int, eps: float = 1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        rms = torch.sqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return self.weight * (x / rms)
```

对比：LayerNorm 要算均值和方差；RMSNorm 只算均方根。

6.8 Post-LN Transformer

原始 Transformer 常被描述为 Post-LN 结构。

一个子层可以写成：

$$h_{\ell+1} = \mathrm{LN}(\mathrm{bigl}(h_{\ell} + F_{\ell}(h_{\ell})\bigr))$$

也就是说，先做子层和残差相加，再做 LayerNorm。

Post-LN 的优点：

1. 每个 block 输出都被归一化。
2. 早期 Transformer 和很多 encoder-decoder 模型中表现很好。
3. 某些设置下最终性能可能较好。

Post-LN 的问题：

1. 深层训练更容易不稳定。
2. 初始化时靠近输出层的梯度可能较大。
3. 通常更依赖 learning rate warmup。
4. 堆很深时优化更困难。

Xiong 等人的分析指出，Post-LN 中 LayerNorm 的位置会导致初始化时部分梯度行为不佳，因此需要 warmup 避免早期大步更新导致不稳定。

6.9 Pre-LN Transformer

Pre-LN 结构把 norm 放到子层内部，也就是子层输入前：

$$h_{\ell+1} = h_{\ell} + F_{\ell}(\mathrm{LN}(h_{\ell}))$$

如果是 RMSNorm，就是：

$$h_{\ell+1} = h_{\ell} + F_{\ell}(\mathrm{RMSNorm}(h_{\ell}))$$

Pre-LN 的优点：

1. 残差主路径更接近 identity。
2. 梯度更容易沿 residual path 传播。
3. 深层训练更稳定。
4. 对 warmup 的依赖可能更小。
5. 现代 decoder-only LLM 常用。

Pre-LN 的代价：

1. 每个子层输出后不立即归一化，residual stream 尺度需要其他机制控制。
2. 某些任务或深度下，最终性能和 Post-LN/改进 Post-LN 有 trade-off。
3. 通常还会在最终输出前加 final norm。

现代 LLM 常见结构是：

```
h = h + Attention(RMSNorm(h))
h = h + MLP(RMSNorm(h))
logits = LMHead(RMSNorm(h))
```

这就是 LLaMA-like 模型常见模式。

6.10 为什么 Warmup 重要

Transformer 训练常用 learning rate warmup：

学习率从很小逐步升到目标值，再衰减。

Warmup 的作用是避免训练初期参数还没稳定时直接用大步长更新。

一个简单线性 warmup 可以写成：

$$\eta_t = \eta_{\max} \frac{t}{T_{\mathrm{warmup}}}, \quad 1 \leq t \leq T_{\mathrm{warmup}}$$

其中 $\eta_{\max}$ 是峰值学习率， T_{warmup} 是 warmup 步数。直觉上，warmup 把早期有效更新幅度压小，给参数尺度、AdamW 动量状态、混合精度和 residual stream 留出稳定时间。

在 Post-LN Transformer 中，初始化时某些层的梯度可能偏大。如果一开始用大学习率，容易导致训练不稳定。

Pre-LN 改善了梯度行为，因此对 warmup 的依赖可能降低。

但这不代表现代 LLM 不需要 warmup。大模型训练还涉及：

1. AdamW 状态建立。
2. 混合精度数值稳定。
3. 数据分布复杂。
4. 大 batch 训练。
5. 分布式通信误差。
6. MoE routing 稳定性。

所以实际工程中，warmup 仍然是常见保险措施。

6.11 DeepNorm：极深 Transformer 的残差缩放

DeepNet/DeepNorm 想解决的问题是：如何把 Transformer 堆到极深层，例如 1000 层。

它提出一种新的 normalization/residual scaling 方式，并配合理论推导的初始化，使模型更新保持有界。

DeepNorm 的目标可以理解为：

既想要 Post-LN 的性能，又想要 Pre-LN 的训练稳定性。

它说明一个事实：当层数极深时，简单的 residual + norm 可能不够，需要对 residual 分支尺度和初始化做系统设计。

虽然当前主流 decoder-only LLM 不一定直接使用 DeepNorm，但它对面试很有价值，因为它体现了深层 Transformer 训练的本质问题：

每层写入 residual stream 的更新幅度必须被控制，否则深度一大就会不稳定。

6.12 Admin：残差依赖和扰动放大

Understanding the Difficulty of Training Transformers 分析了 Transformer 训练困难，并指出一个重要现象：模型对 residual branch 的依赖会影响训练稳定性。

如果模型过度依赖 residual branch，小参数扰动可能被放大，造成输出显著变化，训练不稳定。

但如果 residual branch 依赖太弱，模型能力又受限。

Admin 初始化试图在训练早期降低不稳定，后期释放模型潜力。

这类工作说明：

训练稳定性不是只靠 norm，也和 residual branch、初始化、学习率、优化器共同决定。

6.13 Norm Placement 对表示的影响

Post-LN 和 Pre-LN 不只是训练技巧，也影响表示流动。

Post-LN：

$$h_{\ell+1} = \mathrm{Norm}(\mathrm{bigl}(h_{\ell} + F_{\ell}(h_{\ell})\bigr))$$

每层输出被标准化，尺度更统一，但残差路径被 norm 包住，梯度路径不再是纯 identity。

Pre-LN：

$$h_{\ell+1} = h_{\ell} + F_{\ell}(\mathrm{Norm}(h_{\ell}))$$

残差主路径更干净，梯度更容易直通，但 residual stream 的尺度可能随层数累积，需要 final norm、初始化、residual scaling 等共同控制。

所以不能只说 Pre-LN “更好”。更准确是：

Pre-LN 更利于深层稳定训练，因此在现代 LLM 中更常见；Post-LN 或 DeepNorm 变体在某些任务和结构中仍有价值。

6.14 大模型中的常见配置

现代 LLM 常见配置包括：

1. Pre-Norm。
2. RMSNorm。
3. Final RMSNorm。
4. AdamW。
5. Learning rate warmup + cosine/linear decay。
6. Gradient clipping 或 loss spike 监控。
7. bf16/fp8 混合精度。
8. Carefully designed initialization。

一个 LLaMA-like block 可以简化为：

```
class TransformerBlock(nn.Module):
    def __init__(self, attention, mlp, hidden_size):
        super().__init__()
        self.attn_norm = SimpleRMSNorm(hidden_size)
        self.ffn_norm = SimpleRMSNorm(hidden_size)
        self.attention = attention
        self.mlp = mlp

    def forward(self, x):
        x = x + self.attention(self.attn_norm(x))
        x = x + self.mlp(self.ffn_norm(x))
        return x
```

这段代码非常简单，但背后是多年训练稳定性经验的结果。

6.15 常见训练不稳定现象

训练不稳定可能表现为：

1. loss 突然 spike。
2. loss 变成 NaN。
3. gradient norm 爆炸。
4. activation norm 持续增大。
5. 某些层输出异常。
6. MoE router collapse。
7. attention logits 极端。
8. bf16/fp16 overflow。

排查时不能只看 norm。要同时看：

1. learning rate 和 warmup。
2. optimizer epsilon、beta、weight decay。
3. 初始化。
4. gradient clipping。
5. norm placement。
6. residual scaling。
7. data outlier。
8. mixed precision loss scaling。
9. distributed all-reduce 是否异常。

Norm 是稳定性的一部分，不是全部。

6.16 训练稳定性审计指标

训练稳定性不能只看 loss。对 residual、norm placement 和 warmup，至少要审计四类量。

第一，residual stream 的尺度：

$$a_{\ell} = \frac{\|h_{\ell}\|}{\sqrt{2}}$$

如果 a_{ℓ} 随层数持续放大，说明残差写入、初始化、norm placement 或学习率可能有问题。

第二，子层更新相对主路径的比例：

$$u_{\ell} = \frac{\|F_{\ell}(\text{Norm}(h_{\ell}))\|}{\|h_{\ell}\| + \epsilon}$$

u_{ℓ} 太大表示每层写入过猛，容易导致 residual stream 漂移；太小则可能表示子层几乎不工作。

第三，均值漂移：

$$m_{\ell} = \left| \frac{1}{d} \sum_{i=1}^d h_{\ell, i} \right|$$

LayerNorm 会显式 re-center，RMSNorm 不会显式减均值，因此 RMSNorm/Pre-Norm 结构尤其要配合 residual scaling、初始化和 final norm 控制尺度。

第四，早期更新压力。用一个 toy 指标表示 warmup 对早期大步更新的缓冲：

$$P_{\{\text{early}\}} = \eta_1 \max_{\ell} u_{\ell}$$

其中 η_1 是 warmup 第一步学习率。这个指标不是论文中的标准训练指标，只是面试和教学中用来解释“同样的梯度/更新比例，warmup 会降低早期参数扰动”的直觉。

一个简化门禁可以写成：

$$G_{\{\text{norm}\}} = \mathbb{1}_{\{a_{\max} \leq \tau_a, u_{\max} \leq \tau_u, P_{\{\text{early}\}} \leq \tau_p, \text{NaN}=0\}}$$

真实训练中还要补充 loss spike、grad norm、activation outlier、optimizer state、mixed precision overflow、data outlier、分布式 all-reduce 异常和 MoE router collapse。

6.16.1 最小可运行 Residual / Norm 稳定性审计 demo

下面的 demo 不依赖 PyTorch，只用标准库构造一个 toy residual stack，比较四种结构：

1. plain_no_residual：没有残差主路径，信息容易被每层重写。
2. residual_no_norm：有残差但没有归一化，尺度容易随层数漂移。
3. post_layernorm：每层输出被 LayerNorm 约束，但 early update pressure 仍依赖 warmup。
4. pre_rmsnorm_scaled：Pre-RMSNorm 加 residual scaling，保留残差主路径，同时控制每层写入幅度。

```
from math import sqrt, tanh
```

```
def mean(xs):
```

```
    return sum(xs) / len(xs)
```

```
def l2_norm(xs):
```

```
    return sqrt(sum(x * x for x in xs))
```

```
def layer_norm(xs, eps=1e-6):
```

```
    mu = mean(xs)
```

```
    var = mean([(x - mu) ** 2 for x in xs])
```

```
    scale = sqrt(var + eps)
```

```
    return [(x - mu) / scale for x in xs]
```

```
def rms_norm(xs, eps=1e-6):
```

```
    rms = sqrt(mean([x * x for x in xs]) + eps)
```

```
    return [x / rms for x in xs]
```

```

def sublayer(xs, strength):
    return [strength * tanh(x + 0.2 * (i + 1)) for i, x in enumerate(xs)]

def add(xs, ys):
    return [x + y for x, y in zip(xs, ys)]

def scale(xs, factor):
    return [factor * x for x in xs]

def run_stack(mode, depth=12, strength=0.8, residual_scale=1.0):
    h = [1.0, -0.5, 0.25, 2.0]
    rows = []
    for layer in range(1, depth + 1):
        before_norm = l2_norm(h)
        if mode == "plain_no_residual":
            update = sublayer(h, strength)
            h = update
        elif mode == "residual_no_norm":
            update = sublayer(h, strength)
            h = add(h, scale(update, residual_scale))
        elif mode == "post_layernorm":
            update = sublayer(h, strength)
            h = layer_norm(add(h, scale(update, residual_scale)))
        elif mode == "pre_rmsnorm_scaled":
            update = sublayer(rms_norm(h), strength)
            h = add(h, scale(update, residual_scale))
        else:
            raise ValueError(mode)

        update_norm = l2_norm(update)
        rows.append(
            {
                "layer": layer,
                "before": round(before_norm, 4),
                "update_ratio": round(update_norm / max(before_norm, 1e-9), 4),
                "after": round(l2_norm(h), 4),
                "mean_abs": round(abs(mean(h)), 4),
            }
        )
    return rows

def audit(rows, lr_peak, warmup_steps):
    max_norm = max(row["after"] for row in rows)
    max_update_ratio = max(row["update_ratio"] for row in rows)
    first_lr = lr_peak / warmup_steps
    early_update_pressure = first_lr * max_update_ratio
    return {
        "final_norm": rows[-1]["after"],
        "max_norm": round(max_norm, 4),
    }

```

```

    "max_update_ratio": round(max_update_ratio, 4),
    "final_mean_abs": rows[-1]["mean_abs"],
    "early_update_pressure": round(early_update_pressure, 6),
    "gate_pass": (
        max_norm <= 6.0
        and max_update_ratio <= 0.9
        and early_update_pressure <= 0.0005
    ),
}

configs = {
    "plain_no_residual": ("plain_no_residual", 1.0),
    "residual_no_norm": ("residual_no_norm", 1.0),
    "post_layernorm": ("post_layernorm", 1.0),
    "pre_rmsnorm_scaled": ("pre_rmsnorm_scaled", 1 / sqrt(24)),
}

summaries = {}
for name, (mode, residual_scale) in configs.items():
    rows = run_stack(mode, residual_scale=residual_scale)
    summaries[name] = audit(rows, lr_peak=0.003, warmup_steps=20)

pre_rows = run_stack("pre_rmsnorm_scaled", residual_scale=1 / sqrt(24))
post_rows = run_stack("post_layernorm")

print("summaries=", summaries)
print("pre_rmsnorm_tail=", pre_rows[-3:])
print("post_ln_no_warmup=", audit(post_rows, lr_peak=0.003, warmup_steps=1))
print("post_ln_with_warmup=", audit(post_rows, lr_peak=0.003, warmup_steps=20))

```

预期输出:

```

summaries= {'plain_no_residual': {'final_norm': 1.2631, 'max_norm': 1.2634, 'max_update_ratio': 1.0431, 'final_mean_abs': 0.05, 'gate_pass': False}, 'post_layernorm': {'final_norm': 2.0, 'max_norm': 2.0, 'max_update_ratio': 0.6247, 'final_mean_abs': 0.05, 'gate_pass': True}, 'pre_rmsnorm_scaled': {'final_norm': 5.0581, 'max_norm': 5.0581, 'max_update_ratio': 0.4972, 'final_mean_abs': 0.05, 'gate_pass': True}}
pre_rmsnorm_tail= [{'layer': 10, 'before': 4.3352, 'update_ratio': 0.2879, 'after': 4.5743, 'mean_abs': 1.7837}, {'layer': 11, 'before': 4.5743, 'update_ratio': 0.2879, 'after': 4.8114, 'mean_abs': 1.7837}, {'layer': 12, 'before': 4.8114, 'update_ratio': 0.2879, 'after': 5.0485, 'mean_abs': 1.7837}]
post_ln_no_warmup= {'final_norm': 2.0, 'max_norm': 2.0, 'max_update_ratio': 0.6247, 'final_mean_abs': 0.0, 'early_update_pressure': 0.001874, 'gate_pass': False}
post_ln_with_warmup= {'final_norm': 2.0, 'max_norm': 2.0, 'max_update_ratio': 0.6247, 'final_mean_abs': 0.0, 'early_update_pressure': 0.0005, 'gate_pass': True}

```

这个 demo 要读出四点:

1. plain_no_residual 的 max_update_ratio 超过 1, 说明每层都可能大幅重写表示, 缺少稳定主路径。
2. residual_no_norm 虽然有残差, 但 max_norm 增长到 19.7276, 说明残差写入没有尺度控制会漂移。
3. post_layernorm 的输出尺度被 LayerNorm 固定住, 但如果不 warmup, early_update_pressure 会升到 0.001874, 门禁失败。
4. pre_rmsnorm_scaled 保留残差主路径, 并用 RMSNorm + residual scaling 控制更新比例; 它没有 re-centering, 所以 final_mean_abs 不为 0, 这也是为什么还需要 final norm、初始化和监控。

6.17 常见误区

误区一: LayerNorm 和 BatchNorm 差不多。

更准确: LayerNorm 对单样本 hidden 维做归一化, 不依赖 batch size, 训练和推理一致, 更适合 Transformer。

误区二: RMSNorm 只是 LayerNorm 的近似。

更准确：RMSNorm 去掉 re-centering，保留 re-scaling，计算更简单，在现代 LLM 中被广泛采用。

误区三：Pre-LN 一定全面优于 Post-LN。

更准确：Pre-LN 更稳定、更适合深层 LLM 训练；Post-LN 也有性能优势和改进变体，DeepNorm 等方法试图结合两者优点。

误区四：Warmup 是经验 trick，没有理论意义。

更准确：LayerNorm 位置和初始化梯度行为能解释 warmup 的必要性，尤其是 Post-LN 设置下。

误区五：Residual 只是简单相加。

更准确：Residual 是深层网络信息和梯度流动的主干，残差分支尺度决定训练稳定性。

6.18 面试题

题 1：Residual connection 为什么重要？

参考回答：

Residual connection 让每层学习输入上的增量，而不是从零学习完整映射。它提供信息直通路和梯度直通路，使深层网络更容易训练。

题 2：LayerNorm 和 BatchNorm 有什么区别？

参考回答：

BatchNorm 使用 batch 维度统计，训练和推理行为不同，并依赖 batch size。LayerNorm 对单个样本的 hidden dimension 做归一化，训练和推理行为一致。

题 3：RMSNorm 和 LayerNorm 有什么区别？

参考回答：

LayerNorm 会减去均值并除以标准差，提供 re-centering 和 re-scaling。RMSNorm 不减均值，只用 root mean square 做尺度归一化，计算更简单，实践中在大模型上效果很好，因此 LLaMA 等现代 LLM 常用 RMSNorm。

题 4：Pre-LN 和 Post-LN 的区别是什么？

参考回答：

Post-LN 是先 residual add 再 LayerNorm，即 $x = \text{Norm}(x + \text{Sublayer}(x))$ 。Pre-LN 是先 Norm 再进入子层，然后 residual add，即 $x = \text{Norm}(x) + \text{Sublayer}(x)$ 。LN 每层输出被归一化，但深层训练更容易梯度不稳定，通常依赖 warmup。Pre-LN 保留更干净的 residual 梯度路径，训练更稳定，因此 LLaMA 等现代 LLM 常用 Pre-Norm。

题 5：为什么 Post-LN 更依赖 warmup？

参考回答：

研究表明 Post-LN Transformer 在初始化时，靠近输出层的参数梯度期望可能较大。如果一开始使用较大学习率，容易导致训练不稳定。Pre-LN 的梯度行为更稳定，因此对 warmup 的依赖较小，但实际大模型训练仍常使用 warmup 作为稳定措施。

题 6：DeepNorm 解决什么问题？

参考回答：

DeepNorm 想解决极深 Transformer 的训练稳定性问题。它通过修改 residual connection 的归一化和缩放方式，并配套理论初始化，使 Post-LN 的性能和 Pre-LN 的稳定性，把 Transformer 扩展到上百甚至上千层。

6.19 小练习

1. 画出 Post-LN 和 Pre-LN Transformer block 的结构图。
2. 用 PyTorch 实现 LayerNorm 和 RMSNorm, 并比较它们的计算差异。
3. 解释为什么 LayerNorm 不依赖 batch size。
4. 说明为什么 residual branch 的输出尺度过大可能导致训练不稳定。
5. 阅读 On Layer Normalization in the Transformer Architecture 摘要, 解释 warmup 和 norm placement 的关系。
6. 阅读 RMSNorm 摘要, 解释 re-centering 和 re-scaling 的区别。
7. 设计一个训练稳定性监控面板, 至少包含 loss、grad norm、activation norm、learning rate、NaN 统计。
8. 运行本章 Residual / Norm 稳定性审计 demo, 解释 residual_no_norm 和 post_ln_no_warmup 分别为什么门禁失败。

6.20 本章总结

本章讲了 Residual、LayerNorm、RMSNorm、Pre-Norm 和训练稳定性。

核心结论:

1. Residual connection 是深层 Transformer 信息和梯度流动的主干。
2. Attention 和 FFN 都是在 residual stream 上写入增量信息。
3. LayerNorm 对单样本 hidden 维归一化, 不依赖 batch size, 适合 Transformer。
4. RMSNorm 去掉均值中心化, 只做尺度归一化, 计算更简单, 现代 LLM 常用。
5. Post-LN 是先 residual add 再 norm, Pre-LN 是先 norm 再子层计算后 residual add。
6. Post-LN 往往更依赖 warmup, Pre-LN 深层训练更稳定。
7. DeepNorm、Admin 等方法说明 residual scaling、初始化和 norm placement 共同决定训练稳定性。
8. 大模型训练稳定性不是单一模块决定的, 而是 norm、residual、初始化、optimizer、learning rate、precision 和数据共同作用。

下一章会进入 Position Encoding, 总览 Sinusoidal、Learned、Relative、RoPE、ALiBi 等位置编码方法, 解释 Transformer 为什么必须显式处理位置信息。

第七章: Position Encoding 总览: Sinusoidal、Learned、Relative、RoPE、ALiBi

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修主要对齐 Transformer 原论文、Shaw et al. relative position representation、T5 relative position bias、RoFormer / RoPE、ALiBi、Position Interpolation 和 YaRN 等公开论文。写作时只把这些资料中已经公开且可复核的机制写成确定结论; 不同模型里的 RoPE base、scaling 配置、position id 细节、长上下文微调配方和评测结论不能互相泛化。

本讲的精修重点有三点:

1. 把位置编码的核心公式改成 GitHub Markdown 更稳定的 fenced math block, 避免把代码变量名塞进 LaTeX 的 text 宏。
2. 补一个 0 依赖 Python demo, 用同一段代码检查 sinusoidal、RoPE、ALiBi、Position Interpolation 和 position id 风险。
3. 把本讲新增的 Position Encoding Gate、Position ID Audit、RoPE Scaling 等概念同步到百科、题库、练习、术语表、项目清单和知识图谱。

7.1 本章定位

前面几章已经讲过 attention、FFN、residual 和 normalization。现在进入 Transformer 架构里另一个非常关键、也非常容易被面试追问的主题: 位置编码。

Self-Attention 的核心操作是让每个 token 根据 Q/K/V 和其他 token 交互。这个机制本身并不知道 token 出现在第几个位置。如果不给模型额外的位置信息，下面两个序列在 attention 眼里很容易变成“同一批 token 的不同排列”：

我 喜欢 机器 学习
机器 学习 喜欢 我

但自然语言、代码、数学公式、对话、多模态序列都强依赖顺序。位置编码要解决的问题就是：

如何把 token 的顺序、距离和相对关系注入 Transformer，同时不破坏 attention 的并行性和可扩展性。

本章要回答的问题是：

1. 为什么 Self-Attention 需要位置编码。
2. Sinusoidal position encoding 为什么用 sin/cos。
3. learned absolute position embedding 有什么优缺点。
4. relative position representation 为什么比绝对位置更贴近 attention。
5. RoPE 如何把位置编码成 Q/K 的旋转。
6. ALiBi 为什么可以不用显式 position embedding。
7. 位置编码如何影响长上下文扩展。
8. 现代 LLM 为什么大量采用 RoPE，并围绕 RoPE 做 scaling、interpolation 和 YaRN 等扩展。

7.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need. 提出 Transformer，并使用 sinusoidal positional encoding，也讨论 learned positional embedding 的相近效果。
2. Shaw et al., 2018, Self-Attention with Relative Position Representations. 提出在 self-attention 中显式引入相对位置表示。
3. Huang et al., 2018, Music Transformer. 改进相对位置 attention 的内存实现，使长序列音乐建模更可行。
4. Dai et al., 2019, Transformer-XL. 使用相对位置编码和 segment-level recurrence，影响后续长上下文建模。
5. Raffel et al., 2020, Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. T5 使用 relative position bias，简化相对位置信息注入方式。
6. Su et al., 2021, RoFormer: Enhanced Transformer with Rotary Position Embedding. 提出 RoPE，把绝对位置编码成旋转矩阵，同时在 attention 中体现相对位置关系。
7. Press et al., 2021, Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. 提出 ALiBi，在 attention score 上加入与距离成比例的线性 bias。
8. Touvron et al., 2023, LLaMA: Open and Efficient Foundation Language Models. LLaMA 使用 RoPE，推动 RoPE 成为开源 LLM 常见配置。
9. Chen et al., 2023, Extending Context Window of Large Language Models via Positional Interpolation. 提出通过位置插值扩展 RoPE 模型上下文窗口。
10. Peng et al., 2023, YaRN: Efficient Context Window Extension of Large Language Models. 提出更高效的 RoPE 上下文扩展方法。

需要说明的是，位置编码是一个仍在演进的领域。不同模型、不同数据、不同上下文长度、不同 attention pattern 下，最佳选择可能不同。本章重点讲清主流方法的机制和工程 trade-off，不把某一种方法绝对化。

7.3 为什么 Self-Attention 本身不知道顺序

先看一个没有位置编码的 self-attention。

设输入 token embedding 组成矩阵：

$X = [x_1, x_2, \dots, x_n] \in \mathbb{R}^{n \times d}$

attention 计算可以写成：

$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$

$$\mathrm{Attention}(X)=\mathrm{softmax}\left(\frac{QK^{\mathrm{top}}}{\sqrt{d_k}}\right)V$$

如果把输入 token 的顺序整体打乱，只要 Q/K/V 也跟着同样打乱，attention 输出也会按同样方式打乱。也就是说，self-attention 对序列顺序没有内置的先后概念。

更直观地说，attention 只会问：

这个 token 和那个 token 的内容相似吗？应该从那个 token 读取多少信息？

它不会天然知道：

那个 token 在我前面还是后面？

距离我 1 个位置还是 100 个位置？

这是句首、句中还是句尾？

所以 Transformer 必须通过某种方式注入位置。

常见注入位置的方式有三类：

1. 加到输入表示上，例如 sinusoidal 和 learned absolute position embedding。
2. 加到 attention score 或 attention value 上，例如 relative position representation、relative position bias、ALiBi。
3. 作用到 Q/K 几何结构上，例如 RoPE。

这三类方式的差异，决定了它们在外推、长上下文、KV cache、实现复杂度上的不同表现。

7.4 绝对位置编码：把“第几个位置”加到 token 上

最直接的方法是为每个位置准备一个向量，然后把它加到 token embedding 上：

$$h_i = e_i + p_i$$

其中 e_i 是第 i 个 token 的内容 embedding， p_i 是第 i 个位置的位置向量， h_i 是送入 Transformer block 的表示。

这样第 i 个 token 的输入表示里同时包含两部分信息：

这个 token 是什么

这个 token 在第几个位置

绝对位置编码的优点是简单。Transformer 原论文就是这样做的。

但它也有一个天然问题：attention 里真正重要的往往不只是绝对位置，而是相对关系。

例如语言模型预测下一个 token 时，经常需要知道：

前一个词是什么？

前 5 个 token 里有没有主语？

这个右括号对应的是多远之前的左括号？

这些问题更像是“距离”和“相对位置”，而不是“第 137 个位置”。

7.5 Sinusoidal Position Encoding

Transformer 原论文使用固定的 sinusoidal position encoding。公式是：

$$\mathrm{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right)$$

$$\mathrm{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d}}\right)$$

其中：

1. p 是 token 位置。

2. i 是维度索引。
3. d 是 hidden size。

它的直觉是：

用不同频率的 $\sin/\cos$ 波表示位置。
 低维度变化快，表示局部位置差异。
 高维度变化慢，表示长距离位置趋势。

可以把它想象成给每个位置分配一组“多频率坐标”。位置 10、位置 11 在高频维度上会有明显差别；位置 1000、位置 2000 在低频维度上仍能体现大尺度差异。

Sinusoidal 的一个重要性质是：固定公式可以生成任意长度的位置向量。理论上，即使训练时只见过 512 长度，推理时也可以生成 1024、2048 位置的编码。

但这不等于模型一定能很好外推。原因是：

位置向量能生成，不代表模型在训练中学会了如何使用更长位置上的 attention pattern。

这是面试中很容易混淆的点。

7.6 Sinusoidal 为什么用 $\sin$ 和 $\cos$

$\sin/\cos$ 不是随便选的。它有几个好处。

第一，值域稳定。

$\sin$ 和 $\cos$ 的输出始终在 $[-1, 1]$

不会因为位置变大而数值爆炸。

第二，多频率可以覆盖不同尺度。

短距离和长距离都能有可区分的模式。

第三， $\sin/\cos$ 的相位关系可以帮助模型推断相对位置。

对于同一频率，有：

$$\sin(a+b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

$$\cos(a+b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

这意味着位置 $\text{pos} + \text{offset}$ 的编码，可以通过位置 pos 和 offset 的三角函数关系表达。也就是说，固定正弦位置编码天然包含某种相对位移结构。

不过要注意：Transformer 原论文里 sinusoidal 是加到输入 embedding 上，而不是直接加到 attention score 中。模型是否真正利用这些相对位移结构，要靠训练学出来。

7.7 Learned Absolute Position Embedding

另一种常见方法是 learned absolute position embedding。

做法很简单：

```
position_embedding = nn.Embedding(max_position, d_model)
h_i = token_embedding_i + position_embedding_i
```

数学上仍然是：

$$h_i = e_i + p_i, \quad p_i \in \mathbb{R}^d$$

这和词表 embedding 类似，只不过词表 id 换成了位置 id。

优点：

1. 简单，容易实现。
2. 模型可以根据数据自动学习每个位置的表示。

3. 在固定最大长度任务上通常效果不错。

缺点：

1. 需要预先设定 `max_position`。
2. 超过训练长度的位置没有学过，不能自然外推。
3. 对长上下文扩展不友好。
4. 学到的位置向量可能过拟合训练长度分布。

GPT-2、BERT 等早期 Transformer 模型常使用 learned absolute position embedding。对于固定上下文长度的模型，这种方法足够直接。但随着 LLM 上下文窗口从几百、几千扩展到几万、几十万，learned absolute embedding 的局限变得明显。

7.8 绝对位置编码的核心限制

绝对位置编码最大的问题是：它把位置当作一个“绝对编号”。

但很多语言规律具有平移不变性。

例如：

第 20 个 token 关注第 19 个 token

第 2000 个 token 关注第 1999 个 token

这两个模式本质上都是“关注前一个 token”。如果使用绝对位置，模型可能需要从不同位置编号中学出同一种相对规律。

相对位置方法则更直接：

不问你在第几个位置，只问你和我相距多远。

这就是 relative position representation 的动机。

7.9 Relative Position Representation

Shaw et al. 提出的相对位置表示，把 token i 和 token j 之间的距离引入 attention。

普通 attention score 是：

$$s_{ij} = q_i \cdot k_j$$

加入相对位置后，可以变成类似：

$$s_{ij} = q_i \cdot \text{top}(k_j + r_{i-j})$$

其中 r_{i-j} 是相对距离 $i-j$ 对应的位置向量。

直觉是：

token j 的内容重要不重要，不只取决于它的内容 k_j ，也取决于它距离当前 token i 有多远。

比如在语言模型里，距离当前 token 很近的 token 通常更重要；在括号匹配、代码缩进、长距离依赖中，某些特定距离模式也可能重要。

相对位置的优点：

1. 更符合 attention 的 pairwise 结构。
2. 可以直接建模 token 两两之间的距离。
3. 对长度泛化通常比 learned absolute 更自然。

相对位置的缺点：

1. 实现比绝对位置复杂。
2. 朴素实现可能引入额外的 $n \times n$ 相对位置张量。
3. 长序列下内存和计算压力更大。

Music Transformer 的一个贡献就是改进相对 attention 的实现，降低中间相对信息的内存开销，使更长序列的音乐生成更可行。

7.10 Relative Position Bias

后续很多模型使用更简单的 relative position bias。

它不再为每个相对距离引入完整向量，而是给 attention score 加一个 bias：

$$s_{ij} = \frac{q_i^{\top} k_j}{\sqrt{d_h}} + b_{\text{bucket}(i-j)}$$

其中 $\text{bucket}(i-j)$ 表示把相对距离分桶。

T5 就使用了类似思路。距离分桶的意义是：

近距离区分得细一点，远距离区分得粗一点。

例如：

距离 1、2、3、4 分别有不同 bias

距离 128 到 255 共享一个 bucket

距离 256 以上共享更粗的 bucket

这样做的好处是参数少、实现相对简单，并且贴合 attention score。

Relative bias 和 relative representation 的区别可以简单理解为：

relative representation: 给距离一个向量，影响 Q/K/V 的交互。

relative bias: 给距离一个标量，直接偏置 attention score。

relative bias 更轻量，但表达能力也更受限。

7.11 RoPE: Rotary Position Embedding

RoPE 是现代 decoder-only LLM 中非常常见的位置编码方法。LLaMA、Qwen、ChatGLM、Baichuan 等大量模型都采用或改造了 RoPE。

RoPE 的核心思想是：

不要把位置向量简单加到 token embedding 上，而是根据位置对 Q 和 K 做旋转。

假设 Q/K 的某两个维度组成一个二维平面：

$$x = [x_1, x_2]^{\top}$$

RoPE 会根据位置 pos 和频率 theta 做旋转：

$$\begin{bmatrix} x'_1 \\ x'_2 \end{bmatrix} = \begin{bmatrix} \cos(p\theta) & -\sin(p\theta) \\ \sin(p\theta) & \cos(p\theta) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

也就是把向量在二维平面里转一个角度。不同维度对使用不同频率。

用矩阵表示，就是：

```
R_p=
\begin{bmatrix}
\cos(p\theta) & -\sin(p\theta) \\
\sin(p\theta) & \cos(p\theta)
\end{bmatrix}
```

然后：

$q'_i = R_{iq_i}, \quad k'_j = R_{jk_j}$

attention score 变成：

$s_{ij} = (R_{iq_i})^{\top} (R_{jk_j})$

旋转矩阵有一个关键性质：

$R_i^{\top} R_j = R_{j-i}$

所以：

$s_{ij} = q_i^{\top} R_{j-i} k_j$

这说明 RoPE 虽然用的是绝对位置 i 和 j 分别旋转 Q/K ，但最终 Q/K 点积里自然出现了相对位置 $j - i$ 。

这就是 RoPE 最重要的性质：

用绝对位置实现，点积时体现相对位置。

7.12 RoPE 为什么适合 decoder-only LLM

RoPE 在现代 LLM 中流行，不只是因为效果好，还因为它非常适合 decoder-only attention 的工程形态。

第一，RoPE 只作用在 Q/K 上，不作用在 V 上。

这很自然，因为位置信息主要影响“该关注谁”，也就是 attention score。真正被读取的信息 V 可以保持内容表示。

第二，RoPE 兼容 causal attention。

decoder-only 模型每个位置只能看历史 token。RoPE 通过 Q/K 旋转注入位置，不改变 causal mask。

第三，RoPE 兼容 KV cache。

推理时历史 K/V 会缓存。RoPE 对 K 的旋转可以在生成 K 时完成并缓存下来。后续新 token 只需要对新的 Q/K 应用对应位置旋转。

第四，RoPE 有较好的相对位置归纳偏置。

语言模型常常关心“距离当前 token 多远”，RoPE 在 attention score 中自然产生相对距离项。

第五，RoPE 可以围绕频率和位置索引做上下文扩展。

很多长上下文方法本质上都在调整 RoPE 的位置尺度或频率分布。

7.13 RoPE 的实现直觉

在代码中，RoPE 通常不会真的构造旋转矩阵，而是用 $\cos$ 、 $\sin$ 和半维度交换来实现。

伪代码如下：

```
def rotate_half(x):
    x1 = x[..., ::2]
    x2 = x[..., 1::2]
    return stack([-x2, x1], dim=-1).flatten(-2)
```

```
def apply_rope(x, cos, sin):
    return x * cos + rotate_half(x) * sin
```

对 Q/K 应用：

```
q = apply_rope(q, cos[position_ids], sin[position_ids])
k = apply_rope(k, cos[position_ids], sin[position_ids])
```

需要注意几个工程细节：

1. cos/sin 通常可以预计算。
2. position_ids 必须和真实 token 位置一致。
3. 使用 KV cache 时，新 token 的 position id 要接在历史长度之后。
4. padding、packed sequence、sliding window、prefix cache 都会影响 position id 设计。
5. 如果只对一部分 head dimension 使用 RoPE，也要保证实现和模型配置一致。

7.14 ALiBi: Attention with Linear Biases

ALiBi 是另一条路线。它不把位置编码加到 embedding，也不旋转 Q/K，而是直接修改 attention score。

普通 attention score：

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_h}}$$

ALiBi 加上与距离成比例的线性惩罚：

$$s_{ij}^{(h)} = \frac{q_i \cdot k_j}{\sqrt{d_h}} - m_h |i - j|$$

其中 m_h 是第 h 个 head 的斜率。不同 head 使用不同斜率。

对 causal LM 来说，当前位置 i 只能看过去位置 $j \leq i$ ，距离可以写成：

$$\Delta_{ij} = i - j, \quad j \leq i$$

因此 causal ALiBi 也常写成：

$$s_{ij}^{(h)} = \frac{q_i \cdot k_j}{\sqrt{d_h}} - m_h (i - j), \quad j \leq i$$

ALiBi 的直觉是：

距离越远，attention score 越被惩罚。

不同 head 的惩罚强度不同，有些 head 更偏局部，有些 head 可以看得更远。

ALiBi 的优点：

1. 不需要 position embedding 参数。
2. 不需要预设最大位置 embedding 表。
3. 对长度外推友好。
4. 实现上只是 attention score bias。
5. 训练短上下文、测试更长上下文时有较好表现。

缺点：

1. 它强引入 recency bias，即越近越容易被关注。
2. 对需要远距离精确匹配的任务，线性惩罚可能过强。
3. 它不像 RoPE 那样在 Q/K 几何结构中编码相对相位。
4. 在现代 LLM 生态中，RoPE 的使用更广，相关优化和扩展方法更多。

ALiBi 的核心价值是说明：

位置编码不一定要是 embedding，也可以是 attention score 上的结构化 bias。

7.15 Sinusoidal、Learned、Relative、RoPE、ALiBi 对比

可以从几个维度比较这些方法。

注入位置

Sinusoidal: 加到输入 embedding
Learned absolute: 加到输入 embedding
Relative representation: 加到 attention 的 K/V 或 score 交互
Relative bias: 加到 attention score
RoPE: 旋转 Q/K
ALiBi: 加到 attention score

绝对还是相对

Sinusoidal: 绝对位置为主, 但公式包含可推导的相对结构
Learned absolute: 绝对位置
Relative representation: 相对位置
Relative bias: 相对位置
RoPE: 用绝对位置实现, 相对位置体现在 Q/K 点积中
ALiBi: 相对距离 bias

长度外推

Learned absolute: 较弱, 超过训练长度没有学过 embedding
Sinusoidal: 公式可外推, 但模型行为未必稳定
Relative position: 通常比 learned absolute 更自然, 但受实现和 bucket 影响
RoPE: 有一定外推能力, 但长距离相位可能失配, 需要 scaling/interpolation
ALiBi: 外推友好, 设计目标就是 train short, test long

工程复杂度

Learned absolute: 最低
Sinusoidal: 低
ALiBi: 低到中, 需 attention bias 支持
Relative bias: 中
RoPE: 中, 需处理 Q/K reshape、cos/sin、position id、cache
Relative representation: 中到高, 朴素实现可能内存重

现代 LLM 使用情况

早期 encoder/decoder: sinusoidal 或 learned absolute 常见
BERT/GPT-2: learned absolute 常见
T5: relative position bias
LLaMA-like decoder-only LLM: RoPE 常见
部分长上下文/实验模型: ALiBi 或 RoPE scaling 变体

7.16 位置编码和 KV Cache

位置编码与 KV cache 的关系非常重要, 因为现代 LLM 推理几乎都依赖 KV cache。

对于 learned absolute 或 sinusoidal, 如果位置编码只加在输入 embedding 上, 那么每个 token 的 K/V 已经包含位置信息, 缓存即可。

对于 RoPE, K 在进入 attention score 前会被旋转。推理时通常缓存旋转后的 K:

历史 token 生成 K
根据历史位置对 K 应用 RoPE
缓存旋转后的 K
新 token 生成 Q/K
根据新位置对 Q/K 应用 RoPE
新 Q 与缓存 K 做 attention

如果 position id 出错，模型表现会明显异常。例如：

1. 继续生成时 position id 从 0 重新开始。
2. prefix cache 拼接后位置没有正确偏移。
3. packed batch 中不同样本的位置混在一起。
4. sliding window 中局部位置和全局位置处理不一致。

这些问题在训练 loss 上未必容易发现，但在推理长文本、续写、RAG、多轮对话中会造成质量下降。

7.17 位置编码和长上下文扩展

长上下文扩展不是简单把 max_position 改大。

对于 RoPE 模型，如果训练时最大长度是 2048，推理时直接用 8192，可能出现问题。原因是 RoPE 的旋转角度随位置增长，远超训练范围后，某些频率维度上的相位关系会进入模型没见过的区域。

直观地说：

模型训练时只学过 0 到 2048 的位置相位模式。

推理时突然给它 8192 的相位，attention score 分布可能失真。

常见扩展思路包括：

1. 位置插值。
2. RoPE scaling。
3. NTK-aware scaling。
4. YaRN。
5. 长上下文继续预训练或微调。

7.18 Position Interpolation

Position Interpolation 的思路是：

不要让位置索引直接外推到训练范围之外，而是把更长上下文的位置压缩回原训练范围。

假设原模型训练长度是 $L = 2048$ ，现在希望扩展到 $L' = 8192$ 。

直接外推：

$p=0,1,2,\ldots,8191$

位置插值：

$p'=p\cdot\frac{L_{\mathrm{train}}}{L_{\mathrm{target}}}$

于是最大位置从 8191 压回接近 2048。

这样做的好处是避免让 RoPE 进入过大的未训练相位区域。论文中也强调，相比直接 extrapolation，interpolation 的 attention score 上界更稳定。

但它也有代价：

位置分辨率变低。

原来相邻 token 的位置差是 1，现在可能变成 0.25。模型需要通过少量长上下文微调适应这种新的位置尺度。

7.19 YaRN

YaRN 是 RoPE 长上下文扩展的一类高效方法。它的目标是用更少 token、更少训练步数，把已有 RoPE LLM 扩展到更长上下文。

从工程角度看，YaRN 的意义是：

长上下文扩展不一定要从头预训练，也可以在已有 RoPE 模型上做位置编码改造和少量继续训练。

这类方法通常围绕以下问题设计：

1. 不同频率维度应该如何缩放。
2. 短上下文能力如何尽量保留。
3. 长上下文中 attention score 分布如何避免异常。
4. 需要多少长上下文数据继续训练。
5. passkey retrieval 这类任务能否通过，不代表真实长文理解一定强。

这一点非常重要。长上下文模型评估不能只看“能不能在 100K token 中找出一个 key”。真实任务还包括：

1. 多文档归纳。
2. 长代码仓库理解。
3. 跨段落推理。
4. 多轮对话一致性。
5. 长上下文中的抗干扰能力。

位置编码扩展只是长上下文能力的一部分，attention pattern、训练数据、任务分布、检索增强、推理策略都会影响最终效果。

7.20 位置编码审计指标与最小 demo

工程里位置编码最容易出错的地方不是公式本身，而是 position id、KV cache、packed sequence 和长上下文配置是否一致。可以把位置编码审计抽象成一个门禁：

```
G_{\mathrm{pos}}=
\mathbf{1}[C_{\mathrm{cache}}=1]\cdot
\mathbf{1}[C_{\mathrm{pack}}=1]\cdot
\mathbf{1}[C_{\mathrm{long}}=1]
```

其中：

```
C_{\mathrm{cache}}=
\mathbf{1}[p_{\mathrm{new}},0]=L_{\mathrm{cache}}
```

表示使用 KV cache 继续生成时，新 token 的第一个 position id 必须接在已有缓存长度之后。

```
C_{\mathrm{long}}=
\mathbf{1}[L_{\mathrm{target}}\leq L_{\mathrm{train}}]
+\mathbf{1}[L_{\mathrm{target}}>L_{\mathrm{train}}]\cdot
\mathbf{1}[S_{\mathrm{rope}}=1]
```

这里 S_{rope} 表示已经显式配置 RoPE scaling、Position Interpolation、YaRN 或继续训练策略。这个公式不是说某个策略一定有效，而是提醒：目标上下文超过训练上下文时，不能只改 `max_position`。

下面 demo 用纯 Python 同时检查五件事：

1. sinusoidal 位置向量的多频率结构。
2. RoPE 点积对相同相对距离给出相同分数。
3. ALiBi 对更远历史 token 给出更负的 bias。
4. Position Interpolation 如何把 8192 长度映射回 2048 训练尺度。
5. position id 审计如何发现 cache reset、packed sequence 泄漏和长上下文无 scaling。

```
import math
```



```

def sinusoidal_position(pos, dim=8, base=10000.0):
    values = []
    for i in range(dim // 2):
        angle = pos / (base ** (2 * i / dim))
        values.extend([math.sin(angle), math.cos(angle)])
    return values

def rotate_pair(x, pos, theta):
    angle = pos * theta
    c = math.cos(angle)
    s = math.sin(angle)
    return [x[0] * c - x[1] * s, x[0] * s + x[1] * c]

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def rope_score(q, k, i, j, theta=0.125):
    return dot(rotate_pair(q, i, theta), rotate_pair(k, j, theta))

def alibi_bias(query_pos, key_pos, slope):
    if key_pos > query_pos:
        return None
    return -slope * (query_pos - key_pos)

def interpolate_position(pos, train_length, target_length):
    return pos * train_length / target_length

def audit_position_plan(cache_len, new_position_ids, packed_segments,
                        train_context, target_context, rope_scaling):
    issues = []
    if cache_len > 0 and new_position_ids and new_position_ids[0] != cache_len:
        issues.append("cache_position_reset")
    for name, ids in packed_segments[1:]:
        if ids and ids[0] != 0:
            issues.append("packed_sequence_position_leak")
            break
    if target_context > train_context and rope_scaling is None:
        issues.append("over_train_context_without_scaling")
    return issues, len(issues) == 0

q = [1.0, 0.2]
k = [0.4, -0.7]
rope_scores = {
    "score_2_5": round(rope_score(q, k, 2, 5), 6),
    "score_10_13": round(rope_score(q, k, 10, 13), 6),
    "score_2_6": round(rope_score(q, k, 2, 6), 6),
}

```

```

alibi = {
    "key_6": alibi_bias(7, 6, 0.125),
    "key_2": alibi_bias(7, 2, 0.125),
    "key_8": alibi_bias(7, 8, 0.125),
}

scaled_positions = [
    round(interpolate_position(p, 2048, 8192), 2)
    for p in [0, 1024, 4096, 8191]
]

issues, gate_pass = audit_position_plan(
    cache_len=4,
    new_position_ids=[0, 1],
    packed_segments=[("doc_a", [0, 1, 2]), ("doc_b", [3, 4])],
    train_context=2048,
    target_context=8192,
    rope_scaling=None,
)

print("sin_pos_3=", tuple(round(x, 4) for x in sinusoidal_position(3)))
print("rope_scores=", rope_scores)
print("alibi_bias=", alibi)
print("pi_scaled_positions=", scaled_positions)
print("audit_issues=", issues)
print("position_gate_pass=", gate_pass)

```

一组固定输出如下：

```

sin_pos_3= (0.1411, -0.99, 0.2955, 0.9553, 0.03, 0.9996, 0.003, 1.0)
rope_scores= {'score_2_5': 0.527625, 'score_10_13': 0.527625, 'score_2_6': 0.602123}
alibi_bias= {'key_6': -0.125, 'key_2': -0.625, 'key_8': None}
pi_scaled_positions= [0.0, 256.0, 1024.0, 2047.75]
audit_issues= ['cache_position_reset', 'packed_sequence_position_leak', 'over_train_context_without_scaling']
position_gate_pass= False

```

这组结果可以这样读：

1. sin_pos_3 的不同维度变化速度不同，体现多频率位置坐标。
2. score_2_5 和 score_10_13 相等，因为两组位置差都是 3；score_2_6 不同，因为位置差变成 4。
3. ALiBi 中 key_2 比 key_6 离 query 更远，所以 bias 更负；未来位置 key_8 在 causal attention 中不可见。
4. Position Interpolation 把 8191 映射到 2047.75，避免直接落到训练长度外很远的位置。
5. 审计门禁失败不是因为某个位置编码算法错误，而是因为工程配置中同时出现了 cache position id 重置、packed sequence 位置泄漏和长上下文无 scaling。

7.21 面向专家：RoPE 的频率、相位和外推问题

RoPE 的每一对维度对应一个旋转频率。常见形式是：

$$\theta_i = \theta_{\lfloor \frac{i}{d} \rfloor}$$

其中 base 常见为 10000，后续模型可能使用不同 base 或 scaling 策略。

低维频率高，旋转快；高维频率低，旋转慢。位置变长时，高频维度会经历很多周期，低频维度负责更长尺度的位置变化。

外推问题来自：

训练长度之外的相位组合没有被充分学习。

如果直接把位置扩展很多倍，Q/K 点积中的相对相位会发生模型未适应的变化。RoPE scaling 的各种方法，本质上都是试图重新分配或压缩这些相位变化。

可以把它理解为一个频谱问题：

短上下文需要保留高分辨率局部位置信息。

长上下文需要避免远距离相位过快旋转导致 attention 失真。

这就是为什么简单线性缩放、NTK-aware scaling、YaRN 等方法会在不同频率维度上做更细致处理。

7.22 面向专家：位置编码和 attention pattern 的耦合

位置编码不是孤立模块。它和 attention pattern 强耦合。

对于 full attention，任意 token 可以看到任意历史 token，位置编码需要支持全局距离关系。

对于 sliding window attention，模型主要看到局部窗口，位置编码更强调窗口内局部距离。此时全局位置和局部位置如何选择，会影响模型是否能正确理解跨窗口信息。

对于 prefix LM 或 encoder-decoder attention，位置编码还要区分：

1. prefix 内部位置。
2. target 内部位置。
3. target 对 prefix 的 cross-attention 位置关系。

对于 packed sequence training，如果多个样本拼在同一个 sequence 里，position id 和 attention mask 必须配套设计。否则模型可能把不同样本之间的位置关系当成真实上下文。

对于多模态模型，位置编码更复杂。图像 patch、视频 frame、音频 token、文本 token 可能有不同位置结构。二维 RoPE、三维 RoPE、modality-specific position embedding 都是在解决这个问题。

7.23 常见误区

误区 1：有 position embedding 就一定能处理长上下文

不一定。位置编码只是让模型知道位置，长上下文能力还依赖训练长度、attention 计算、数据分布和评估任务。

误区 2：Sinusoidal 可以生成无限位置，所以天然能无限外推

公式能生成，不代表模型学会了使用训练范围外的位置。

误区 3：Learned position embedding 一定比固定位置编码强

在固定长度任务上 learned embedding 可能很好，但外推和扩展通常较弱。

误区 4：RoPE 是相对位置编码，所以没有外推问题

RoPE 在点积中体现相对位置，但仍有训练长度外的相位分布问题。

误区 5：ALiBi 总是优于 RoPE

ALiBi 外推友好、简单高效，但它引入强 recency bias。不同任务和模型规模下，效果不一定全面优于 RoPE。

误区 6：passkey retrieval 通过就说明长上下文理解强

passkey 更偏检索能力，不能完全代表长文推理、多证据整合和抗干扰能力。

7.24 面试高频问题

题 1：为什么 Transformer 需要位置编码？

参考回答：

Self-Attention 本身主要根据 token 内容计算 Q/K/V 交互，对输入顺序没有内置建模能力。如果不加入位置信息，模型很难区分同一

题 2：Sinusoidal position encoding 的公式和直觉是什么？

参考回答：

Transformer 原论文使用不同频率的 $\sin/\cos$ 函数表示位置：偶数维用 $\sin$ ，奇数维用 $\cos$ ，频率随维度变化。直觉是用多频率信号为

题 3：learned absolute position embedding 有什么问题？

参考回答：

Learned absolute position embedding 简单有效，但需要预设最大长度，并且每个位置向量只在训练中被学习。超过训练长度的位置

题 4：relative position representation 为什么更贴合 attention？

参考回答：

Attention 本质上是 token pair 之间的交互，所以很多时候需要知道 query token 和 key token 的相对距离。Relative position rep

题 5：RoPE 的核心机制是什么？

参考回答：

RoPE 根据位置对 Q 和 K 做二维旋转。第 i 个位置的 q 使用 $R(i)$ 旋转，第 j 个位置的 k 使用 $R(j)$ 旋转。由于旋转矩阵满足 $R(i)^T = R(i)^{-1}$ ，所以 Q/K 点积中自然出现相对位置 $j-i$ 。RoPE 的特点是用绝对位置实现，但在 attention score 里体现相对位置关系。

题 6：为什么 RoPE 适合 decoder-only LLM？

参考回答：

RoPE 只作用在 Q/K 上，直接影响 attention score，不改变 V，也不破坏 causal mask。它能自然表达相对距离，兼容 KV cache，推理 like 模型的常见选择。

题 7：ALiBi 和 RoPE 有什么区别？

参考回答：

ALiBi 不使用显式 position embedding，也不旋转 Q/K，而是在 attention score 上加入和距离成比例的线性 bias，距离越远惩罚越 only LLM 更常见的选择。

题 8：为什么 RoPE 模型扩展上下文不能只改 max length？

参考回答：

RoPE 的旋转角度随位置变化。训练时模型只见过某个长度范围内的相位模式，直接推理到更长位置会让 Q/K 点积出现未训练过的相位

7.25 小练习

1. 用自己的话解释为什么没有位置编码的 self-attention 不知道 token 顺序。
2. 写出 sinusoidal position encoding 的公式，并解释高频和低频维度分别起什么作用。
3. 比较 learned absolute position embedding 和 relative position bias 的优缺点。
4. 推导 RoPE 中为什么 $(R(i)q)^T(R(j)k)$ 会和 $j-i$ 有关。
5. 用 PyTorch 写一个简化版 `apply_rope` 函数。

6. 解释 ALiBi 为什么有利于长度外推，以及它可能带来什么偏置。
7. 思考 packed sequence training 中 position id 和 attention mask 如果设计错误，会导致什么问题。
8. 阅读 Position Interpolation 摘要，解释为什么插值可能比直接外推更稳定。

7.26 本章总结

本章讲了 Transformer 位置编码的主要路线：Sinusoidal、learned absolute、relative position、RoPE 和 ALiBi。

核心结论：

1. Self-Attention 本身没有内置顺序感知能力，必须注入位置信息。
2. 绝对位置编码简单，但对相对距离和长度外推不一定友好。
3. Sinusoidal 用多频率 $\sin/\cos$ 表示位置，可生成任意长度编码，但模型外推仍取决于训练。
4. Learned absolute position embedding 简单有效，但超过训练长度时天然受限。
5. Relative position representation 和 relative bias 更贴合 attention 的 pairwise 结构。
6. RoPE 通过旋转 Q/K 注入位置，在点积中自然体现相对位置，是现代 decoder-only LLM 的常见选择。
7. ALiBi 通过 attention score 的线性距离 bias 注入位置，简单且外推友好，但有 recency bias。
8. 长上下文扩展不仅是改最大长度，还涉及 RoPE 相位、位置尺度、继续训练和真实长文任务评估。

下一章会进入 Causal Mask、Prefix LM、Bidirectional Attention 与注意力模式，解释不同模型如何通过 mask 和 attention pattern 控制信息可见性。

第八章：Causal Mask、Prefix LM、Bidirectional Attention 与注意力模式

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修主要对齐 Transformer 原文文中的 encoder self-attention、masked decoder self-attention 和 encoder-decoder attention，BERT 的 deep bidirectional encoder，UniLM 用不同 self-attention mask 统一理解、生成和 seq2seq 目标的思路，UL2 对不同语言建模范式的混合，Sparse Transformer / Mistral 等长序列 attention pattern，以及 PyTorch scaled_dot_product_attention 对 attn_mask、is_causal 和高效 kernel 的公开语义。

写作边界如下：

1. 本讲重点是 mask 语义、可见性边界、信息泄漏和工程审计，不展开所有高效 attention kernel 的实现细节。
2. FlashAttention、block sparse、sliding window、varlen packing 等工程实现会因框架、GPU、版本和 kernel 选择而变化；正文只写稳定的概念边界，不把某个版本的支持矩阵泛化成永久结论。
3. 第二轮精修会把关键公式改成 GitHub Markdown 更稳的 fenced math block，并补一个 0 依赖 Python demo，专门审计 causal、prefix、packed、sliding window 和错误 mask 的可见性。

8.1 本章定位

上一章讲了位置编码：模型如何知道 token 的顺序和距离。本章继续讲另一个决定 Transformer 行为的核心机制：attention mask 和 attention pattern。

Self-Attention 的公式本身只说明“token 之间如何读信息”，但没有规定“哪些 token 可以被读”。这个可见性规则由 mask 决定。

同样是 Transformer，只要 mask 不同，就会变成完全不同的模型：

Bidirectional attention：当前位置可以看左边和右边，适合理解。

Causal attention：当前位置只能看自己和过去，适合自回归生成。

Encoder-decoder attention：encoder 双向理解输入，decoder 因果生成输出，并通过 cross-attention 读取输入。

Prefix LM：prefix 部分双向可见，生成部分因果可见。

Sparse / sliding window attention：只看局部或稀疏位置，用效率换长上下文。

本章要回答的问题是：

1. attention mask 到底屏蔽什么。
2. causal mask 为什么能防止信息泄漏。
3. bidirectional attention 为什么适合理解任务。
4. decoder-only、encoder-only、encoder-decoder 的 mask 有什么区别。
5. Prefix LM 和普通 causal LM 有什么不同。
6. padding mask、loss mask、attention mask 为什么不能混淆。
7. packed sequence、KV cache、FlashAttention 中 mask 有哪些工程坑。
8. sparse attention、sliding window attention 等注意力模式想解决什么问题。

本章的核心观点是：

Transformer 的架构形态，很大程度上是由 attention 可见性规则决定的。

8.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need. 提出 Transformer encoder-decoder 架构，包含 encoder self-attention、masked decoder self-attention 和 encoder-decoder attention。
2. Devlin et al., 2018/2019, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. 代表 encoder-only 双向注意力路线。
3. Raffel et al., 2019/2020, Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. T5 统一 text-to-text 框架，使用 encoder-decoder 架构和 span corruption 预训练。
4. Lewis et al., 2019, BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. BART 可视为双向 encoder 加自回归 decoder 的去噪 seq2seq 模型。
5. Dong et al., 2019, Unified Language Model Pre-training for Natural Language Understanding and Generation. UniLM 用不同 self-attention masks 统一 bidirectional、unidirectional 和 seq2seq 目标。
6. Tay et al., 2022, UL2: Unifying Language Learning Paradigms. UL2 讨论多种语言建模范式和 mixture-of-denoisers。
7. Child et al., 2019, Generating Long Sequences with Sparse Transformers. 提出稀疏 attention pattern，降低长序列计算成本。

需要说明的是，本章重点讲 mask 与注意力模式的架构含义，不展开所有高效 attention 算法。 $O(n^2)$ attention、KV cache、FlashAttention、长上下文推理会在后续章节继续深入。

8.3 Attention Mask 是什么

普通 scaled dot-product attention 是：

$$S = \frac{QK^{\text{top}}}{\sqrt{d_k}}$$

$$A = \text{softmax}(S)$$

$$O = AV$$

mask 的作用是在 softmax 之前，把不允许看的位置对应的 score 变成一个极小值：

$$\tilde{S} = S + M$$

常见做法是：

$$M_{ij} = \begin{cases} 0, & V_{ij} = 1 \\ -\infty, & V_{ij} = 0 \end{cases}$$

其中 $V_{ij}=1$ 表示 key 位置 j 对 query 位置 i 可见, $V_{ij}=0$ 表示不可见。

这样 softmax 后, 被禁止位置的概率接近 0。

例如:

```
\mathrm{softmax}([2.0,1.0,-\infty])  
\approx[0.731,0.269,0.000]
```

所以 attention mask 控制的是:

每个 query 位置可以从哪些 key/value 位置读取信息。

它不是删除 token, 也不是直接控制 loss, 而是控制 attention 的信息流。

8.4 三种容易混淆的 mask

工程里经常同时出现三种 mask。

Attention Mask

attention mask 控制 token 之间能不能互相看。

例如:

当前位置不能看未来 token

padding token 不能被真实 token 关注

不同 packed sample 之间不能互相看

Padding Mask

padding mask 用来标记哪些 token 是补齐出来的。

例如 batch 中两条样本长度不同:

样本 A: 长度 5

样本 B: 长度 3, 需要 pad 到 5

padding 位置不应该被 attention 读取, 也通常不应该参与 loss。

Loss Mask

loss mask 控制哪些 token 参与训练 loss。

例如 SFT 中通常只让 assistant 回复部分参与 loss:

system/user token: labels = -100, 不参与 loss

assistant token: 参与 loss

这三者不能混淆:

attention mask: 控制能看谁。

padding mask: 标记哪些是 pad。

loss mask: 控制哪些位置算 loss。

一个 token 不参与 loss, 不代表它不能被 attention 看见。SFT 中 user prompt 通常不参与 loss, 但 assistant 生成时必须能看见 user prompt。

8.5 Causal Mask

Causal mask 也叫 autoregressive mask、look-ahead mask、future mask。

它的规则是:

$$V_{ij}^{\mathrm{causal}} = \mathbf{1}[j \leq i]$$

也就是第 i 个位置只能看第 0 到第 i 个位置，不能看 i 之后的未来 token。

用矩阵表示，1 表示可见，0 表示不可见：

	key position
	0 1 2 3 4
q=0	1 0 0 0 0
q=1	1 1 0 0 0
q=2	1 1 1 0 0
q=3	1 1 1 1 0
q=4	1 1 1 1 1

这是一个下三角矩阵。

为什么需要 causal mask？因为语言模型训练时常把整段序列并行输入模型：

输入：x₀, x₁, x₂, x₃

目标：x₁, x₂, x₃, x₄

如果没有 causal mask，位置 x₁ 在预测 x₂ 时可能已经通过 attention 看到了 x₂，这就是信息泄漏。训练 loss 会虚低，但模型推理时不能提前看到未来 token，性能会崩。

8.6 Causal LM 的训练和推理

Causal LM 的目标是 next-token prediction：

$$P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_{1:t-1})$$

训练时：

整段 token 并行输入

用 causal mask 禁止每个位置看未来

每个位置预测下一个 token

推理时：

已经生成的 token 作为上下文

模型预测下一个 token

采样或贪心选出下一个 token

把新 token 追加到上下文

重复

训练和推理的关键一致性是：

训练中第 t 个位置看到的信息，不能超过推理中第 t 个位置可用的信息。

causal mask 正是保证这件事的机制。

8.7 手写 Causal Mask

最小 PyTorch 写法如下：

```
import torch
```

```
def make_causal_mask(seq_len, device):
    mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool, device=device))
    return mask[None, None, :, :]
```

```
def apply_mask(scores, mask):
    return scores.masked_fill(~mask, torch.finfo(scores.dtype).min)
```


其中 scores shape 通常是：

[batch, heads, query_len, key_len]

mask shape 可以是：

[1, 1, query_len, key_len]

通过 broadcasting 作用到所有 batch 和 heads。

如果用 additive mask, 也可以写成：

```
def make_additive_causal_mask(seq_len, device, dtype):
    allowed = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool, device=device))
    mask = torch.zeros(seq_len, seq_len, dtype=dtype, device=device)
    mask = mask.masked_fill(~allowed, torch.finfo(dtype).min)
    return mask[None, None, :, :]
```

工程里要注意：

1. mask 方向不能反。
2. dtype 要和 attention kernel 兼容。
3. fp16/bf16 中不要随意用过大的负数导致数值问题。
4. FlashAttention 等 kernel 可能不接受任意形状的显式 mask。
5. query_len 和 key_len 在 KV cache 场景下可能不相等。

8.8 Bidirectional Attention

Bidirectional attention 的规则是：

每个非 padding token 可以看同一序列中的所有非 padding token。

可见性矩阵大致是全 1：

	key position				
	0	1	2	3	4
q=0	1	1	1	1	1
q=1	1	1	1	1	1
q=2	1	1	1	1	1
q=3	1	1	1	1	1
q=4	1	1	1	1	1

BERT 是典型的 bidirectional encoder。它通过 masked language modeling 训练：把部分 token 替换成 [MASK] 或其他形式，然后用左右上下文预测被 mask 的 token。

例如：

输入：我 喜欢 [MASK] 学习

目标：机器

模型可以同时看左边 “我 喜欢” 和右边 “学习”。这对理解任务非常有利。

Bidirectional attention 适合：

1. 文本分类。
2. 句子匹配。
3. 信息抽取。
4. 阅读理解。
5. reranking。
6. embedding 表征。

但它不适合直接做标准自回归生成，因为每个位置在训练时看到了未来上下文。

8.9 Encoder-Only, Decoder-Only, Encoder-Decoder

Transformer 的三大经典形态，可以从 attention 可见性来理解。

Encoder-Only

代表模型：BERT、RoBERTa。

self-attention: 双向

训练目标：MLM、span prediction、对比学习等

擅长任务：理解、分类、抽取、embedding、rerank

Encoder-only 的每一层都允许 token 看左右上下文，输出是深层上下文表征。

Decoder-Only

代表模型：GPT 系列、LLaMA、Qwen、Mistral。

self-attention: causal

训练目标：next-token prediction

擅长任务：开放式生成、对话、代码、工具调用、in-context learning

Decoder-only 的优点是训练和推理形式高度统一：都是给定前文预测后文。

Encoder-Decoder

代表模型：原始 Transformer、T5、BART。

encoder self-attention: 双向

decoder self-attention: causal

cross-attention: decoder 读取 encoder 输出

训练目标：seq2seq、denoising、translation、summarization

Encoder-decoder 适合输入输出结构差异明显的任务，例如翻译、摘要、文本改写、语音识别后处理等。

8.10 Encoder-Decoder 的三种 Attention

原始 Transformer decoder 中有三种 attention 相关模块。

第一，encoder self-attention:

source token 之间双向可见

例如翻译中英文输入句子内部可以互相理解。

第二，decoder masked self-attention:

target token 只能看已经生成的 target token

防止生成目标句子时看到未来目标词。

第三，encoder-decoder cross-attention:

decoder 的每个 target position 可以读取 encoder 的所有 source positions

cross-attention 的 Q 来自 decoder hidden states, K/V 来自 encoder outputs:

$$Q=H_{\{\mathrm{dec}\}}W_Q,\quad$$

$$K=H_{\{\mathrm{enc}\}}W_K,\quad$$

$$V=H_{\{\mathrm{enc}\}}W_V$$

这使得 decoder 在生成每个目标 token 时，都能对输入序列做对齐和读取。

8.11 Prefix LM

Prefix LM 介于 bidirectional LM 和 causal LM 之间。

它把序列分成两段：

prefix: 作为条件输入，可以双向可见

target: 作为生成部分，只能因果可见

可见性规则是：

1. prefix token 之间可以互相看。
2. target token 可以看全部 prefix。
3. target token 之间只能看自己和过去 target。
4. prefix 通常不看 target，避免条件输入泄漏生成答案。

如果 prefix 长度为 P，总长度为 N，可以把 Prefix LM 的可见性写成：

```
V_{ij}^{\mathrm{prefix}} = \begin{cases} 1, & i < P, \ j < P \\ 1, & i \geq P, \ j < P \\ 1, & i \geq P, \ P \leq j \leq i \\ 0, & \mathrm{otherwise} \end{cases}
```

矩阵示例，假设前 3 个 token 是 prefix，后 3 个 token 是 target：

	k0	k1	k2		k3	k4	k5
q0	1	1	1		0	0	0
q1	1	1	1		0	0	0
q2	1	1	1		0	0	0
	-----				-----		
q3	1	1	1		1	0	0
q4	1	1	1		1	1	0
q5	1	1	1		1	1	1

Prefix LM 适合把“理解输入”和“生成输出”放在一个 decoder-like self-attention 结构里。

例如：

prefix: 请把下面英文翻译成中文: I love machine learning.

target: 我喜欢机器学习。

target 可以完整读取 prefix，但不能看未来 target。

8.12 Prefix LM 和 Encoder-Decoder 的区别

Prefix LM 和 encoder-decoder 都能做条件生成，但结构不同。

Encoder-decoder:

输入经过 encoder 双向编码。

输出经过 decoder 因果生成。

decoder 通过 cross-attention 读取 encoder。

Prefix LM:

输入和输出放在同一个序列里。

通过 self-attention mask 区分 prefix 和 target 的可见性。

没有独立 encoder-decoder cross-attention。

工程 trade-off:

1. Prefix LM 结构统一，更接近 decoder-only。

2. Encoder-decoder 对输入输出分离更明确。
3. Encoder-decoder 可以预先编码 source, 适合 source 很长、target 多次生成的场景。
4. Prefix LM 在现代 decoder-only 大模型中更容易和 prompt/instruction 格式结合。
5. Encoder-decoder 的 cross-attention 增加模块复杂度, 但对 seq2seq 任务很自然。

8.13 UniLM: 用 Mask 统一任务

UniLM 的思想很适合理解本章主题:

同一套 Transformer 参数, 通过不同 self-attention mask 支持不同语言建模目标。

它可以构造:

1. Bidirectional mask: 用于理解任务。
2. Unidirectional causal mask: 用于生成任务。
3. Seq2seq mask: source 双向可见, target 因果可见且可看 source。

这说明架构形态不一定完全由模块数量决定, 也可以由 mask 决定。

当然, mask 统一并不意味着所有任务效果都会一样好。不同训练目标、数据分布、位置编码、推理方式仍然会影响最终能力。

8.14 UL2: 不同语言建模范式的混合

UL2 进一步强调, 不同任务可能需要不同的语言建模范式。

它把预训练目标拆成多种 denoising / language modeling 模式, 例如:

1. 短 span corruption。
2. 长 span corruption。
3. causal language modeling。

这些目标背后对应不同的信息可见性和预测方式。

直觉上:

理解任务需要双向上下文。

生成任务需要从左到右的因果建模。

填空/去噪任务需要根据可见片段恢复不可见片段。

UL2 的意义在于提醒我们: 不要把 “Transformer 架构” 和 “预训练目标” 混为一谈。模型能看什么、预测什么、训练数据如何构造, 都会塑造最终能力。

8.15 Sparse Attention 和 Sliding Window Attention

full attention 的可见性是:

每个 token 可以看所有允许位置

对于长度 n , attention score 矩阵大小是 $n \times n$, 计算和显存都随 n^2 增长:

$$|S|=n^2, \quad C_{\{\mathrm{attn}\}}=O(n^2d_h)$$

长上下文下, 这会非常贵。

Sparse attention 的想法是:

不是所有 token pair 都需要直接相连, 只保留一部分重要连接。

常见模式包括:

1. Local window: 每个 token 只看附近窗口。
2. Strided attention: 按固定步长看远处 token。
3. Global token: 少数特殊 token 可以看全局。

4. Block sparse: 按 block 组织可见性。
5. Sliding window: 窗口随位置滑动, 常用于长文本 decoder。

例如 sliding window causal attention:

$$V_{ij}^{\mathrm{window}} = \mathbf{1}[0 \leq i-j < w]$$

也就是第 i 个 token 只能看 $[i-w+1, \dots, i]$ 范围内的历史 token。

它的优点是显著降低长序列成本。缺点是远距离信息不能直接访问, 必须通过多层传播或额外 global mechanism。

Mistral 等模型使用 sliding window attention, 说明在小模型和长上下文场景里, 局部注意力是非常实用的 trade-off。

8.16 Attention Pattern 的图视角

可以把 attention pattern 看成一张有向图:

每个 token 是一个节点。

如果 token i 可以看 token j , 就有一条从 i 到 j 的边。

Causal attention 是一张只指向过去的图。

Bidirectional attention 是近似完全图。

Sliding window attention 是局部带状图。

Global sparse attention 是少数中心节点连接很多节点。

这个图视角有助于理解信息传播。

如果第 10 个 token 不能直接看第 0 个 token, 但第 10 层后可能间接获得第 0 个 token 信息吗? 取决于图连通性和层数。

例如 window size 为 4 的 sliding window attention 中, 每层最多把信息向前传播 4 个位置。多层叠加后, 感受野会扩大:

- 1 层: 最多看到前 4 个 token
- 2 层: 可能间接看到前 8 个 token
- L 层: 理论上可看到更远

但间接传播不等于 full attention。长距离信息会经历多层压缩和混合, 效果可能不同。

8.17 Packed Sequence Training 中的 Mask

为了提高训练效率, 很多大模型训练会把多条短样本 packing 到一个长 sequence 中。

例如:

[样本 A tokens] [样本 B tokens] [样本 C tokens]

如果只用普通 causal mask, 会出现问题: 样本 B 的 token 可以看见样本 A 的 token, 因为 A 在 B 的左边。

这通常是不希望的。正确做法是 block-diagonal causal mask:

样本内部: 因果可见

样本之间: 完全不可见

可见性大致是:

A 内部下三角		A-B 不可见		A-C 不可见
B-A 不可见		B 内部下三角		B-C 不可见
C-A 不可见		C-B 不可见		C 内部下三角

如果 packed mask 做错, 会导致:

1. 样本之间信息泄漏。
2. loss 虚低。
3. 模型学习到错误的跨样本依赖。
4. 对话边界、文档边界混乱。
5. 线上推理表现和训练不一致。

8.18 KV Cache 场景下的 Mask

训练时通常 $\text{query_len} = \text{key_len} = \text{seq_len}$ 。

推理使用 KV cache 时，新 token 的 query_len 可能是 1， key_len 是历史长度加 1：

Q: [batch, heads, 1, head_dim]

K: [batch, heads, past_len + 1, head_dim]

V: [batch, heads, past_len + 1, head_dim]

这时 causal mask 不再是简单的 1×1 下三角矩阵，而要允许新 query 看全部历史 K/V 和当前 K。

例如新 token 位于全局位置 t：

允许看 $\text{key position} \leq t$

禁止看 $\text{key position} > t$

在逐 token decoding 中，通常没有未来 key，所以可以不显式构造 causal mask。但在 batch decoding、prefill、chunked prefill、speculative decoding 中，mask 仍然很重要。

常见坑：

1. prefill 阶段和 decode 阶段 mask 逻辑不一致。
2. cache position 和 position id 不一致。
3. 不同样本 past length 不同，batch mask 没处理好。
4. prefix cache 复用时，后续 token 的可见范围错误。
5. sliding window cache 丢弃历史后，mask 和位置编码没有同步。

8.19 FlashAttention 和 Mask

FlashAttention 通过分块计算避免显式 materialize 完整 attention matrix，从而降低显存并提升速度。

但这也意味着：

不是所有任意形状的 mask 都能高效支持。

常见高效支持的模式包括：

1. causal mask。
2. no mask。
3. padding mask 或 varlen sequence。
4. sliding window causal mask。

如果你传入一个非常复杂的任意 $n \times n$ mask，可能会：

1. 退化到普通 attention。
2. 触发不支持的 kernel。
3. 需要 block-sparse kernel。
4. 增加显存开销。

所以工程上设计 attention pattern 时，不只要考虑数学可行，还要考虑 kernel 是否支持。

这也是现代架构设计的现实约束：

好的 attention pattern 必须同时满足建模需求和硬件/kernel 友好性。

8.20 Mask 的数值稳定性

mask 通常在 softmax 前加极小值。很多代码会写：

```
scores = scores.masked_fill(mask == 0, -1e9)
```

这在 fp32 中通常没问题，但在 fp16/bf16 或特定 kernel 中要更谨慎。

更稳妥的写法通常是：

```
scores = scores.masked_fill(mask == 0, torch.finfo(scores.dtype).min)
```

但也要注意，如果一整行全被 mask，softmax 可能产生 NaN。

例如某个 query 没有任何可见 key：

```
softmax([-inf, -inf, -inf]) -> NaN
```

因此需要保证每个有效 query 至少能看一个 key，或者在 kernel/逻辑里专门处理全 mask 行。

常见导致全 mask 的原因：

1. padding mask 和 causal mask 合并方向错误。
2. 左 padding 时 position 和 mask 没对齐。
3. packed sequence 边界处理错误。
4. query_len/key_len 不一致时 causal mask 偏移错误。
5. 空样本或截断后只剩特殊 token。

8.21 Mask 可见性审计指标与最小 demo

本章最容易落地出错的不是“知道 causal mask 是下三角”，而是多个 mask、样本边界、目标函数和高效 kernel 同时存在时，可见性是否仍然正确。可以定义一个简化的 mask 审计门禁：

$$G_{\{\mathrm{mask}\}} = \mathbf{1}[R_{\{\mathrm{future}\}}=0] \cdot \mathbf{1}[R_{\{\mathrm{sample}\}}=0] \cdot \mathbf{1}[C_{\{\mathrm{row}\}}=1] \cdot \mathbf{1}[C_{\{\mathrm{kernel}\}}=1]$$

其中：

$$R_{\{\mathrm{future}\}} = \frac{\sum_i \sum_j \mathbf{1}[j > i] \mathbf{1}[V_{ij}=1]}{\max(1, \sum_i \sum_j \mathbf{1}[V_{ij}=1])}$$

表示 causal 或 decoder 场景下未来位置泄漏的比例。

$$C_{\{\mathrm{row}\}} = \prod_i \mathbf{1}[\left[\sum_j V_{ij}\right] \geq 1]$$

表示每个有效 query 至少有一个可见 key，避免全 mask 行造成 softmax NaN。

$$C_{\{\mathrm{kernel}\}} = \mathbf{1}[P_{\{\mathrm{mask}\}} \in \mathcal{P}_{\{\mathrm{fast}\}}]$$

表示当前 mask pattern 是否属于高效 kernel 支持的常见模式。这里不是说复杂 mask 不能用，而是提醒复杂任意 dense mask 可能导致 kernel 退化或需要专门的 block-sparse 实现。

下面 demo 用纯 Python 做一个最小可见性审计：它生成 causal、Prefix LM、packed causal、sliding window mask，并故意构造三个 bad case：普通 causal mask 用在 packed sequence 上造成跨样本泄漏，一整行全被 mask，以及任意 dense mask 不在常见 fast path 中。

```

import math

def causal_mask(n):
    return [[j <= i for j in range(n)] for i in range(n)]

def prefix_lm_mask(prefix_len, target_len):
    n = prefix_len + target_len
    rows = []
    for i in range(n):
        row = []
        for j in range(n):
            if i < prefix_len:
                row.append(j < prefix_len)
            else:
                row.append(j < prefix_len or (prefix_len <= j <= i))
        rows.append(row)
    return rows

def packed_causal_mask(lengths):
    n = sum(lengths)
    sample_id = []
    for sid, length in enumerate(lengths):
        sample_id.extend([sid] * length)
    rows = []
    for i in range(n):
        rows.append([sample_id[i] == sample_id[j] and j <= i
                     for j in range(n)])
    return rows, sample_id

def sliding_window_causal_mask(n, window):
    return [[0 <= i - j < window for j in range(n)] for i in range(n)]

def visible_counts(mask):
    return [sum(row) for row in mask]

def leak_count(mask, sample_id=None):
    leaks = 0
    for i, row in enumerate(mask):
        for j, allowed in enumerate(row):
            if not allowed:
                continue
            if j > i:
                leaks += 1
            if sample_id is not None and sample_id[i] != sample_id[j]:
                leaks += 1
    return leaks

def softmax_visible(scores, mask_row):

```



```

visible = [score for score, allowed in zip(scores, mask_row) if allowed]
if not visible:
    return None
max_score = max(visible)
exps = [math.exp(score - max_score) if allowed else 0.0
        for score, allowed in zip(scores, mask_row)]
total = sum(exps)
return [round(value / total, 4) for value in exps]

```

```

causal = causal_mask(5)
prefix = prefix_lm_mask(prefix_len=3, target_len=3)
plain = causal_mask(5)
packed, sample_id = packed_causal_mask([2, 3])
sliding = sliding_window_causal_mask(6, window=3)
bad_all_mask = [[True, False], [False, False]]

```

```

scores = [0.2, 1.0, 2.0, 3.0, 4.0]
masked_probs = softmax_visible(scores, causal[2])
leaky_probs = softmax_visible(scores, [True] * 5)

```

```

issues = []
if leak_count(causal) != 0:
    issues.append("causal_future_leak")
if leak_count(plain, sample_id) != 0:
    issues.append("plain_causal_cross_sample_leak")
if any(count == 0 for count in visible_counts(bad_all_mask)):
    issues.append("all_mask_row")
if "arbitrary_dense_mask" not in {
    "causal", "none", "padding", "sliding_window"
}:
    issues.append("kernel_pattern_not_fast_path")

```

```

gate_pass = not issues

```

```

print("causal_counts=", visible_counts(causal))
print("prefix_counts=", visible_counts(prefix))
print("sliding_counts=", visible_counts(sliding))
print("masked_probs_q2=", masked_probs)
print("leaky_future_mass_q2=", round(sum(leaky_probs[3:]), 4))
print("plain_causal_packing_leak=", leak_count(plain, sample_id))
print("block_diag_packing_leak=", leak_count(packed, sample_id))
print("mask_issues=", issues)
print("mask_gate_pass=", gate_pass)

```

一组固定输出如下：

```

causal_counts= [1, 2, 3, 4, 5]
prefix_counts= [3, 3, 3, 4, 5, 6]
sliding_counts= [1, 2, 3, 3, 3, 3]
masked_probs_q2= [0.1078, 0.2399, 0.6522, 0.0, 0.0]
leaky_future_mass_q2= 0.8683
plain_causal_packing_leak= 6
block_diag_packing_leak= 0
mask_issues= ['plain_causal_cross_sample_leak', 'all_mask_row', 'kernel_pattern_not_fast_path']
mask_gate_pass= False

```

这组结果可以这样读：

1. causal mask 中第 i 行可见数量依次为 1,2,3,4,5，符合只能看自己和过去。
2. Prefix LM 的前 3 行都能看 3 个 prefix token，target 行可见数量逐步增加，说明 target 能看全部 prefix 和过去 target。
3. sliding window 在窗口填满后每行只看 3 个 token，用局部可见性换取更低成本。
4. masked_probs_q2 的未来位置概率为 0；如果错误地让第 2 行看全序列，未来 token 概率质量会达到 0.8683。
5. 普通 causal mask 用在 packed sequence [2,3] 上会产生 6 个跨样本可见关系，block-diagonal causal mask 则把泄漏降为 0。
6. 门禁失败是刻意构造的：它暴露了 packed sequence 边界、全 mask 行和 kernel fast path 三类常见工程风险。

8.22 面向专家：Mask、目标函数和数据格式的耦合

mask 不是单独的工程细节，它和目标函数、数据格式强耦合。

以 SFT 为例：

attention mask: assistant token 可以看 system/user/历史 assistant。

loss mask: 通常只在 assistant 回复 token 上算 loss。

causal mask: assistant token 不能看未来 assistant token。

如果把 user token 的 attention 也 mask 掉，assistant 就看不到问题。

如果 user token 参与 loss，模型会学习复述 user 输入。

如果 causal mask 方向错，模型会看到未来答案。

所以排查训练异常时，不能只看 input_ids。至少要检查：

1. decoded text。
2. attention mask。
3. labels。
4. labels 中的 -100 分布。
5. position ids。
6. sample boundary。
7. logits/labels shift 是否正确。

很多“模型不听话”“loss 很低但生成很差”“SFT 后复读用户问题”的问题，本质都是 mask 或格式错。

8.23 面向专家：Bidirectional 不是不能生成，Causal 也不是不能理解

一个常见误解是：

BERT 只能理解，GPT 只能生成。

更准确地说：

BERT 的训练目标和双向 mask 更适合理解表征，不适合直接 left-to-right 生成。

GPT 的 causal mask 和 next-token objective 更适合自回归生成，但也能通过 prompt 完成理解任务。

Bidirectional 模型可以用于生成类任务，例如通过 mask filling、迭代填空、encoder-decoder 组合等方式。但它不是天然的逐 token 左到右生成器。

Causal LM 可以做分类、抽取、阅读理解，因为这些任务可以转化成文本生成或选择题格式。但它在一些纯表征任务上未必比专门训练的 encoder embedding 模型更高效。

架构倾向不是绝对能力边界，而是训练目标、mask 和推理方式共同形成的优势区域。

8.24 常见误区

误区 1: attention mask 和 loss mask 是一回事

不是。attention mask 控制能看谁, loss mask 控制哪里算 loss。

误区 2: Causal LM 训练是一个 token 一个 token 串行训练

不是。训练时整段序列并行输入, 通过 causal mask 防止看未来。

误区 3: BERT 双向看上下文, 所以一定比 GPT 强

不对。BERT 的双向 attention 适合理解表征, GPT 的 causal LM 适合自回归生成和统一任务接口。强弱取决于任务、规模、数据和训练方式。

误区 4: Prefix LM 等同于普通 causal LM

不等同。Prefix LM 的 prefix 内部可以双向可见, 而普通 causal LM 从第一个 token 到最后一个 token 都只能看过去。

误区 5: 只要 mask 数学上对, 工程上就高效

不一定。复杂 mask 可能无法使用高效 attention kernel, 导致速度和显存退化。

误区 6: packed sequence 只要拼起来就能训练

不行。必须处理样本边界, 否则不同样本之间会信息泄漏。

8.25 面试高频问题

题 1: 为什么 decoder-only LLM 需要 causal mask?

参考回答:

Decoder-only LLM 使用 next-token prediction。训练时整段序列并行输入, 如果没有 causal mask, 当前位置可能通过 self-attention 看到未来 token, 导致信息泄漏和 loss 虚低。Causal mask 保证第 i 个位置只能看自己和过去位置, 使训练时可用信息和

题 2: attention mask、padding mask、loss mask 有什么区别?

参考回答:

Attention mask 控制 token 之间能不能互相看, padding mask 标记哪些位置是补齐 token, loss mask 控制哪些 token 参与 loss。SF

题 3: Bidirectional attention 为什么适合理解任务?

参考回答:

Bidirectional attention 允许每个 token 同时利用左侧和右侧上下文, 因此能形成更完整的上下文表征。分类、抽取、阅读理解、en only 双向模型很适合理解任务。

题 4: Encoder-only、decoder-only、encoder-decoder 的 attention mask 有什么区别?

参考回答:

Encoder-only 通常使用双向 self-attention, 所有非 padding token 互相可见。Decoder-only 使用 causal self-attention, 只能看过去和当前位置。Encoder-decoder 中 encoder 是双向 self-attention, decoder 是 causal self-attention, 同时 decoder 通过 cross-attention 读取 encoder 输出。

题 5: Prefix LM 的 mask 是什么样的?

参考回答:

Prefix LM 把序列分为 prefix 和 target。Prefix 内部 token 可以双向可见; target token 可以看全部 prefix; target 内部只能 causal attention 序列中, 通过 mask 控制可见性。

题 6: packed sequence training 中 mask 为什么重要?

参考回答:

Packing 会把多条短样本拼成一个长序列。如果只用普通 causal mask, 后面的样本可以看到前面样本的 token, 造成跨样本信息泄漏 diagonal causal mask。

题 7: 为什么复杂 attention mask 可能影响性能?

参考回答:

高效 attention kernel 通常只针对常见模式优化, 例如 causal、无 mask、padding varlen 或 sliding window。如果 mask 是任意 sparse kernel。因此 attention pattern 设计要考虑硬件和 kernel 支持。

题 8: KV cache 下 causal mask 有什么变化?

参考回答:

训练时 query_len 和 key_len 通常相等, 可以用标准下三角 causal mask。KV cache 推理时, 新 token 的 query_len 可能是 1, key_

8.26 小练习

1. 画出长度为 6 的 causal mask 矩阵。
2. 画出 prefix 长度为 3、target 长度为 3 的 Prefix LM mask。
3. 用 PyTorch 实现一个 make_causal_mask(seq_len)。
4. 解释为什么 SFT 中 user token 不参与 loss, 但 assistant token 仍要能看见 user token。
5. 设计一个 packed sequence 的 block-diagonal causal mask。
6. 比较 BERT、GPT、T5/BART 的 attention 可见性。
7. 思考 sliding window attention 如何通过多层扩大感受野。
8. 排查一个 loss 很低但生成很差的模型, 列出你会检查的 mask 和格式项。

8.27 本章总结

本章讲了 Causal Mask、Prefix LM、Bidirectional Attention 和注意力模式。

核心结论:

1. Self-Attention 本身不规定可见性, attention mask 决定每个 token 能看谁。
2. Causal mask 防止未来信息泄漏, 是 decoder-only LLM 自回归训练的关键。
3. Bidirectional attention 允许左右上下文联合建模, 适合理解和表征任务。
4. Encoder-only、decoder-only、encoder-decoder 的核心差异可以从 attention 可见性理解。
5. Prefix LM 让 prefix 双向可见、target 因果可见, 是条件生成的一种统一 mask 形式。
6. Attention mask、padding mask、loss mask 是三件不同的事, 混淆会导致严重训练事故。
7. Packed sequence、KV cache、FlashAttention 都要求 mask 逻辑和工程实现严格对齐。
8. Sparse/sliding window attention 通过改变可见性降低长上下文成本, 但会改变信息传播路径。

下一章会进入 Transformer 的并行性、表达力和 scaling 优势, 解释为什么 Transformer 能在大规模数据、参数和算力下表现出强扩展性。

第九章：Transformer 的并行性、表达力和 scaling 优势

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修主要对齐 Transformer 原文中关于摆脱 recurrence / convolution、提高序列并行度和缩短最大路径长度的论述，Kaplan scaling laws 对 cross-entropy loss、参数量、数据量和计算量的经验幂律拟合，GPT-3 对 few-shot / in-context learning 的展示，Chinchilla 对固定 compute 下参数和训练 token 配比的修正，PaLM 对大规模系统训练的案例，以及 emergent abilities 相关论文对“涌现”现象的边界讨论。

写作边界如下：

1. scaling law 是经验规律，不是跨模型族、跨数据分布、跨训练 recipe 都固定成立的物理定律。
2. 本讲不把 Kaplan、Chinchilla 或 PaLM 中的具体指数、token/parameter 比例、硬件规模写成通用标准，只讲它们带来的方法论。
3. 第二轮精修会把并行性、路径长度、attention 成本、训练 FLOPs 和 scaling gate 写成稳定 MathJax 公式，并补一个 0 依赖 demo，用 toy 数字解释“最低 toy loss”不等于“最适合上线或最 compute-balanced”。

9.1 本章定位

第一章已经从全局角度回答过“Transformer 为什么能成为大模型基础架构”。本章进入第二部分，开始系统拆解 Transformer 的优势和瓶颈。

本章先讲优势：并行性、表达力和 scaling。

Transformer 能在大模型时代胜出，不只是因为 attention 很优雅，而是因为它同时满足了三类条件：

算法上：能让 token 之间做灵活的信息路由。

工程上：能把大部分计算变成大矩阵乘法，适合 GPU/TPU 并行。

规模上：随着参数、数据和计算增加，loss 和能力通常能稳定改善。

这三点叠加，形成了大模型发展的主线：

更大模型 + 更多数据 + 更多计算 + 稳定训练 recipe + Transformer 主干

本章要回答的问题是：

1. Transformer 为什么比 RNN 更适合大规模并行训练。
2. Self-Attention 的表达力来自哪里。
3. 多层、多头、FFN 和 residual stream 如何共同提升表达能力。
4. 为什么 Transformer 对 GPU/TPU 友好。
5. scaling law 到底说明了什么。
6. Chinchilla 为什么改变了“只堆参数”的观念。
7. GPT-3、PaLM 等模型展示的 few-shot、reasoning 和 emergent abilities 该如何理解。
8. scaling 的边界是什么，为什么不能把 scaling law 当成万能定律。

本章的核心观点是：

Transformer 的成功不是单一结构胜利，而是表达力、并行计算和可预测扩展三者共振的结果。

9.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need。提出 Transformer，强调完全基于 attention，摆脱 recurrence 和 convolution，并在机器翻译中实现更高并行性和更低训练成本。
2. Kaplan et al., 2020, Scaling Laws for Neural Language Models。系统研究语言模型 cross-entropy loss 随模型规模、数据规模和训练 compute 的幂律关系。
3. Brown et al., 2020, Language Models are Few-Shot Learners。GPT-3 展示大规模 autoregressive Transformer 的 few-shot/in-context learning 能力。
4. Hernandez et al., 2021, Scaling Laws for Transfer。研究预训练迁移中的 scaling 规律，说明预训练可转化为有效数据增益。

5. Hoffmann et al., 2022, Training Compute-Optimal Large Language Models。Chinchilla 工作指出固定 compute 下模型参数和训练 token 需要共同扩展，很多早期大模型 undertrained。
6. Chowdhery et al., 2022, PaLM: Scaling Language Modeling with Pathways。展示大规模 dense Transformer 在多任务、few-shot、多步推理、代码和多语言上的 scaling 效果。
7. Wei et al., 2022, Emergent Abilities of Large Language Models。讨论一些能力在小模型中不明显、在大模型中突然出现的现象。

需要说明的是，scaling law 是经验规律，不是物理定律。它依赖模型族、数据分布、训练 recipe、优化稳定性和评估指标。本章会同时讲 scaling 的价值和边界。

9.3 Transformer 的并行性优势

先看 RNN/LSTM。

RNN 的计算有强时间依赖：

$$h_t = f(h_{t-1}, x_t)$$

第 t 个 hidden state 必须等第 $t-1$ 个 hidden state 算完。即使 batch 很大，单条序列内部仍然难以完全并行。

Transformer 不一样。对一整段序列：

$$X = [x_1, x_2, \dots, x_n] \in \mathbb{R}^{n \times d}$$

它可以一次性计算：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

再一次性计算所有 token pair 的 attention score：

$$S = \frac{QK^{\text{top}}}{\sqrt{d_k}}$$

这意味着训练时，序列中所有位置可以并行计算。

并行性带来的不是小优化，而是大模型时代的根本前提。因为预训练需要处理万亿级 token，如果序列内部强串行，训练墙钟时间会非常难接受。

9.4 并行性不等于计算量少

Transformer 并不是每个维度都更省。

Self-Attention 的 score matrix 是 $n \times n$ ：

$$|S| = n^2$$

如果 head dimension 是 d_h ，attention score 的主要矩阵乘成本可以粗略写成：

$$C_{\{\text{score}\}} = O(n^2 d_h)$$

所以 attention 在长序列下有 $O(n^2)$ 计算和显存瓶颈。这也是下一章要重点讲的问题。

但 Transformer 的优势在于：

虽然总计算量可能大，但这些计算高度并行、规则、密集，适合 GPU/TPU。

相比之下，RNN 的计算量可能看起来是 $O(n)$ ，但时间步依赖导致并行度低。

可以用一句话概括：

Transformer 用更多可并行计算，换掉了 RNN 的串行依赖。

在大规模硬件上，这个 trade-off 非常划算。

9.5 GPU/TPU 友好性

现代加速器最擅长的是大规模矩阵乘法。

Transformer 的核心操作大多是矩阵乘法：

1. Q/K/V projection。
2. attention score 的 QK^T 。
3. attention output 的 AV 。
4. output projection。
5. FFN 中的两个或三个大 linear layers。

这些操作可以高效映射到 GPU tensor cores 或 TPU matrix units。

同时，Transformer 的结构很规则：

attention block + FFN block + residual + norm

重复堆叠几十层、上百层，工程系统可以围绕这种规则结构做大量优化：

1. fused kernels。
2. FlashAttention。
3. tensor parallelism。
4. pipeline parallelism。
5. sequence parallelism。
6. activation checkpointing。
7. ZeRO/FSDP optimizer sharding。
8. KV cache 推理优化。

这就是为什么 Transformer 不只是一个算法结构，也是一个硬件生态友好的计算模板。

9.6 Self-Attention 的表达力：动态信息路由

Self-Attention 的核心表达力来自动态路由。

对每个 query token，模型会根据当前内容动态决定从哪些 token 读取信息：

$$a_{ij} = \frac{\exp(q_i^T \cdot k_j / \sqrt{d_k})}{\sum_{\ell \in \mathcal{V}_i} \exp(q_i^T \cdot k_\ell / \sqrt{d_k})}$$

其中 $\mathcal{V}_i$ 是第 i 个 query 允许看到的 key 集合。这个集合由上一章讲的 attention mask 决定。

这和固定卷积核不同。

CNN 的局部卷积通常在固定邻域内混合信息：

当前位置看附近几个位置

Self-Attention 则可以让当前位置直接看任意位置：

当前位置可以看前一个词、句首主题、函数定义、远处括号、系统提示、示例答案

更重要的是，这个“看哪里”的模式不是手写规则，而是由 Q/K 内容匹配动态决定。

所以 attention 更像一个可学习的信息检索系统：

Q：我现在需要什么信息？

K：每个位置提供什么索引？

V：每个位置真正携带什么内容？

这使 Transformer 很适合语言、代码、数学推理、对话上下文、工具调用等需要动态依赖的任务。

9.7 一跳长距离交互

在 RNN 中，位置 i 的信息传到位置 j ，需要经过很多时间步：

$i \rightarrow i+1 \rightarrow i+2 \rightarrow \dots \rightarrow j$

路径长，梯度和信息都容易衰减或被覆盖。

在 CNN 中，如果卷积核很小，也需要多层堆叠才能扩大感受野。

Self-Attention 中，任意两个 token 可以在一层内直接交互：

$i \rightarrow j$

这对长距离依赖非常重要。

例如：

1. 代码中函数调用要回看函数定义。
2. 数学证明中后文要引用前文假设。
3. 多轮对话中当前回答要遵守系统提示。
4. 长文摘要中结论要整合前面多个段落。
5. JSON/XML/括号结构中远距离符号要匹配。

当然，一跳可达不等于一定能用好。长上下文中的 lost-in-the-middle、精确检索和干扰问题会在后续章节专门讨论。

9.8 多头注意力：多个子空间的并行路由

单个 attention head 只能学习一种 Q/K/V 投影方式。多头注意力把 hidden representation 切成多个子空间：

$\text{head}_1, \text{head}_2, \dots, \text{head}_h$

每个 head 可以学习不同的信息路由模式。

例如：

1. 某些 head 关注局部邻近 token。
2. 某些 head 关注句法依赖。
3. 某些 head 关注分隔符或特殊 token。
4. 某些 head 关注复制、引用、括号匹配。
5. 某些 head 在特定任务中承担检索或路由作用。

不要把这些解释绝对化。attention head 的可解释性有启发价值，但 raw attention weight 不是完整因果解释。

从表达力角度看，多头注意力的意义是：

同一层中并行学习多种关系，而不是用一个相似度函数解释所有依赖。

9.9 FFN 的表达力：逐 token 的非线性计算

Attention 负责 token 间信息交换，FFN 负责每个 token 内部的非线性变换。

一个 Transformer block 可以粗略理解为：

Attention：从上下文读取信息

FFN：对读取后的信息做非线性加工

FFN 通常占 Transformer 参数量的大头。现代 LLM 中的 SwiGLU/Gated MLP 进一步提升了 FFN 的表达能力。

所以不能把 Transformer 的表达力全部归因于 attention。更准确地说：

Attention 提供上下文路由，FFN 提供局部计算，残差流负责在层间积累和传递状态。

多层堆叠后，模型能进行复杂的上下文组合。

9.10 Residual Stream：逐层写入和读取的共享工作区

Transformer 中每层都通过 residual connection 更新 hidden states：

$$x_{\ell+1} = x_{\ell} + F_{\ell}(\mathrm{Norm}(x_{\ell}))$$

可以把 residual stream 想成一个共享工作区。

每层 attention 和 FFN 都可以从这个工作区读取信息，再写入新的增量信息。

这带来一个重要直觉：

Transformer 不是每层重新表示所有东西，而是在 residual stream 中逐层积累特征、关系和中间计算结果。

这有助于理解为什么 depth 有价值：更深层可以在前面层构造出的表示基础上继续组合。

但 depth 也带来训练稳定性问题，所以需要 Pre-Norm、RMSNorm、初始化、学习率 warmup、residual scaling 等配套机制。第六章已经详细讲过这部分。

9.11 Scaling Law 到底说了什么

Scaling law 研究的是：当模型参数量、训练数据量和训练计算量变大时，语言模型 loss 如何变化。

Kaplan 等人的工作发现，在一定范围内，cross-entropy loss 和规模变量之间可以用幂律关系描述：

$$L(N) \approx A_{NN}^{-\alpha} + B_N$$

$$L(D) \approx A_{DD}^{-\beta} + B_D$$

$$L(C) \approx A_{CC}^{-\gamma} + B_C$$

其中：

1. N 是模型参数量。
2. D 是训练数据量。
3. Compute 是训练计算量。
4. α 、 β 、 γ 是经验拟合出来的指数。

不要死记公式。核心意义是：

在相当大的尺度范围内，扩大模型、数据和计算，loss 会以相对平滑、可预测的方式下降。

这给大模型训练带来了工程上的可规划性。

训练一个百亿、千亿参数模型前，可以先做一系列小模型实验，拟合趋势，再预测更大模型可能达到的 loss 区间。

9.12 Scaling Law 为什么重要

Scaling law 的影响不只是学术上的。

它改变了大模型研发方式。

在没有 scaling law 之前，大规模训练更像豪赌：

投入巨大算力，训练完才知道结果。

有了 scaling 经验规律后，可以做更系统的资源规划：

1. 给定 compute budget，选多大模型。
2. 给定模型大小，训练多少 token。
3. 训练曲线是否偏离预期。
4. 数据质量提升是否带来 loss 改善。
5. 是否值得继续扩大模型或数据。
6. 小规模实验能否预测大规模趋势。

这也是为什么 scaling law 巩固了 Transformer 路线。它让产业和研究更愿意沿着同一主干持续优化，而不是频繁更换架构。

9.13 Chinchilla: 从“更大”到“compute-optimal”

早期 scaling 叙事容易被简化成：

模型越大越好。

Chinchilla 修正了这个观念。

Hoffmann 等人的工作指出，固定 compute budget 下，很多已有大模型参数量很大，但训练 token 不足，也就是 undertrained。

他们的核心启发是：

参数量和训练 token 数要一起讨论。

在同样训练计算量下，一个更小但训练 token 更多的模型，可能超过一个更大但训练不足的模型。

常见的 dense Transformer 训练 FLOPs 粗估是：

$$C_{\{\mathrm{train}\}} \approx 6ND$$

其中 N 是参数量， D 是训练 token 数。这个公式只用于粗略估算，不替代真实 profiler、激活重算、并行效率、optimizer 开销和硬件利用率分析。

Chinchilla 用 70B 参数、更多训练数据，在很多任务上超过了 Gopher 280B、GPT-3 175B 等更大模型。

面试里不要把 Chinchilla 机械背成某个固定比例。更准确的表达是：

Chinchilla 提供的是 compute-optimal 思路：在给定算力下，需要平衡模型容量和训练 token，而不是只追求参数量。

这个思想也推动了后来的高质量小模型路线：参数不一定最大，但训练更充分、数据质量更高、recipe 更好，实际能力可以很强。

9.14 Scaling 不只是参数，还包括数据质量

Scaling law 常用参数量、token 数和 compute 描述趋势，但真实训练中，数据质量同样关键。

两个模型都训练 1T token，不代表它们获得了同样有效的信息。

数据质量会影响：

1. 重复内容比例。
2. 垃圾网页和模板文本比例。
3. 代码、数学、知识、对话等数据配比。
4. 多语言覆盖。
5. 数据污染和 benchmark leakage。
6. harmful/toxic 内容分布。
7. 训练样本格式一致性。

所以更准确的说法是：

Scaling 需要参数、有效数据、计算、优化稳定性和评估体系共同成立。

单纯增加低质量 token，可能只是浪费 compute，甚至损害能力和安全性。

9.15 GPT-3: Scaling 带来的 In-Context Learning

GPT-3 的重要性在于，它展示了大规模 decoder-only Transformer 的 few-shot 能力。

传统范式是：

预训练模型 + 任务数据 fine-tuning

GPT-3 展示的是：

在 prompt 中给任务说明和少量示例，不更新参数，模型也能完成新任务。

例如：

English: cat

French: chat

English: dog

French: chien

English: book

French:

模型可以从上下文示例中推断任务模式。

这背后至少涉及三件事：

1. 预训练中见过大量任务、格式和文本模式。
2. attention 能在上下文中读取示例并匹配当前 query。
3. 大模型容量足够强，可以把 prompt 当作临时任务说明。

In-context learning 不等于模型真正像人一样学习，也不等于参数更新。它更像是在上下文中做模式识别、检索、归纳和条件生成。

后续第 12 章会专门展开 In-Context Learning 与显式 token 检索能力。

9.16 PaLM：大规模训练和系统能力

PaLM 展示了另一个角度：大规模 dense Transformer 需要系统工程配套。

PaLM 540B 使用 Pathways 在 6144 个 TPU v4 上训练。它说明大模型 scaling 不只是模型结构问题，还包括：

1. 分布式训练系统。
2. 数据管线。
3. 稳定性控制。
4. 高效并行策略。
5. 训练监控和故障恢复。
6. 大规模评估。

PaLM 报告了多语言、代码、few-shot、多步推理等能力，并观察到一些 BIG-bench 任务随规模出现不连续提升。

这说明 Transformer scaling 的成功，是算法和系统共同作用的结果。

如果没有高效并行系统，大模型无法训练；如果没有稳定训练 recipe，scaling 曲线会偏离；如果没有足够数据，参数会 undertrained。

9.17 Emergent Abilities：涌现能力怎么理解

涌现能力通常指：

小模型上几乎看不到，模型变大后突然表现出来的能力。

例如某些多步推理、复杂指令跟随、少样本任务、符号操作，在小模型中接近随机，到了某个规模后明显提升。

这个现象很吸引人，但要谨慎理解。

第一，涌现可能和评估指标有关。

如果指标是离散 accuracy，模型从 49% 到 51% 可能看起来像突然跨过阈值。连续指标下趋势可能更平滑。

第二，涌现不一定只由参数量决定。

训练数据、prompt 格式、chain-of-thought、评估任务、采样策略都会影响是否观察到涌现。

第三，涌现不代表能力完全不可研究。

它提醒我们小规模实验可能无法预测所有能力，但仍可以通过更细的指标、更好的 scaling 实验和机制分析来理解。

更稳妥的表述是：

Scaling 通常带来平滑 loss 改善，但部分下游能力可能在某些评估上表现出非线性或阈值式变化。

9.18 Transformer 的表达力和 Scaling 为什么能配合

一个架构适合 scaling，至少需要满足几个条件。

第一，容量可扩展。

增加层数、宽度、head 数、FFN hidden size 后，模型能有效利用新增参数。

第二，优化可稳定。

模型变深变宽后，loss 仍能稳定下降，不频繁发散。

第三，硬件可扩展。

新增计算能高效分摊到更多 GPU/TPU 上。

第四，数据可吸收。

模型能从海量多样文本、代码、数学、对话中学习可迁移模式。

Transformer 恰好在这些方面都表现不错。

这不是说 Transformer 是理论上唯一选择，而是说它在过去几年形成了最成熟的 scaling 证据和工程生态。

9.19 为什么不是 CNN 或 RNN 成为 LLM 主干

RNN/LSTM 的问题主要是串行依赖和长距离路径。

优点：有状态、天然适合流式。

缺点：训练并行性差，长距离信息需要多步传递，大规模训练生态弱。

CNN 的问题主要是局部感受野。

优点：并行、局部性强、硬件友好。

缺点：远距离依赖需要多层或大卷积核，动态内容路由不如 attention 灵活。

Transformer 的优势是：

既能并行训练，又能让任意 token 直接交互，还能把计算变成大矩阵乘法。

但这不代表 RNN/CNN 思想无价值。后续 Mamba、RWKV、RetNet、Hyena 等架构，很多都在重新探索状态、卷积、线性时间和硬件效率，只是它们需要在表达力、并行训练和生态上追赶 Transformer。

9.20 Scaling 的边界

Scaling 很强，但不是万能。

它至少有以下边界。

第一，loss 改善不等于所有能力等比例改善。

困惑度下降，不保证数学推理、安全拒答、工具调用、长上下文检索都同步提升。

第二，数据质量可能成为瓶颈。

低质量、高重复、污染数据会浪费 token budget。

第三，训练稳定性是前提。

loss spike、NaN、分布式错误、mask 错误、优化器状态异常都会破坏 scaling 趋势。

第四，推理成本会成为约束。

更大模型训练后，部署延迟、显存、吞吐、KV cache 都可能成为瓶颈。

第五，安全和对齐不自动随规模解决。

规模变大可能提高能力，也可能放大幻觉、偏见、越狱、隐私记忆等风险。

第六，架构瓶颈仍然存在。

Transformer 的 $O(n^2)$ attention、长上下文检索失败、状态记忆不足、流式低延迟困难，都是后续章节要讲的核心瓶颈。

9.21 并行与 Scaling 审计指标与最小 demo

工程里判断一个 Transformer scaling 方案，不能只看“参数更大”或“toy loss 更低”。至少要同时看四类门禁：

```
G_{\mathrm{scale}}=
\mathbf{1}[C_{\mathrm{path}}=1]\cdot
\mathbf{1}[C_{\mathrm{balance}}=1]\cdot
\mathbf{1}[C_{\mathrm{data}}=1]\cdot
\mathbf{1}[C_{\mathrm{serve}}=1]
```

其中：

```
C_{\mathrm{path}}=\mathbf{1}[P_{\mathrm{model}}\leq P_{\mathrm{max}}]
```

表示序列内信息路径是否足够短。Transformer 的一层 attention 可以让任意可见 token 直接交互；RNN 的最坏路径随序列长度增长；小卷积核 CNN 需要多层扩大感受野。

```
C_{\mathrm{balance}}=
\mathbf{1}[\tau_{\min}\leq D/N\leq\tau_{\max}]
```

表示训练 token 数和参数量是否大致平衡。这里的 $\tau_{\min}$ 和 $\tau_{\max}$ 是教学 demo 的经验门槛，不是 Chinchilla 原论文的通用比例。

```
C_{\mathrm{serve}}=\mathbf{1}[S_{\mathrm{infer}}\leq S_{\max}]
```

表示候选模型的推理压力是否在线预算内。训练 loss 更低的模型，如果推理成本太高，也未必是业务上最优选择。

下面 demo 用纯 Python 做一个简化审计。它不训练模型，也不拟合真实 scaling law，只用 toy 公式展示三件事：

1. Transformer 用 n^2 个 attention pair 换来一层内长距离交互。
2. 固定训练 compute 时，参数变大意味着可训练 token 变少。
3. toy loss 最低的候选，不一定同时满足 token/parameter balance 和 serving gate。

```
import math
```

```
def train_flops(params_b, tokens_b):
    return 6 * params_b * tokens_b
```

```
def toy_loss(params_b, tokens_b, data_quality=1.0):
    model_term = 0.55 * params_b ** -0.32
    data_term = 0.90 * (tokens_b * data_quality) ** -0.28
    return 1.35 + model_term + data_term
```

```
def architecture_cost(seq_len, kernel=5):
```

```

    return {
        "rnn_steps": seq_len,
        "cnn_layers_for_full_range": math.ceil((seq_len - 1) / (kernel - 1)),
        "transformer_path_length": 1,
        "attention_score_millions": round(seq_len * seq_len / 1_000_000, 2),
        "attention_vs_rnn_pair_ratio": seq_len,
    }

def compute_candidates(budget_flops_b2, data_quality):
    rows = []
    for params_b in [1, 3, 7, 13, 30]:
        tokens_b = budget_flops_b2 / (6 * params_b)
        tokens_per_param = tokens_b / params_b
        loss = toy_loss(params_b, tokens_b, data_quality)
        serving_pressure = min(1.0, params_b / 30)
        balanced = 10 <= tokens_per_param <= 40
        rows.append({
            "params_b": params_b,
            "tokens_b": round(tokens_b, 1),
            "tokens_per_param": round(tokens_per_param, 1),
            "toy_loss": round(loss, 4),
            "serving_pressure": round(serving_pressure, 3),
            "balanced": balanced,
        })
    return rows

def scaling_gate(best, data_quality, train_stability, serving_limit):
    failed = []
    if not best["balanced"]:
        failed.append("compute_balance")
    if data_quality < 0.8:
        failed.append("data_quality")
    if not train_stability:
        failed.append("train_stability")
    if best["serving_pressure"] > serving_limit:
        failed.append("serving_pressure")
    return failed, not failed

seq_len = 4096
budget = train_flops(7, 140)
records = compute_candidates(budget, data_quality=0.92)
loss_best = min(records, key=lambda row: row["toy_loss"])
ready_candidates = [
    row for row in records
    if row["balanced"] and row["serving_pressure"] <= 0.7
]
ready_best = min(ready_candidates, key=lambda row: row["toy_loss"])
failed, gate_pass = scaling_gate(
    ready_best,
    data_quality=0.92,
    train_stability=True,
    serving_limit=0.7,

```

```
)

print("arch_cost=", architecture_cost(seq_len))
print("compute_budget_1e21_flops=", round(budget / 1000, 3))
print("records=", records)
print("loss_best=", loss_best)
print("ready_best=", ready_best)
print("failed_gates=", failed)
print("scaling_gate_pass=", gate_pass)
```

一组固定输出如下：

```
arch_cost= {'rnn_steps': 4096, 'cnn_layers_for_full_range': 1024, 'transformer_path_length': 1, 'attention_score_millions': 1678}
compute_budget_1e21_flops= 5.88
records= [{'params_b': 1, 'tokens_b': 980.0, 'tokens_per_param': 980.0, 'toy_loss': 2.0339, 'serving_pressure': 0.033, 'balance_gate': False}, {'params_b': 7, 'tokens_b': 140.0, 'tokens_per_param': 20.0, 'toy_loss': 1.876, 'serving_pressure': 0.233, 'balance_gate': True}, {'params_b': 13, 'tokens_b': 75.4, 'tokens_per_param': 5.8, 'toy_loss': 1.8667, 'serving_pressure': 0.433, 'balance_gate': True}]
loss_best= {'params_b': 13, 'tokens_b': 75.4, 'tokens_per_param': 5.8, 'toy_loss': 1.8667, 'serving_pressure': 0.433, 'balance_gate': True}
ready_best= {'params_b': 7, 'tokens_b': 140.0, 'tokens_per_param': 20.0, 'toy_loss': 1.876, 'serving_pressure': 0.233, 'balance_gate': True}
failed_gates= []
scaling_gate_pass= True
```

这组结果可以这样读：

1. seq_len=4096 时，RNN 最坏串行步数是 4096，小卷积核 CNN 要很多层扩大感受野，而 Transformer 的可见 token 一层内路径长度为 1。
2. Transformer 的代价是 attention score 有 1678 万个 pair，因此下一章要继续讲 $O(n^2)$ 成本和长上下文瓶颈。
3. 在固定 toy compute 下，1B/3B 模型 token 很多但容量小，13B/30B 模型 token-per-parameter 太低，可能 undertrained。
4. loss_best 是 13B，但它没有通过 balance gate；ready_best 是 7B，因为它同时满足 toy loss、tokens-per-parameter 和 serving pressure。
5. demo 数字只用于教学，不是任何真实模型的 benchmark。真实训练要结合数据质量、优化稳定性、硬件效率、评估和上线成本。

9.22 面向专家：Scaling Law 的正确用法

在真实项目中，scaling law 更像一个资源规划工具，而不是最终真理。

正确用法包括：

1. 用小模型和中模型拟合 loss vs compute 趋势。
2. 检查大模型训练曲线是否偏离预测。
3. 比较不同数据配比的有效性。
4. 估算给定预算下模型大小和 token 数。
5. 判断模型是否 undertrained。
6. 评估继续训练、扩模型、换数据哪个收益更高。

错误用法包括：

1. 把某篇论文的指数当成跨所有模型固定常数。
2. 只看参数量，不看 token 数。
3. 只看 token 数，不看数据质量。
4. 用 loss 直接预测所有复杂 benchmark。
5. 忽略训练稳定性和系统瓶颈。
6. 把 Chinchilla 简化成绝对比例公式。

面试中能说清这些边界，会比只背“loss 幂律下降”更有深度。

9.23 面向专家：表达力、优化和归纳偏置的张力

Transformer 表达力强，但归纳偏置相对弱。

它不像 CNN 那样强假设局部平移不变，也不像 RNN 那样天然维护递推状态。它更多依赖数据和规模去学出结构。

这带来两面性。

好处是：

足够通用，可以统一文本、代码、多模态 token、工具调用轨迹等多种序列。

代价是：

需要大量数据和计算来学出本可由强归纳偏置提供的规律。

这也是为什么后续架构创新经常在做两件事：

1. 保留 Transformer 的通用性和并行性。
2. 引入更合适的归纳偏置或效率改进。

例如：

1. RoPE 引入相对位置结构。
2. Sliding window 引入局部性。
3. GQA/MQA 降低 KV cache 成本。
4. MoE 扩大参数容量但保持稀疏激活。
5. Mamba/RWKV/RetNet 重新引入状态和线性时间处理。

9.24 常见误区

误区 1：Transformer 比 RNN 更省计算

不一定。Transformer attention 有 $O(n^2)$ 成本。它的优势主要是并行性和硬件友好，而不是所有情况下计算量更少。

误区 2：Scaling law 说明只要模型变大就一定更好

不对。参数、数据、compute、数据质量、训练稳定性需要匹配。Chinchilla 已经说明很多大模型可能 undertrained。

误区 3：涌现能力说明大模型能力完全不可预测

不准确。loss scaling 往往可预测，但部分下游能力在特定指标上可能呈现非线性或阈值效应。

误区 4：Attention 是 Transformer 表达力的全部来源

不对。Attention 负责路由，FFN 负责非线性计算，residual stream 负责跨层状态积累，norm 和初始化保证训练稳定。

误区 5：Scaling law 是理论定律

Scaling law 是经验规律，依赖实验条件，不能脱离模型族、数据分布和训练 recipe 使用。

误区 6：更低 loss 一定意味着更安全、更可靠

不一定。安全、事实性、鲁棒性、隐私、对齐需要专门数据、训练和评估。

9.25 面试高频问题

题 1：Transformer 为什么适合大规模并行训练？

参考回答：

Transformer 没有 RNN 那种强时间递推依赖。训练时整段序列可以一次性计算 Q/K/V 和所有 attention scores，大部分操作都是矩阵

题 2: Self-Attention 的表达力来自哪里?

参考回答:

Self-Attention 的表达力来自动态信息路由。每个 token 根据自己的 query 和其他 token 的 key 动态决定读取哪些 value。相比固定

题 3: Transformer 中 attention 和 FFN 分别负责什么?

参考回答:

可以粗略理解为 attention 负责 token 间信息交换和上下文路由, FFN 负责每个 token 位置上的非线性计算。Residual stream 在层

题 4: Scaling law 对大模型训练有什么意义?

参考回答:

Scaling law 说明在一定模型族、数据和训练设置下, 语言模型 loss 随参数量、数据量和计算量增加呈现相对平滑的幂律下降。这让

题 5: Chinchilla 的核心启发是什么?

参考回答:

Chinchilla 的核心启发是固定 compute 下参数量和训练 token 数需要平衡。很多早期大模型参数很大但训练 token 不足, 因此 under-optimal training 和高质量数据。

题 6: GPT-3 的 few-shot 能力说明了什么?

参考回答:

GPT-3 展示了大规模 decoder-only Transformer 可以通过 prompt 中的任务说明和少量示例完成新任务, 而不需要梯度更新。这说明 context learning 范式。但这不等同于参数级学习, 也不代表模型像人一样真正理解任务。

题 7: 如何理解 emergent abilities?

参考回答:

涌现能力指一些能力在小模型上不明显, 到了更大规模后突然出现。它说明部分下游能力可能不是简单线性可外推的。但要谨慎, 因

题 8: Scaling 的边界是什么?

参考回答:

Scaling 不是万能。loss 改善不保证所有任务、安全性、事实性、长上下文能力都同步提升。数据质量、训练稳定性、推理成本和对

9.26 小练习

1. 用一张表比较 RNN、CNN、Transformer 在并行性、长距离依赖和硬件友好性上的差异。
2. 解释为什么 Transformer 的 attention 是 $O(n^2)$, 但仍然适合大规模训练。
3. 用自己的话说明 Q/K/V 如何构成动态信息路由。
4. 解释多头注意力为什么比单头更有表达力。
5. 阅读 Kaplan scaling laws 摘要, 说明 loss、参数、数据、compute 之间的关系。
6. 阅读 Chinchilla 摘要, 解释为什么更大模型不一定 compute-optimal。
7. 讨论 GPT-3 few-shot learning 和 fine-tuning 的区别。
8. 举一个例子说明 “loss 下降不等于某项能力必然提升”。

9.27 本章总结

本章讲了 Transformer 的并行性、表达力和 scaling 优势。

核心结论：

1. Transformer 摆脱 RNN 的时间步串行依赖，训练时序列内部可以高度并行。
2. Transformer 的核心计算是规则的大矩阵乘法，非常适合 GPU/TPU 和分布式训练。
3. Self-Attention 提供动态内容路由，让任意 token 可以一层内直接交互。
4. 多头注意力、FFN、residual stream、normalization 共同构成 Transformer 的表达力。
5. Scaling law 让大模型训练从经验扩展走向更可预测的资源规划。
6. Chinchilla 强调 compute-optimal training，参数量和训练 token 数必须匹配。
7. GPT-3、PaLM 等工作展示了大规模 Transformer 的 few-shot、reasoning 和多任务能力。
8. Scaling 有边界，不能忽视数据质量、训练稳定性、推理成本、安全对齐和架构瓶颈。

下一章会进入 $O(n^2)$ Attention 的计算与显存瓶颈，解释 Transformer 的核心优势为什么也带来了长上下文和推理成本问题。

第十章： $O(n^2)$ Attention 的计算与显存瓶颈

10.0 本讲资料边界与第二轮精修口径

截至 2026-06-10，本讲只使用公开论文、官方文档和主流框架资料来校准概念边界。这里讨论的是标准 scaled dot-product attention 的计算、显存和 IO 瓶颈，不把某个框架版本的 kernel 选择、某个模型的长上下文 recipe、或某个 toy demo 的阈值写成通用标准。

本讲重点区分四层问题：

1. 数学连接模式：full attention 允许每个 query token 和每个可见 key token 交互，因此 pair 数随序列长度平方增长。
2. Kernel / IO 优化：FlashAttention 等方法做 exact attention 的分块、在线 softmax 和 IO 优化，不把 full attention 变成线性 attention。
3. 模型结构优化：sliding window、sparse attention、linear attention、GQA / MQA / MLA 等会改变可见模式、近似形式或 KV 表示。
4. Serving 内存管理：PagedAttention 主要管理 KV cache 的 block 分配、复用和碎片，不是替代 scaled dot-product attention 的数学结构。

后面的公式默认使用：

1. B 表示 batch size。
2. H 表示 query head 数。
3. T_q 表示 query length, T_k 表示 key/value length。
4. D_h 表示 head dimension。
5. s 表示每个元素的字节数，例如 fp16/bf16 通常可粗略取 $s=2$ 。

10.1 本章定位

上一章讲了 Transformer 的优势：并行性、表达力和 scaling。现在开始讲它的核心瓶颈。

Transformer 最大的结构性瓶颈之一，就是 self-attention 对序列长度的二次复杂度。

如果序列长度是 n ，每个 token 都和每个 token 计算一次相关性，那么 attention score 矩阵就是：

$n \times n$

当上下文从 2K 扩到 32K、128K、1M 时，问题会迅速放大。

本章要回答的问题是：

1. 为什么标准 self-attention 是 $O(n^2)$ 。
2. 这个 $O(n^2)$ 具体来自哪些张量和操作。

3. 训练时 attention 的显存瓶颈在哪里。
4. 为什么长上下文不只是“显存多一点”的问题。
5. 为什么 FlashAttention 能省显存、提速度，但不改变 attention 数学复杂度。
6. 为什么很多 linear/sparse attention 方法没有完全替代标准 attention。
7. 训练、prefill、decode 阶段的瓶颈有什么区别。
8. 面试中如何准确区分 $O(n^2)$ attention、KV cache、FlashAttention、PagedAttention。

本章的核心观点是：

Attention 的强表达力来自 token-to-token 全连接路由，而这个全连接路由也带来了序列长度平方级的计算、显存和 IO 压力。

10.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need. 提出 scaled dot-product attention, 其 attention score 来自 QK^T 。
2. Beltagy et al., 2020, Longformer: The Long-Document Transformer. 指出标准 self-attention 随序列长度二次增长, 并用 local window + global attention 处理长文档。
3. Wang et al., 2020, Linformer: Self-Attention with Linear Complexity. 尝试用低秩近似把 self-attention 降到线性复杂度。
4. Choromanski et al., 2020, Rethinking Attention with Performers. 用随机特征近似 softmax attention, 实现线性时间和空间复杂度。
5. Dao et al., 2022, FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 提出 IO-aware exact attention, 通过 tiling 减少 HBM 读写, 不近似 attention。
6. Dao, 2023, FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 进一步优化 work partitioning 和并行效率, 提高 attention kernel 性能。
7. PyTorch scaled_dot_product_attention 官方文档。用于校准当前框架中 math kernel、memory-efficient kernel、FlashAttention kernel、mask、dropout 和 grouped query attention 等接口边界。
8. Kwon et al., 2023, PagedAttention/vLLM. 讨论 LLM serving 中 KV cache 动态增长和内存管理问题。它更偏第 11 章, 但本章会用来区分 FlashAttention 和 PagedAttention。

需要说明的是, 本章重点讲 attention 的训练和 prefill 阶段的 $O(n^2)$ 瓶颈。KV cache 在自回归 decode 阶段的显存增长, 会在下一章专门展开。

10.3 标准 Attention 的计算流程

标准 scaled dot-product attention:

$$Q=XW_Q, \quad K=XW_K, \quad V=XW_V$$

$$S=\frac{QK^T}{\sqrt{D_h}}, \quad A=\mathrm{softmax}(S+M), \quad O=AV$$

这里 M 是可选 mask。causal mask、padding mask 或 packed sequence mask 最终都会影响 softmax 前的 score, 可见位置加 0, 不可见位置加一个足够小的负数。

常见 shape 是:

$$X \in \mathbb{R}^{B \times T \times d}, \quad Q, K, V \in \mathbb{R}^{B \times H \times T \times D_h}$$

$$S, A \in \mathbb{R}^{B \times H \times T_q \times T_k}, \quad O \in \mathbb{R}^{B \times H \times T_q \times D_h}$$

其中 $d=H \cdot D_h$ 。如果是普通 self-attention, 通常 $T_q=T_k=T$; 如果是 decode 单步, 通常 $T_q=1$, T_k 是当前历史长度。

1. X shape 是 [batch, seq_len, hidden]。
2. $Q/K/V$ shape 通常是 [batch, heads, seq_len, head_dim]。
3. S shape 是 [batch, heads, seq_len, seq_len]。
4. A shape 也是 [batch, heads, seq_len, seq_len]。
5. O shape 是 [batch, heads, seq_len, head_dim]。

瓶颈主要来自 S 和 A :

$$N_{\{\mathrm{pair}\}} = B \cdot H \cdot T_q \cdot T_k$$

这就是 $O(n^2)$ 的来源。

10.4 为什么是 $O(n^2)$

假设单个 batch、单个 head、序列长度 n 、head dimension d 。

QK^T 的计算是：

$$Q \in \mathbb{R}^{n \times d}, \quad K \in \mathbb{R}^{n \times d}, \quad QK^T \in \mathbb{R}^{n \times n}$$

每个 query token 都要和每个 key token 做一次 dot product。

dot product 的成本是 d ，token pair 数量是 n^2 ，所以计算量约为：

$$C_{\{QK\}} \approx 2n^2d$$

这里按一次 multiply-add 记 2 FLOPs。不同资料可能差一个常数因子，但不影响随 n^2 增长的结论。

再看 AV ：

$$A \in \mathbb{R}^{n \times n}, \quad V \in \mathbb{R}^{n \times d}, \quad AV \in \mathbb{R}^{n \times d}$$

这同样是：

$$C_{\{AV\}} \approx 2n^2d$$

所以 attention 主要计算量是：

$$C_{\{\mathrm{attn}\}} \approx 4n^2d$$

如果有 h 个 heads、batch size 是 b ，就是：

$$C_{\{\mathrm{attn}\}} \approx 4B \cdot H \cdot n^2 \cdot D_h$$

因为 $h \cdot d = \text{hidden_size}$ ，也可以粗略看成：

$$C_{\{\mathrm{attn}\}} = O(B \cdot n^2 \cdot d_{\{\mathrm{model}\}})$$

10.5 二次复杂度为什么可怕

序列长度翻倍，attention score 数量变成 4 倍。

例如单 head、单样本：

$$N_{\{\mathrm{pair}\}} = n^2$$

$n=2K$ -> 约 4M token pairs

$n=4K$ -> 约 16M token pairs

$n=8K$ -> 约 64M token pairs

$n=32K$ -> 约 1B token pairs

$n=128K$ -> 约 16B token pairs

这还只是单 head、单层、单样本。真实模型有几十层、几十个 heads、多个 batch。

所以长上下文训练不是简单地把 `max_position` 改大，也不是只要显存稍微多一点。

它会同时放大：

1. attention 计算量。
2. attention 中间激活显存。
3. HBM 读写压力。
4. backward 计算和保存张量。
5. 分布式通信和激活重计算成本。
6. 数据 packing 和 batch 构造复杂度。

10.6 Attention 显存主要花在哪里

训练时需要保存中间激活用于反向传播。

标准 attention 中，显存压力来自：

1. Q/K/V。
2. attention scores S 。
3. softmax 后的 attention probabilities A 。
4. dropout mask, 如果 attention dropout 开启。
5. output O 。
6. backward 所需的中间状态。

其中 S 和 A 是 $n \times n$ ，最容易爆炸。

粗略估算，假设：

$B=1$

$H=32$

$T=32768$

$s=2$ bytes

一个 attention score/probability 张量大小约为：

$$M_{\{\mathrm{score}\}} = BHT^2s$$

$$M_{\{\mathrm{score}\}} = 1 \times 32 \times 32768^2 \times 2 \times \mathrm{bytes} \approx 68.7 \times \mathrm{GB} \approx 64 \times \mathrm{GiB}$$

这只是一个 S 或 A 张量，还没算 $Q/K/V$ 、其他层、梯度、optimizer state。如果朴素实现同时 materialize scores 和 probabilities，粗略就是两倍：

$$M_{\{\mathrm{score+prob}\}} \approx 2BHT^2s$$

所以如果朴素 materialize 完整 attention matrix，32K 训练几乎不可接受。

10.7 训练显存不只是参数显存

很多初学者会先算模型权重：

$$7B \text{ 参数} \times 2 \text{ bytes} \approx 14GB$$

然后误以为一张 24GB 卡就可以轻松训练 7B。

真实训练显存包括：

1. 模型权重。
2. 梯度。
3. optimizer states, 例如 Adam 的 m/v 。
4. 前向激活。
5. attention scores/probabilities。
6. temporary buffers。
7. fragmentation 和框架开销。

在长序列训练中，activation 和 attention 中间张量可能比参数更难处理。

这也是为什么训练长上下文模型常用：

1. activation checkpointing。
2. FlashAttention。
3. sequence parallelism。
4. ZeRO/FSDP。
5. gradient accumulation。
6. mixed precision。
7. batch size / sequence length trade-off。

10.8 计算瓶颈和 IO 瓶颈

大模型训练不只是 FLOPs 问题，也受内存带宽影响。

GPU 有不同层级的内存：

HBM：容量较大，但访问比片上 SRAM 慢。

SRAM/shared memory/register：容量小，但访问快。

朴素 attention 可能会把 S 和 A 写入 HBM，再读回来继续计算。对于 $n \times n$ 的大矩阵，这会产生巨大 IO。

FlashAttention 的核心洞察就是：

attention 慢和占显存，不只是因为计算量大，也因为中间矩阵在 HBM 和片上内存之间反复读写。

所以高效 attention kernel 要关心 IO-awareness，而不只是数学复杂度。

10.9 FlashAttention 解决了什么

FlashAttention 是 exact attention，不改变数学结果。

它不是近似 attention，也不是 sparse attention。

它的核心做法是：

把 Q/K/V 分块加载到片上 SRAM 中。

分块计算 attention。

在线维护 softmax 的归一化统计。

避免 materialize 完整 $n \times n$ attention matrix 到 HBM。

普通 attention 可能显式产生：

$$S = QK^T$$

$$A = \text{softmax}(S)$$

$$O = AV$$

FlashAttention 逻辑上仍然计算同样的结果，但不把完整 S 和 A 常驻显存。

它带来的收益：

1. 显著减少 HBM 读写。
2. 训练时 attention activation 显存从二次级别显著降低。
3. wall-clock 速度提升。
4. 支持更长序列。
5. 保持 exact attention，不牺牲模型质量。

FlashAttention-2 进一步优化了 work partitioning 和 GPU 并行，使 attention kernel 更接近 GEMM 效率。

10.10 FlashAttention 没解决什么

FlashAttention 很重要，但不能误解。

它没有改变 full attention 的数学连接模式。

也就是说，每个 token 仍然要和每个 token 交互。理论计算复杂度仍然是：

$$O(n^2)$$

它主要优化的是：

IO、显存占用、kernel 执行效率

不是把 full attention 变成真正的 $O(n)$ 。

所以当序列长度极端变大，例如 1M tokens，即使用 FlashAttention，full attention 的计算量也可能不可接受。

此外，FlashAttention 对 mask pattern 和硬件 kernel 有要求。任意复杂 mask 不一定能高效支持。

10.11 为什么 softmax attention 难以简单线性化

标准 attention 是：

$$O = \mathrm{softmax}(QK^{\top})V$$

问题在于 softmax 作用在每个 query 对所有 keys 的 score 上。每个 query 的归一化分母不同：

$$Z_i = \sum_{j=1}^n \exp(q_i^{\top} k_j)$$

这使得 attention 天然像一个全局 pairwise 操作。

如果想降到线性复杂度，常见思路包括：

1. 稀疏化：只算部分 token pair。
2. 低秩近似：认为 attention matrix 可压缩。
3. kernel approximation：用特征映射近似 softmax kernel。
4. 局部窗口：限制可见范围。
5. recurrence/state：用固定大小状态摘要历史。
6. memory/retrieval：只取相关片段进入上下文。

每条路线都在 trade-off：

效率、精度、表达力、训练稳定性、硬件友好性、生态兼容性

10.12 Sparse Attention

Sparse attention 的思路是：不是所有 token pair 都计算。

例如：

local window: 每个 token 只看附近 w 个 token

global token: 少数 token 可以看全局

strided pattern: 按步长连接远处 token

block sparse: 按块组织稀疏连接

如果每个 token 只看 w 个 token，复杂度可以从：

$$O(n^2)$$

降到：

$$O(nw)$$

当 w 远小于 n 时，接近线性。

Longformer 就使用 local window attention 加 task-motivated global attention，适合长文档任务。

Sparse attention 的问题是：

1. 远距离 token 不一定能直接交互。
2. pattern 设计影响任务效果。
3. 某些稀疏 pattern 硬件上不一定高效。
4. 对需要任意位置精确检索的任务可能不如 full attention。

10.13 Linear Attention 和 Kernel Approximation

Linear attention 试图避免显式构造 $n \times n$ attention matrix。

一种思路是把 softmax kernel 近似成特征映射内积：

$$\exp(q^{\top} k) \approx \phi(q)^{\top} \phi(k)$$

于是：

$$\mathrm{softmax}(QK^{\top})V$$

可以改写为先聚合 K/V，再与 Q 交互，从而把复杂度降到线性级别。

Performer 使用 FAVOR+ 随机特征近似 softmax attention，目标是在理论上提供近似保证。

Linformer 则从低秩角度出发，认为 self-attention matrix 可以被低秩投影近似，从而降低时间和空间复杂度。

这些方法的值是探索了 full attention 之外的可能性。

但它们没有完全取代标准 attention，原因包括：

1. 近似可能影响模型质量。
2. 对不同任务、规模、数据的稳定性不一。
3. 工程生态和 kernel 优化不如标准 attention 成熟。
4. 大规模 LLM 中 full attention + FlashAttention 的路线效果强且可靠。
5. 现代模型还可以通过 GQA、sliding window、RoPE scaling、检索等组合缓解瓶颈。

10.14 $O(n^2)$ 在训练、Prefill、Decode 中不同

大模型使用有三个阶段要区分。

训练

训练时通常整段序列并行输入：

$$T_q=T_k=n$$

attention 是完整的 $n \times n$ ，即使 causal mask 屏蔽未来，朴素矩阵仍然是三角形级别，复杂度仍是 $O(n^2)$ 。

Prefill

推理开始时，用户 prompt 一次性送入模型，构建第一批 KV cache。

如果 prompt 长度是 n ：

$$C_{\{\mathrm{prefill}\}} \propto n^2$$

长 prompt 的首 token 延迟，也就是 time-to-first-token，经常受 prefill 影响。

Decode

自回归生成时，每次只生成一个新 token。

新 token 的 $query_len$ 是 1， key_len 是历史长度 t ：

$$T_q=1, \quad T_k=t, \quad C_{\{\mathrm{decode-step}\}} \propto t$$

如果生成 m 个 token，总成本会累积。

$$C_{\{\mathrm{decode-total}\}} \propto \sum_{r=1}^m (n+r-1)$$

Decode 阶段的核心瓶颈通常还包括 KV cache 显存和内存带宽。下一章会重点讲。

10.15 Causal Mask 是否把复杂度减半

Causal attention 只允许看过去，所以有效可见 token pair 是下三角：

$$N_{\{\mathrm{causal}\}} = \frac{n(n+1)}{2}$$

相比 full bidirectional attention 的 n^2 ，大约少一半。

但复杂度阶数仍然是：

$$O(n^2)$$

所以 causal mask 不能从根本上解决长序列瓶颈。

它只是把常数因子降低，并保证不看未来。

10.16 长上下文训练为什么难

把上下文从 4K 扩到 32K，不只是长度变成 8 倍。

attention token pair 变成：

$$8^2 = 64 \text{ 倍}$$

这会影响：

1. 单 step 训练时间。
2. 激活显存。
3. batch size。
4. 梯度累计策略。
5. 数据吞吐。
6. 分布式通信。
7. checkpoint 保存和恢复时间。
8. 训练稳定性和 loss spike 排查。

很多长上下文训练必须牺牲 batch size 或使用更复杂并行策略。

这也解释了为什么“支持 128K 上下文”的模型不一定是在 128K 上从头 full attention 训练出来的。它可能结合了：

1. RoPE scaling / interpolation。
2. 长上下文继续训练。
3. sliding window 或 sparse attention。
4. synthetic long-context tasks。
5. retrieval-augmented training。
6. FlashAttention / sequence parallel。

10.17 Attention 瓶颈和 FFN 瓶颈的关系

Transformer block 中还有 FFN。

FFN 计算大约随序列长度线性增长：

$$C_{\{\mathrm{ffn}\}} = O(n d_{\{\mathrm{model}\}} d_{\{\mathrm{ffn}\}})$$

Attention 则有：

$$C_{\{\mathrm{attn}\}} = O(n^2 d_{\{\mathrm{model}\}})$$

当 n 较短时，FFN 可能占主要 FLOPs，尤其现代 LLM 的 FFN hidden size 很大。

当 n 很长时，attention 的 n^2 项会快速增长，成为主要瓶颈。

所以不同场景瓶颈不同：

短上下文大模型：FFN/矩阵乘可能很重。

长上下文 full attention: attention score 和 AV 可能成为瓶颈。

高并发推理：KV cache 和内存带宽可能成为瓶颈。

面试中要避免一句“attention 一定是最大瓶颈”说死。要结合序列长度、模型结构、训练/推理阶段和硬件实现判断。

10.18 Memory-Efficient Attention 和 Activation Checkpointing

除了 FlashAttention, 还有一些通用减显存方法。

Activation checkpointing 的思想是:

前向时少存中间激活, 反向时重新计算。

这可以降低显存, 但会增加计算时间。

Memory-efficient attention 的思想是避免保存完整 attention probabilities, 通过重计算或分块计算降低显存。

这些方法本质上都在做 trade-off:

显存 ↓

计算/实现复杂度 ↑

FlashAttention 更进一步利用 GPU memory hierarchy, 把分块、在线 softmax 和 IO 优化结合起来, 因此成为现代训练和推理中非常常见的 attention kernel。

10.19 FlashAttention、PagedAttention、GQA 不是一回事

这三个词经常被混淆。

FlashAttention:

优化 attention kernel。

不改变数学 attention。

主要减少 HBM 读写和中间矩阵显存。

PagedAttention:

优化 serving 中 KV cache 的内存管理。

灵感来自操作系统分页。

减少碎片和重复, 提升 batch size 和吞吐。

GQA/MQA:

改变 attention 架构中的 K/V head 数。

从源头减少 KV cache 大小和读取带宽。

三者可以组合使用, 但解决的问题层级不同:

FlashAttention: kernel/IO 层。

PagedAttention: serving memory manager 层。

GQA/MQA: 模型架构层。

10.20 面向专家: 为什么 IO-aware 很关键

理论 FLOPs 不是全部。

一个算法如果需要频繁读写 HBM, 即使 FLOPs 不高, 也可能很慢。

朴素 attention 的问题之一是:

QK^T 生成 S, 写到 HBM。

softmax 读 S, 写 A。

AV 读 A 和 V, 写 O。

当 S/A 是巨大 $n \times n$ 矩阵时, HBM 流量非常大。

FlashAttention 用 tiling 让计算尽量在片上 SRAM 中完成, 并用 online softmax 避免保存完整 attention matrix。

这就是 IO-aware 的意义:

同样的数学结果，如果减少慢速内存读写，就能显著提升真实 wall-clock 性能。

这也是为什么一些理论上低复杂度的近似 attention，不一定在真实硬件上更快。稀疏、不规则、小矩阵操作可能无法充分利用 GPU。

10.21 面向专家：为什么近似 Attention 难替代 Full Attention

近似 attention 的目标很明确：降低复杂度。

但替代 full attention 很难，因为 full attention 有几个强优势：

1. 任意 token pair 可直接交互。
2. 数学形式简单稳定。
3. 大规模预训练效果反复验证。
4. kernel 生态成熟。
5. 支持 causal、bidirectional、cross-attention 等多种模式。
6. 与 RoPE、GQA、FlashAttention、tensor parallel 等生态高度兼容。

近似方法常见问题：

1. 任务依赖强。
2. 长距离精确检索可能受损。
3. 训练稳定性和超参更敏感。
4. 理论复杂度低但硬件利用率不一定高。
5. 大规模 LLM 复现和生态成本高。

所以实际路线经常是折中：

核心层保留 full attention 或局部 full attention。

用 FlashAttention 优化 exact attention。

用 GQA/MQA 降 KV cache。

用 sliding window/sparse pattern 扩上下文。

用 retrieval/memory 减少必须放进上下文的 token。

10.22 Attention 成本审计指标与最小 demo

第二轮精修时，本章建议把 attention 瓶颈落到一组可审计指标，而不是只背 “ $O(n^2)$ ”：

1. attention_pair_count: full、causal、sliding window 下分别有多少 query-key pair。
2. score_tensor_memory: 如果 materialize score matrix，单个 S 张量要多少显存。
3. score_plus_prob_memory: 如果同时 materialize S 和 A，中间矩阵要多少显存。
4. prefill_tflops_per_layer: 长 prompt prefill 每层 attention 的粗略 FLOPs。
5. decode_step_tflops_per_layer: 单步 decode 在同样历史长度下的 attention FLOPs。
6. window_pair_ratio: sliding window 相对 full attention 保留了多少 pair。
7. attention_cost_gate: 把显存、FLOPs、上下文长度和部署预算组合成门禁。

下面是一个 0 依赖 demo。它不模拟真实 GPU kernel，只演示 “从 4K 到 32K 为什么是平方级放大”，以及 prefill 和 decode 的成本口径为什么不能混用。

```
BYTES_PER_ELEM = 2
```

```
def gib(num_bytes):  
    return num_bytes / (1024 ** 3)
```

```
def full_pairs(batch, heads, q_len, k_len):  
    return batch * heads * q_len * k_len
```

```

def causal_visible_pairs(batch, heads, seq_len):
    return batch * heads * seq_len * (seq_len + 1) // 2

def attention_tflops(pair_count, head_dim):
    # QK^T and AV each cost roughly 2 * pairs * head_dim FLOPs.
    return 4 * pair_count * head_dim / 1e12

batch = 1
heads = 32
head_dim = 128
short_len = 4096
long_len = 32768
window = 4096

short_full = full_pairs(batch, heads, short_len, short_len)
long_full = full_pairs(batch, heads, long_len, long_len)
long_causal = causal_visible_pairs(batch, heads, long_len)
window_pairs = full_pairs(batch, heads, long_len, min(window, long_len))
decode_pairs = full_pairs(batch, heads, 1, long_len)

failed_gates = []
if gib(long_full * BYTES_PER_ELEM) > 16:
    failed_gates.append("full_score_tensor_over_budget")
if attention_tflops(long_full, head_dim) > 10:
    failed_gates.append("prefill_tflops_over_budget")
if window_pairs / long_full > 0.25:
    failed_gates.append("window_not_sparse_enough")

print("short_vs_long_pair_multiplier=", round(long_full / short_full, 2))
print("full_32k_score_tensor_gib=", round(gib(long_full * BYTES_PER_ELEM), 2))
print("full_32k_score_plus_prob_gib=", round(gib(2 * long_full * BYTES_PER_ELEM), 2))
print("causal_visible_32k_score_gib=", round(gib(long_causal * BYTES_PER_ELEM), 2))
print("prefill_32k_tflops_per_layer=", round(attention_tflops(long_full, head_dim), 4))
print("decode_one_step_32k_tflops_per_layer=", round(attention_tflops(decode_pairs, head_dim), 6))
print("sliding_window_4k_pair_ratio=", round(window_pairs / long_full, 3))
print("failed_gates=", failed_gates)
print("attention_cost_gate_pass=", len(failed_gates) == 0)

```

示例输出：

```

short_vs_long_pair_multiplier= 64.0
full_32k_score_tensor_gib= 64.0
full_32k_score_plus_prob_gib= 128.0
causal_visible_32k_score_gib= 32.0
prefill_32k_tflops_per_layer= 17.5922
decode_one_step_32k_tflops_per_layer= 0.000537
sliding_window_4k_pair_ratio= 0.125
failed_gates= ['full_score_tensor_over_budget', 'prefill_tflops_over_budget']
attention_cost_gate_pass= False

```

这个 demo 的读法是：

1. 4K 到 32K 的长度是 8 倍，但 full attention pair 是 64 倍。
2. 32K 下单个 full score tensor 已经约 64 GiB；如果还 materialize probability tensor，就是约 128 GiB。
3. causal mask 的可见 pair 约少一半，但阶数仍然是平方级。

4. prefill 的每层 attention FLOPs 和 decode 单步不是一个量级, 因此长 prompt 的 TTFT 和长输出 decode 的瓶颈要分开分析。
5. sliding window 能把 pair ratio 降到 0.125, 但它改变了可见模式, 需要再评估长距离证据召回和任务质量。

10.23 常见误区

误区 1: FlashAttention 把 $O(n^2)$ 变成 $O(n)$

不对。FlashAttention 是 exact attention, 主要优化 IO 和显存, 不改变 full attention 的二次计算关系。

误区 2: $O(n^2)$ 只是训练问题

不完全。训练和 prefill 有明显 $O(n^2)$ attention 成本; decode 每步是 $O(t)$, 但会受到 KV cache 显存和内存带宽限制。

误区 3: Causal mask 解决了二次复杂度

不对。Causal mask 只保留下三角, 约少一半 token pair, 但阶数仍是 $O(n^2)$ 。

误区 4: 参数显存够就能训练长上下文

不对。长上下文训练中 attention activation、中间矩阵、optimizer state、梯度和临时 buffer 都会占显存。

误区 5: Linear attention 一定比 full attention 好

不一定。它可能降低理论复杂度, 但可能带来近似误差、任务效果下降、训练不稳定或硬件利用率不足。

误区 6: PagedAttention 是一种模型 attention 结构

不准确。PagedAttention 主要是 serving 系统中的 KV cache 管理方法, 不是像 MHA/GQA 那样的模型结构。

10.24 面试高频问题

题 1: 为什么标准 self-attention 是 $O(n^2)$?

参考回答:

因为每个 query token 都要和每个 key token 计算一次相关性。序列长度为 n 时, attention score matrix 的形状是 $n \times n$ 。 QK^T 和

题 2: Attention 的显存瓶颈在哪里?

参考回答:

训练时除了 $Q/K/V$, 还可能需要保存 attention scores 和 softmax 后的 attention probabilities, 它们的形状是 $batch * heads * s$

题 3: FlashAttention 解决了什么?

参考回答:

FlashAttention 是 IO-aware 的 exact attention。它通过 tiling、在线 softmax 等方法, 避免把完整 attention score/probability

题 4: FlashAttention 是否把复杂度从 $O(n^2)$ 降到 $O(n)$?

参考回答:

没有。FlashAttention 不改变 full attention 的 token pair 交互模式, 每个 token 仍然需要和其他 token 计算注意力, 所以理论计

题 5: 训练、prefill、decode 的 attention 成本有什么区别?

参考回答:

训练时整段序列并行输入, query_len 和 key_len 都是 n, attention 是 $O(n^2)$ 。Prefill 阶段处理 prompt, 也需要对 prompt 内 token 进行 attention 计算。

题 6: Sparse attention 和 linear attention 的区别是什么?

参考回答:

Sparse attention 是减少 token pair, 只计算局部窗口、全局 token 或块稀疏连接等部分 attention。Linear attention 通常试图通过近似来降低复杂度。

题 7: 为什么理论复杂度低的方法不一定真实更快?

参考回答:

GPU 性能不仅取决于 FLOPs, 还取决于内存访问、kernel fusion、矩阵形状、并行度和硬件利用率。一些稀疏或近似 attention 理论复杂度低, 但实际性能不一定优于优化良好的 full attention + FlashAttention。

题 8: FlashAttention、PagedAttention、GQA 分别优化什么?

参考回答:

FlashAttention 优化 attention kernel 和 GPU IO, 不改变 attention 数学结果。PagedAttention 优化 LLM serving 中 KV cache 的内存管理。

10.25 小练习

1. 推导单 head attention 中 QK^T 和 AV 的计算复杂度。
2. 计算 batch=1、heads=32、seq_len=8192、fp16 下一个 attention probability 矩阵大约占多少显存。
3. 解释为什么序列长度从 4K 到 32K, attention token pair 增长 64 倍。
4. 用自己的话解释 FlashAttention 为什么是 exact attention。
5. 比较 full attention、sliding window attention、linear attention 的优缺点。
6. 解释为什么 causal mask 不能从根本上解决 $O(n^2)$ 。
7. 画出训练、prefill、decode 三个阶段的 query_len/key_len 关系。
8. 用一张表区分 FlashAttention、PagedAttention、GQA/MQA。
9. 运行 10.22 的 attention 成本审计 demo, 把 long_len 从 32768 改成 65536, 解释 score tensor 显存和 prefill FLOPs 如何变化。
10. 把 window 从 4096 改成 8192, 观察 sliding_window_4k_pair_ratio 的变化, 并说明为什么更大的窗口不是免费提升。

10.26 本章总结

本章讲了 $O(n^2)$ Attention 的计算与显存瓶颈。

核心结论:

1. 标准 self-attention 的 QK^T 会产生 $n \times n$ attention score matrix。
2. Attention 的计算复杂度随序列长度平方增长, 长上下文下会迅速放大。
3. 训练时 attention scores/probabilities 等中间张量会造成巨大显存压力。
4. 长上下文训练不只是改 max length, 还涉及显存、IO、batch size、分布式和稳定性。
5. FlashAttention 是 IO-aware exact attention, 减少 HBM 读写和中间显存, 但不改变 $O(n^2)$ 数学复杂度。
6. Sparse/linear attention 试图降低复杂度, 但会带来表达力、近似误差、硬件效率和生态兼容性 trade-off。
7. 训练、prefill、decode 的 attention 瓶颈不同, 不能混为一谈。
8. FlashAttention、PagedAttention、GQA/MQA 分别优化 kernel、serving memory manager 和模型结构层面。
9. Attention 成本审计要同时看 pair count、score/prob memory、prefill FLOPs、decode step、window ratio 和质量风险。

下一章会进入 KV Cache、长上下文推理和显存增长问题，专门解释自回归 decode 阶段为什么 KV cache 会成为 serving 的核心瓶颈。

第十一章：KV Cache、长上下文推理和显存增长问题

11.0 本讲资料边界与第二轮精修口径

截至 2026-06-10，本讲用公开论文、官方文档和主流框架资料校准 KV cache 的通用概念。这里讨论 decoder-only LLM 推理中的 K/V 缓存、显存估算、decode 带宽、PagedAttention 和 MQA / GQA / MLA 的结构差异，不把某个 serving 框架的 block size、scheduler 策略、cache layout、量化配置或某个模型报告中的压缩比例写成通用标准。

本讲重点区分五层问题：

1. 推理语义：KV cache 是推理时的运行时状态，不是模型权重，也通常不用于训练反向传播。
2. 张量规模：每层历史 token 的 K 和 V 都要缓存，大小随层数、batch、上下文长度、KV head 数、head dim 和精度线性增长。
3. Decode 带宽：decode 单步计算量不再是完整 T^2 ，但每步要读取历史 K/V，长上下文和高并发下常受显存容量与带宽限制。
4. 架构压缩：MQA / GQA 减少 KV head 数，MLA 压缩 KV 表示，属于模型结构层面的 KV cache 优化。
5. 系统管理：PagedAttention、prefix cache、continuous batching 和 block manager 管理动态请求和 KV blocks，不改变 scaled dot-product attention 的数学定义。

后面的公式默认使用：

1. L 表示 Transformer 层数。
2. B 表示活跃请求 batch size。
3. T 表示每个请求当前上下文长度。
4. H_q 表示 query head 数， $H_{\{\mathrm{kv}\}}$ 表示 K/V head 数。
5. D_h 表示每个 head 的维度。
6. s 表示每个 cache 元素的字节数，例如 fp16/bf16 通常粗略取 $s=2$ 。

11.1 本章定位

上一章讲了训练和 prefill 阶段中 $O(n^2)$ attention 的计算与显存瓶颈。本章进入自回归推理阶段的核心问题：KV Cache。

对 decoder-only LLM 来说，推理不是一次性输出完整答案，而是逐 token 生成：

prompt -> 生成第 1 个 token -> 追加到上下文 -> 生成第 2 个 token -> ...

如果每生成一个 token 都重新计算所有历史 token 的 K/V，成本会非常高。KV cache 的作用就是缓存历史 token 在每一层的 key 和 value，后续生成时直接复用。

KV cache 让自回归推理变快，但也带来了新的问题：

上下文越长、batch 越大、层数越多、KV heads 越多，显存占用越大。

本章要回答的问题是：

1. KV cache 缓存的到底是什么。
2. 没有 KV cache 时推理为什么会重复计算。
3. KV cache 显存如何估算。
4. 为什么长上下文 serving 经常被 KV cache 卡住。
5. prefill、decode、continuous batching 的瓶颈有什么区别。
6. MHA、MQA、GQA、MLA 如何影响 KV cache。
7. PagedAttention/vLLM 解决了什么问题。
8. 长上下文推理中延迟、吞吐、显存和质量如何 trade-off。

本章的核心观点是：

KV cache 是 decoder-only LLM 高效推理的必要条件，也是长上下文、高并发 serving 的核心显存和带宽瓶颈。

11.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Vaswani et al., 2017, Attention Is All You Need。提出 Transformer attention 结构，decoder 生成时需要读取历史上下文。
2. Shazeer, 2019, Fast Transformer Decoding: One Write-Head is All You Need。提出 Multi-Query Attention，指出 incremental decoding 中反复加载 K/V 的内存带宽成本很高。
3. Ainslie et al., 2023, GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints。提出 GQA，在质量和推理速度之间折中。
4. Kwon et al., 2023, Efficient Memory Management for Large Language Model Serving with PagedAttention。提出 PagedAttention/vLLM，用分页式管理 KV cache，降低碎片和重复。
5. Dao et al., 2022, FlashAttention。作为对比：FlashAttention 优化 attention kernel 和 IO，不从架构上减少 KV cache 总量。
6. DeepSeek-AI, 2024, DeepSeek-V2。提出 MLA，压缩 KV cache 到 latent vector，并报告大幅减少 KV cache 和提升生成吞吐。
7. Hugging Face Transformers cache 官方文档。用于校准 past_key_values、cache position、attention mask 和生成时 cache 更新的实现口径。
8. vLLM 官方文档与设计说明。用于校准 KV cache block、block table、PagedAttention 和 continuous batching 的系统边界。

需要说明的是，KV cache 的具体实现和显存占用会受框架、并行策略、量化、分页粒度、batch scheduler、模型结构影响。本章重点讲通用原理和估算方法。

11.3 自回归推理为什么需要 KV Cache

decoder-only LLM 每一步都要预测下一个 token。

假设 prompt 是：

$x_1, x_2, \dots, x_n$

生成第一个新 token 时，需要对这 n 个 token 做前向计算。

生成第二个新 token 时，上下文变成：

$x_1, x_2, \dots, x_n, y_1$

如果没有 cache，模型会重新计算所有 $x_1 \dots x_n$ 的 K/V。生成第 t 个 token 时，又会重复计算前面所有历史 token。

这很浪费，因为历史 token 的 hidden state 在 causal attention 中不会因为未来 token 出现而改变。

对于已经处理过的 token，它们在每层产生的 K/V 可以缓存下来：

历史 token 的 K/V 不变，新 token 只需要计算自己的 Q/K/V。

这就是 KV cache。

11.4 KV Cache 缓存什么

在每一层 attention 中，模型会计算：

$Q=XW_Q, K=XW_K, V=XW_V$

对新 token 来说，当前层的 query q_t 要和所有历史 key 做点积：

$s_{t,j}=\frac{q_t \cdot k_j}{\sqrt{D_h}}$

然后用 attention weights 加权所有历史 value：

$$o_t = \sum_{j=1}^t a_{t,j} v_j$$

因此推理时需要保存每一层、每个历史 token 的：

K cache
V cache

通常 shape 可以理解为：

$$K_{\{\mathrm{cache}\}}, V_{\{\mathrm{cache}\}} \in \mathbb{R}^{B \times H_{\{\mathrm{kv}\}} \times T \times D_h}$$

实际系统可能为了 kernel 访问效率使用不同布局，例如按 block、page、layer 或 head 组织。

11.5 有无 KV Cache 的复杂度对比

假设 prompt 长度为 n ，要生成 m 个 token。

没有 KV Cache

每一步都重新计算完整上下文。

第 1 步上下文长度约 n 。第 2 步上下文长度约 $n+1$ 。第 m 步上下文长度约 $n+m-1$ 。

每一步都要重新跑历史 token 的 K/V 和 attention。

这会造成大量重复计算。

有 KV Cache

prefill 阶段：

一次性处理 prompt，缓存 prompt 的 K/V。

decode 阶段：

每步只计算新 token 的 Q/K/V。

新 token 的 Q 读取历史 K/V cache。

把新 token 的 K/V 追加进 cache。

这样每步不再重复计算历史 token 的 K/V。

KV cache 的收益非常大，是现代 decoder-only LLM serving 的基础。

11.6 Prefill 和 Decode

LLM 推理一般分为两个阶段。

Prefill

prefill 处理输入 prompt。

如果 prompt 长度是 n ：

$$T_q = T_k = n$$

它需要对 prompt 内部做 causal attention，成本近似 $O(n^2)$ 。

prefill 的输出包括：

1. 最后一个位置的 logits，用来生成第一个 token。
2. prompt 中所有 token 的 KV cache。

prefill 影响首 token 延迟，也就是 TTFT：

time to first token

Decode

decode 每次生成一个新 token。

此时：

$T_q=1, \quad T_k=T_{\mathrm{past}}+1$

单步 attention 成本约 $O(\text{past_len})$ ，但需要读取历史 K/V cache。

decode 影响输出速度，也就是：

tokens per second

长上下文推理中，prefill 和 decode 都可能成为瓶颈，只是瓶颈类型不同。

11.7 KV Cache 显存公式

KV cache 粗略大小：

$M_{\mathrm{kv}} = L \cdot B \cdot T \cdot H_{\mathrm{kv}} \cdot D_h \cdot 2 \cdot s$

其中：

1. num_layers 是层数。
2. batch_size 是同时服务的请求数。
3. seq_len 是每个请求当前上下文长度。
4. num_kv_heads 是 K/V head 数。
5. head_dim 是每个 head 的维度。
6. 2 是 K 和 V 两份。
7. bytes_per_element 对 fp16/bf16 通常是 2。

注意，这里用的是 num_kv_heads，不一定等于 num_q_heads。MHA、MQA、GQA 的差异主要就在这里。

如果只看单个 token 给一个请求追加到所有层的 KV cache，粗略是：

$M_{\mathrm{kv/token}} = L \cdot H_{\mathrm{kv}} \cdot D_h \cdot 2 \cdot s$

这也是长输出 decode 时 cache 持续增长的基本单位。

11.8 KV Cache 粗略估算

假设一个模型：

```
num_layers = 32
batch_size = 16
seq_len = 8192
num_kv_heads = 32
head_dim = 128
dtype = bf16 = 2 bytes
```

KV cache 大小：

$M_{\mathrm{kv}} = 32 \cdot 16 \cdot 8192 \cdot 32 \cdot 128 \cdot 2 \cdot 2 \cdot \mathrm{bytes}$

$M_{\mathrm{kv}} \approx 68.7 \cdot \mathrm{GB} \approx 64 \cdot \mathrm{GiB}$

这只是 KV cache，不包括：

1. 模型权重。
2. temporary buffers。
3. logits。
4. runtime workspace。
5. memory fragmentation。
6. 框架和通信开销。

如果上下文扩到 32K, 其他不变:

seq_len 变成 4 倍

KV cache 也变成 4 倍

这就是为什么长上下文 + 高并发 serving 会非常吃显存。

11.9 KV Cache 是线性增长, 但仍然可怕

和训练 attention matrix 的 $O(n^2)$ 不同, KV cache 对序列长度是线性增长:

$$M_{\{\mathrm{kv}\}} \propto T$$

但它仍然可怕, 因为常数很大:

$$L B H_{\{\mathrm{kv}\}} D_h \times 2$$

而且 serving 时需要同时服务多个请求。

单个请求 128K 上下文可能还能勉强放下, 但如果要并发服务几十个请求, KV cache 会迅速吃光显存。

所以长上下文产品化的难点不是“单条样本能不能跑”, 而是:

在可接受延迟和成本下, 能不能高吞吐稳定服务大量长请求。

11.10 Decode 为什么常受内存带宽限制

decode 阶段每步只处理一个新 token, 矩阵乘法规模较小, 不一定能充分利用 GPU 的峰值算力。

同时, 每一步都要读取历史 K/V:

new query $\rightarrow$ read all historical K/V $\rightarrow$ compute attention $\rightarrow$ produce next token

当上下文很长、batch 很大时, 读取 KV cache 的内存带宽会成为瓶颈。

这也是 Shazeer 提出 MQA 的关键动机:

incremental decoding 慢, 很多时候不是因为 FLOPs 不够, 而是因为反复加载巨大的 K/V tensors。

所以优化 decode, 不只是优化计算, 也要减少:

1. KV cache 大小。
2. KV cache 读取带宽。
3. cache 碎片。
4. 重复 cache。
5. batch 内 padding 和无效计算。

11.11 MHA、MQA、GQA 对 KV Cache 的影响

标准 MHA:

$$H_q = H_{\{\mathrm{kv}\}} = H$$

每个 query head 有独立 K/V head, 表达能力完整, 但 KV cache 最大。

MQA:

$$H_q = H, \quad H_{\{\mathrm{kv}\}} = 1$$

所有 query heads 共享一组 K/V, KV cache 约降到 MHA 的 $1/H$ 。

GQA:

$$H_q = H, \quad H_{\{\mathrm{kv}\}} = G, \quad 1 < G < H$$

每组 query heads 共享一个 K/V head, 是 MHA 和 MQA 之间的折中。

例如:

$H = 32$

$GQA \text{ num_kv_heads} = 8$

KV cache 约为 MHA 的：

$$\frac{H_{\{\mathrm{kv}\}}}{H_q} = \frac{8}{32} = \frac{1}{4}$$

这就是为什么现代 LLM 很常用 GQA。它在质量和推理成本之间更平衡。

11.12 MLA：从减少 KV Heads 到压缩 KV 表示

MLA，也就是 Multi-head Latent Attention，是更激进的 KV cache 优化路线。

GQA/MQA 的思路是减少 K/V head 数。

MLA 的思路更像是：

不要直接缓存完整 K/V，而是缓存更低维的 **latent** 表示，需要时再恢复或投影出 **attention** 所需信息。

DeepSeek-V2 报告中，MLA 显著压缩 KV cache，并提升最大生成吞吐。

从架构演进角度看：

MHA：每个 head 都缓存 K/V。

MQA/GQA：减少 K/V head 数。

MLA：压缩 K/V 表示本身。

MLA 的代价是实现复杂度更高，对训练、推理 kernel、权重格式、生态兼容性都有更高要求。

11.13 PagedAttention 解决什么

KV cache 不只是大，还会动态变化。

Serving 中每个请求长度不同：

请求 A：prompt 200 tokens，生成 50 tokens

请求 B：prompt 8000 tokens，生成 500 tokens

请求 C：prompt 1000 tokens，生成 20 tokens

请求会不断加入、结束、扩展。KV cache 会动态增长和释放。

如果用连续大块内存为每个请求分配 KV cache，容易出现：

1. 内存碎片。
2. 预留过多。
3. batch size 受限。
4. prefix 共享困难。
5. beam search 或 parallel sampling 中重复 cache。

PagedAttention 借鉴操作系统分页思想，把 KV cache 划分为固定大小的 blocks/pages。

逻辑上请求有连续 token 序列，物理上 KV cache 可以分散存储在不同 blocks 中，通过 block table 映射。

这样可以：

1. 减少碎片。
2. 接近按需分配。
3. 更灵活地释放和复用 blocks。
4. 支持 prefix cache sharing。
5. 提升可容纳 batch size。

vLLM 就是围绕 PagedAttention 构建的高吞吐 LLM serving 系统。

11.14 Continuous Batching

传统 batching 可能要求一批请求一起开始、一起结束。

但 LLM 生成长度不同，如果等待最长请求完成，会浪费大量算力。

Continuous batching 的思想是：

每个 `decode step` 动态调度请求。

完成的请求移出 `batch`。

新的请求加入 `batch`。

这能提高 GPU 利用率和吞吐。

但它也要求系统高效管理 KV cache：

1. 请求动态加入。
2. 请求动态结束。
3. 每个请求 `past length` 不同。
4. KV cache blocks 频繁分配和释放。
5. attention kernel 要支持 `varlen batch`。

所以 continuous batching 和 PagedAttention 往往是配套问题。

11.15 Prefix Sharing

很多请求共享相同前缀。

例如：

1. 同一个 `system prompt`。
2. 同一个长文档问多个问题。
3. beam search 中多个候选共享前缀。
4. parallel sampling 中多个采样分支共享 `prompt`。
5. Agent 多次调用模型时共享历史上下文。

如果每个请求都复制一份 prefix KV cache，会浪费显存。

Prefix sharing 的目标是：

共享相同 `prefix` 的 KV cache，只为分叉后的 `token` 单独存储。

PagedAttention 的 block/page 设计有利于实现这种共享。

但 prefix sharing 也带来工程复杂度：

1. 引用计数。
2. block 生命周期。
3. cache 命中判断。
4. 不同请求 `position ids` 对齐。
5. 安全隔离和多租户问题。

11.16 Sliding Window Cache

如果模型使用 sliding window attention，每个新 token 只关注最近 `w` 个历史 token。

那么理论上不需要保留全部历史 K/V 给每层 attention 使用。

可以只保留窗口内 cache：

$\text{cache length} \approx \text{window size}$

这能显著降低长上下文 decode 的 cache 显存。

但代价是：

1. 模型无法直接访问窗口外历史。
2. 长距离信息需要通过其他机制保留。
3. 需要和位置编码、mask、cache eviction 配套。
4. 对任务质量有影响。

有些模型会结合 local window、global tokens、summary memory 或 retrieval 来弥补窗口限制。

11.17 Cache Quantization

KV cache 可以量化。

如果 fp16/bf16 cache 每元素 2 bytes，量化到 int8 可以近似减半，量化到更低 bit 可以进一步减少。

好处：

1. 降显存。
2. 降内存带宽。
3. 提高可并发 batch size。

风险：

1. attention score 或 value 聚合误差。
2. 长上下文中误差累积。
3. 不同层、不同 head 对量化敏感性不同。
4. kernel 实现复杂。
5. 质量和延迟收益需要实测。

KV cache quantization 是实用工程优化，但不能把它当成无损免费的优化。

11.18 长上下文推理的延迟拆解

一个请求的总延迟可以粗略拆成：

$$T_{\mathrm{e2e}} = T_{\mathrm{prefill}} + T_{\mathrm{decode}} + T_{\mathrm{queue}} + T_{\mathrm{post}}$$

prefill latency 主要受：

1. prompt length。
2. $O(n^2)$ attention。
3. model size。
4. batching。
5. attention kernel。

decode latency 主要受：

1. output length。
2. 每步读取 KV cache 的成本。
3. batch size。
4. sampling/beam search。
5. memory bandwidth。
6. KV cache 管理。

长 prompt + 长输出是最难的组合：

prefill 很重，decode cache 也越来越大。

11.19 吞吐、延迟、显存的三角关系

Serving 系统经常要在三者之间折中：

吞吐：单位时间生成多少 tokens。

延迟：单个请求等多久。

显存：能放多少权重、cache、batch。

提高 batch size 通常能提升吞吐，但会增加每个请求等待和 KV cache 显存。

支持更长上下文能提升产品能力，但会降低并发和吞吐。

压缩 KV cache 能提高并发，但可能引入质量或实现复杂度风险。

所以真实工程里通常要按场景优化：

1. 聊天助手：更重视低延迟和稳定吞吐。
2. 长文档分析：更重视长 prompt prefill 能力。
3. 批量离线生成：更重视吞吐和成本。
4. 代码仓库分析：长上下文和检索结合更重要。
5. Agent 场景：多轮上下文、prefix sharing、工具调用格式很重要。

11.20 面向专家：KV Cache 和 Tensor Parallel

大模型常用 tensor parallel 把 attention heads 分到多张 GPU 上。

如果按 head 切分，每张卡只保存自己负责 heads 的 KV cache。

这能分摊 cache，但也带来通信和调度问题。

例如：

1. logits 需要跨卡聚合。
2. attention output 可能需要 all-reduce。
3. cache layout 要和 tensor parallel partition 对齐。
4. GQA/MQA 的 KV heads 较少时，head 切分可能不均衡。
5. pipeline parallel 下不同层的 cache 分布在不同 stage。

所以 KV cache 优化不能只看单卡公式，还要考虑分布式并行布局。

11.21 面向专家：为什么 KV Cache 影响架构设计

过去很多架构设计更关注训练效果。

现代 LLM 越来越多地把 serving 成本前置考虑。

原因是：

模型训练一次，推理服务可能运行数十亿次。

如果一个结构训练时效果稍好，但 KV cache 大很多，线上成本可能不可接受。

这就是为什么 MQA、GQA、MLA 变得重要。

它们都在回答同一个问题：

能否减少每个历史 token 必须长期保存和反复读取的信息量？

这也是后续架构演进的重要方向：

1. 更少 KV heads。
2. 更低维 KV cache。
3. KV cache quantization。
4. sliding window cache。
5. retrieval/memory 替代一部分上下文。
6. state-space/recurrent 架构减少显式缓存。

11.22 KV Cache 成本审计指标与最小 demo

第二轮精修时，本章建议把 KV cache 问题落到一组可审计指标：

1. kv_cache_gib：不同 KV head 数、上下文长度和 batch 下的 KV cache 显存。

2. kv_cache_per_token: 每个新 token 会给每个请求追加多少 cache。
3. decode_read_per_step: decode 单步需要读取的历史 K/V 量级, 用来估算带宽压力。
4. gqa_vs_mha_saving: GQA / MQA 相比 MHA 的 cache 节省比例。
5. paged_block_waste: 按 fixed-size blocks 管理 KV cache 时尾部浪费多少 token。
6. kv_cache_gate: 把显存容量、decode 带宽、分页浪费和上下文预算组合成上线门禁。

下面是一个 0 依赖 demo。它不模拟真实 kernel, 也不代表某个 serving 框架的默认配置, 只演示 KV cache 的线性增长、KV head 压缩收益和分页块尾部浪费。

```
BYTES_PER_ELEM = 2
```

```
def gib(num_bytes):
    return num_bytes / (1024 ** 3)
```

```
def kv_cache_bytes(layers, batch, seq_len, kv_heads, head_dim, bytes_per_elem):
    return layers * batch * seq_len * kv_heads * head_dim * 2 * bytes_per_elem
```

```
def block_stats(prompt_lens, block_size):
    total_blocks = sum((length + block_size - 1) // block_size for length in prompt_lens)
    total_capacity = total_blocks * block_size
    used_tokens = sum(prompt_lens)
    waste_tokens = total_capacity - used_tokens
    return total_blocks, waste_tokens, waste_tokens / total_capacity if total_capacity else 0.0
```

```
layers = 32
batch = 16
seq_len = 8192
head_dim = 128
q_heads = 32
mha_kv_heads = 32
gqa_kv_heads = 8
mqa_kv_heads = 1
```

```
mha_gib = gib(kv_cache_bytes(layers, batch, seq_len, mha_kv_heads, head_dim, BYTES_PER_ELEM))
gqa_gib = gib(kv_cache_bytes(layers, batch, seq_len, gqa_kv_heads, head_dim, BYTES_PER_ELEM))
mqa_gib = gib(kv_cache_bytes(layers, batch, seq_len, mqa_kv_heads, head_dim, BYTES_PER_ELEM))
long_gqa_gib = gib(kv_cache_bytes(layers, batch, seq_len * 4, gqa_kv_heads, head_dim, BYTES_PER_ELEM))
per_token_gqa_kib = kv_cache_bytes(layers, 1, 1, gqa_kv_heads, head_dim, BYTES_PER_ELEM) / 1024
```

```
decode_read_gib_per_step = gib(kv_cache_bytes(layers, batch, seq_len, gqa_kv_heads, head_dim, BYTES_PER_ELEM))
```

```
prompt_lens = [210, 8192, 997, 64, 4097]
blocks, waste, waste_ratio = block_stats(prompt_lens, block_size=16)
```

```
failed_gates = []
if gqa_gib > 20:
    failed_gates.append("gqa_cache_over_20gib")
if long_gqa_gib > 80:
    failed_gates.append("long_context_cache_over_80gib")
if waste_ratio > 0.08:
    failed_gates.append("paged_block_waste_over_8pct")
if decode_read_gib_per_step > 25:
```



```

failed_gates.append("decode_bandwidth_pressure")

print("mha_8k_cache_gib=", round(mha_gib, 2))
print("gqa_8k_cache_gib=", round(gqa_gib, 2))
print("mqa_8k_cache_gib=", round(mqa_gib, 2))
print("gqa_vs_mha_saving=", round(1 - gqa_gib / mha_gib, 3))
print("gqa_32k_cache_gib=", round(long_gqa_gib, 2))
print("gqa_cache_per_token_kib=", round(per_token_gqa_kib, 1))
print("decode_read_gib_per_step=", round(decode_read_gib_per_step, 2))
print("paged_blocks=", blocks)
print("paged_waste_tokens=", waste)
print("paged_waste_ratio=", round(waste_ratio, 4))
print("failed_gates=", failed_gates)
print("kv_cache_gate_pass=", len(failed_gates) == 0)

```

示例输出：

```

mha_8k_cache_gib= 64.0
gqa_8k_cache_gib= 16.0
mqa_8k_cache_gib= 2.0
gqa_vs_mha_saving= 0.75
gqa_32k_cache_gib= 64.0
gqa_cache_per_token_kib= 128.0
decode_read_gib_per_step= 16.0
paged_blocks= 850
paged_waste_tokens= 40
paged_waste_ratio= 0.0029
failed_gates= []
kv_cache_gate_pass= True

```

这个 demo 的读法是：

1. 同样 8K 上下文、16 并发、32 层、D_h=128，MHA cache 约 64 GiB，GQA 8 个 KV heads 约 16 GiB，MQA 约 2 GiB。
2. GQA 从 32 个 KV heads 降到 8 个 KV heads，cache 节省约 75%。
3. GQA 下上下文从 8K 扩到 32K，cache 仍然线性变成 4 倍，约 64 GiB。
4. 每个新 token 给单请求追加约 128 KiB cache；长输出和多并发会持续吃掉显存。
5. block/page 管理可以把不同长度请求映射到固定块，尾部浪费由 block size 和请求长度分布共同决定。

11.23 常见误区

误区 1：KV cache 会减少模型权重显存

不会。KV cache 是额外的推理状态，不是模型权重压缩。

误区 2：KV cache 越大，模型效果一定越好

不一定。更长上下文提供更多可见信息，但模型是否能有效利用，还受位置编码、训练长度、attention pattern、干扰信息和任务影响。

误区 3：FlashAttention 能解决 KV cache 过大

不准确。FlashAttention 优化 attention kernel 的 IO 和中间显存，不从源头减少历史 K/V cache 总量。

误区 4：PagedAttention 是 GQA 的替代品

不是。PagedAttention 管理 KV cache 内存，GQA 减少 KV cache 大小。两者可以组合使用。

误区 5：长上下文只影响 prefill

不对。长 prompt 影响 prefill；长历史和长输出还会增加 decode 阶段的 KV cache 读取和显存占用。

误区 6：MQA/GQA 只是减少参数

主要不是。它们的关键收益是减少 KV cache 和 decode 内存带宽。

11.24 面试高频问题

题 1：KV cache 是什么？为什么需要它？

参考回答：

KV cache 是 decoder-only LLM 推理时缓存每层历史 token 的 key 和 value。自回归生成每步只新增一个 token，历史 token 的 K/V 不

题 2：KV cache 显存如何估算？

参考回答：

KV cache 大小约等于 $\text{num_layers} * \text{batch_size} * \text{seq_len} * \text{num_kv_heads} * \text{head_dim} * 2 * \text{bytes_per_element}$ 。其中 2 表示 K 和 V

题 3：Prefill 和 decode 的区别是什么？

参考回答：

Prefill 是处理输入 prompt 的阶段，query_len 和 key_len 都是 prompt 长度，需要构建 prompt 的 KV cache，长 prompt 下 attention

题 4：为什么长上下文推理会被 KV cache 卡住？

参考回答：

KV cache 随上下文长度、batch size、层数、KV head 数线性增长。长上下文下每个请求都需要保存大量历史 K/V，高并发时显存很快

题 5：MQA/GQA 为什么能提升推理速度？

参考回答：

MQA/GQA 减少 K/V head 数。标准 MHA 每个 query head 都有独立 K/V head，KV cache 大且 decode 时读取带宽高。MQA 让所有 query h

题 6：PagedAttention 解决什么问题？

参考回答：

PagedAttention 解决 LLM serving 中 KV cache 动态内存管理问题。不同请求长度不同、不断加入和结束，连续分配容易造成碎片和预

题 7：FlashAttention 和 PagedAttention 有什么区别？

参考回答：

FlashAttention 是 attention kernel 优化，通过 tiling 和 IO-aware 计算减少 HBM 读写和中间显存，不改变 attention 数学结果。

题 8：MLA 和 GQA 的区别是什么？

参考回答：

GQA 通过减少 KV head 数降低 KV cache。MLA 更进一步，把 K/V 信息压缩到 latent 表示中缓存，需要时再恢复或投影出 attention 所

11.25 小练习

1. 推导 KV cache 显存公式，并解释每一项含义。
2. 计算 32 层、batch=8、seq_len=32768、num_kv_heads=8、head_dim=128、bf16 的 KV cache 大小。
3. 画出 prefill 和 decode 阶段的 query/key 长度关系。
4. 比较 MHA、MQA、GQA 的 KV cache 大小。
5. 解释为什么 decode 阶段常常受内存带宽限制。
6. 用自己的话解释 PagedAttention 为什么像操作系统分页。
7. 讨论 prefix sharing 在多轮对话和多样本采样中的作用。
8. 设计一个长上下文 serving 的监控面板，至少包含 TTFT、tokens/s、KV cache 使用率、batch size、cache fragmentation、OOM 数量。
9. 运行 11.22 的 KV Cache 成本审计 demo，把 gqa_kv_heads 从 8 改成 4，解释 cache 显存、decode 读取量和质量风险之间的 trade-off。
10. 把 block_size 从 16 改成 64，观察 paged_waste_ratio 的变化，并解释 block size 为什么不是越小越好。

11.26 本章总结

本章讲了 KV Cache、长上下文推理和显存增长问题。

核心结论：

1. KV cache 缓存每层历史 token 的 K/V，是 decoder-only LLM 高效自回归推理的基础。
2. Prefill 负责处理 prompt 并构建 cache，长 prompt 下受 $O(n^2)$ attention 影响。
3. Decode 每步只处理一个新 token，但要读取历史 K/V，常受显存和内存带宽限制。
4. KV cache 大小随层数、batch、seq_len、num_kv_heads、head_dim 线性增长。
5. 长上下文和高并发 serving 中，KV cache 往往比模型权重更限制 batch size。
6. MQA/GQA 通过减少 KV head 数降低 cache；MLA 通过压缩 KV 表示进一步降低 cache。
7. PagedAttention/vLLM 通过分页式 cache 管理减少碎片和重复，提高吞吐。
8. 长上下文推理需要在延迟、吞吐、显存、质量和实现复杂度之间做系统 trade-off。
9. KV cache 审计要同时看 cache GiB、per-token cache 增长、decode 读取带宽、GQA/MQA 节省、block/page 浪费和上线门禁。

下一章会进入 In-Context Learning 与显式 token 检索能力，解释模型如何在上下文中利用示例、指令和证据 token 完成临时任务。

第十二章：In-Context Learning 与显式 token 检索能力

12.0 本讲资料边界与第二轮精修口径

截至 2026-06-10，本讲用公开论文、综述和 mechanistic interpretability 资料校准 In-Context Learning，简称 ICL，以及上下文内显式 token 检索能力。这里讨论的是 decoder-only / Transformer LLM 在固定参数前向推理中利用 prompt 示例、任务说明、文档证据和工具返回结果的能力，不把某个 benchmark、某个 prompt 排列技巧、某个 induction head 观察或某个长上下文模型报告泛化成通用结论。

本讲重点区分五层问题：

1. 任务适配：ICL 不更新参数，任务信息通过 prompt 进入上下文，并在前向计算中被利用。
2. 示例作用：few-shot demonstrations 同时提供 label space、输入分布、输出格式、任务 framing 和输入输出映射，正确标签的重要性随任务而变。
3. 内部检索：self-attention 可以把当前 query 位置路由到上下文中的相关 token，但“可见”不等于“稳定使用”。
4. 位置风险：长上下文中的首尾偏置、中间证据遗失、示例顺序和噪声 token 都会影响 ICL。
5. 架构评估：比较 Transformer、SSM、Mamba-like、RWKV、RetNet 或混合架构时，除了复杂度和吞吐，还要测 few-shot ICL、copy、passkey、变量绑定和多文档证据读取。

后面的公式默认使用：

1. x 表示当前输入， y 表示目标输出。

2. $\mathcal{C}$ 表示 prompt 中的 demonstration / evidence context。
3. θ 表示固定模型参数。
4. m 表示上下文示例数。
5. q 表示当前预测位置的 query, k_j 和 v_j 表示第 j 个可见 token 的 key 和 value。
6. $a_{\{qj\}}$ 表示 query 位置对 token j 的 attention 权重。

12.1 本章定位

前面几章讲了 Transformer 的核心优势和瓶颈：并行性、 $O(n^2)$ attention、KV cache 和长上下文推理成本。本章进入一个更偏“能力机制”的主题：In-Context Learning, 简称 ICL。

GPT-3 之后，大家意识到大模型不一定每个任务都要 fine-tuning。只要在 prompt 里放任务说明、少量示例、相关文档或工具返回结果，模型就可能在更新参数的情况下完成任务。

例如：

English: cat
French: chat

English: dog
French: chien

English: book
French:

模型看到前两个例子后，可以推断这是英文到法文翻译任务，然后输出 livre。

这就是 In-Context Learning 的直观形式：

模型把上下文当作临时任务说明、临时数据集和临时工作记忆。

本章要回答的问题是：

1. In-Context Learning 到底是什么。
2. 它和 fine-tuning、RAG、prompt engineering 有什么区别。
3. 为什么 attention 对 ICL 很关键。
4. “显式 token 检索能力”是什么意思。
5. 示例、标签空间、输入分布、格式分别起什么作用。
6. 为什么 ICL 不等于真正参数更新。
7. ICL 为什么会受位置、顺序、干扰信息和上下文长度影响。
8. 为什么显式检索能力是 Transformer 与 Mamba/SSM 等替代架构比较时的关键问题。

本章的核心观点是：

ICL 是 Transformer 把上下文窗口变成运行时工作空间的能力；attention 让模型可以显式读取 prompt 中的示例、标签、约束和证据。

12.2 资料来源和可信边界

本章主要参考以下公开资料：

1. Brown et al., 2020, Language Models are Few-Shot Learners。GPT-3 展示大规模自回归语言模型可以通过 prompt 中的 few-shot demonstrations 完成任务。
2. Min et al., 2022, Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?。研究 demonstrations 中哪些因素影响 ICL，指出真实标签不总是必要，label space、input distribution 和 format 很关键。
3. Dai et al., 2022/2023, Why Can GPT Learn In-Context? Language Models Implicitly Perform Gradient Descent as Meta-Optimizers。从 meta-optimizer 角度解释 ICL，提出 Transformer attention 与梯度下降之间的类比。
4. Dong et al., 2023, A Survey on In-context Learning。系统综述 ICL 的定义、技术、应用和挑战。
5. Liu et al., 2023, Lost in the Middle。说明长上下文模型虽然能接收长输入，但对中间位置相关信息利用不稳定。

6. Guu et al., 2020, REALM: Retrieval-Augmented Language Model Pre-Training。代表把外部检索与语言模型结合的早期路线，用来区分上下文内部检索和外部 RAG。
7. mechanistic interpretability 中关于 induction heads 的研究。它说明某些 attention head 会实现类似“看到 A 后跟 B，当前再看到 A 时预测 B”的复制/归纳模式。本章只讲直觉，不把 mechanistic 结论绝对化。

需要说明的是，ICL 的机制仍是开放研究问题。不同论文从不同角度解释：模式匹配、贝叶斯推断、隐式梯度下降、检索、归纳头、数据分布记忆等都能解释一部分现象。本章重点建立工程和面试中实用的理解框架。

12.3 什么是 In-Context Learning

In-Context Learning 指的是：

模型在推理时仅通过输入上下文中的任务描述、示例或证据，临时适配当前任务，不更新模型参数。

关键点有三个。

第一，不更新参数。

模型权重固定，没有反向传播，没有 optimizer step。

第二，任务信息在 prompt 中。

任务说明、格式、标签、示例、文档证据都以 token 形式放进上下文。

第三，适配发生在前向计算中。

模型通过 attention 和深层非线性计算，把上下文信息转化为当前输出条件。

所以 ICL 可以看作一种运行时能力：

参数里存储通用能力，上下文里提供临时任务配置。

用公式写，few-shot context 可以表示为：

$$\mathcal{C} = \{(x_i, y_i)\}_{i=1}^m$$

其中 m 是上下文中的示例数量， x_i 是第 i 个示例输入， y_i 是对应输出或标签。模型参数固定为 θ 时，对当前输入 x 的生成分布可以写成：

$$P_{\theta}(y \mid x, \mathcal{C}) = \prod_{t=1}^T y_t P_{\theta}(y_t \mid y_{1:t-1}, x, \mathcal{C})$$

这条公式强调两点：第一，条件里多了 $\mathcal{C}$ ，说明输出受 prompt 中示例和证据影响；第二，下标仍是 θ ，说明参数没有更新。ICL 改变的是推理时的条件上下文，不是把新任务写入权重。

12.4 Zero-shot、One-shot、Few-shot

ICL 常见形式包括 zero-shot、one-shot 和 few-shot。

Zero-shot：

只给任务说明，不给示例。

例如：

请判断下面评论是正面还是负面：

“这个电影节奏太慢了。”

One-shot：

给一个示例。

例如：

评论：这个手机很好用。

情感：正面

评论：这个电影节奏太慢了。

情感：

Few-shot：

给多个示例。

few-shot 示例能提供更多信息：

1. 任务格式。
2. 标签空间。
3. 输入分布。
4. 输出风格。
5. 边界样例。
6. 隐含规则。

GPT-3 的冲击在于，它展示了模型规模足够大时，few-shot prompt 可以在很多任务上接近或超过一些传统 fine-tuning baseline。

12.5 ICL 和 Fine-tuning 的区别

Fine-tuning 是参数更新：

训练样本 -> 计算 loss -> 反向传播 -> 更新参数

ICL 是上下文条件化：

示例和任务说明放进 prompt -> 前向推理 -> 输出答案

对比：

Fine-tuning：慢，需训练数据和算力，结果固化到参数中。

ICL：快，不更新参数，任务配置保留在上下文中。

ICL 的优点：

1. 快速切换任务。
2. 不需要训练流程。
3. 可解释性较好，因为示例和规则在 prompt 中可见。
4. 适合用户临时指令、工具结果、文档证据。
5. 可以和 RAG、Agent、workflow 结合。

ICL 的缺点：

1. 受上下文长度限制。
2. 每次调用都要携带示例，增加 token 成本。
3. 对 prompt 格式、顺序、示例选择敏感。
4. 容易被干扰 token 影响。
5. 不一定能学习真正复杂的新技能。

12.6 ICL 和 Prompt Engineering 的关系

Prompt engineering 是设计输入上下文的方法。

ICL 是模型利用上下文适配任务的能力。

两者关系是：

Prompt engineering 负责把任务信息组织成 token。

ICL 负责让模型在前向计算中利用这些 token。

例如一个好的 few-shot prompt 会考虑：

1. 示例是否代表目标分布。
2. 标签是否清晰。
3. 输出格式是否一致。
4. 示例顺序是否合理。
5. 任务说明是否和示例一致。
6. 是否避免无关干扰信息。
7. 关键证据是否放在模型容易利用的位置。

Prompt engineering 不是玄学堆词，而是在给 ICL 提供更清晰的运行时数据结构。

12.7 ICL 和 RAG 的区别

RAG 是 Retrieval-Augmented Generation。

典型流程是：

用户问题 -> 外部检索系统找文档 -> 文档放入 prompt -> LLM 基于上下文回答

ICL 和 RAG 的关系：

RAG 提供外部资料进入上下文。

ICL/attention 在上下文内部读取这些资料并生成答案。

所以 RAG 可以看成一种 ICL 输入构造方式。

区别在于：

ICL 强调模型如何利用上下文完成任务。

RAG 强调如何从外部知识库找相关内容并放进上下文。

Self-Attention 像上下文内部的可微分检索；RAG 是上下文外部的显式检索。

二者配合后：

外部检索扩展知识边界。

内部 attention 决定哪些 token 被真正使用。

12.8 为什么 Attention 对 ICL 很关键

ICL 需要模型在当前输入中找到相关信息。

例如 few-shot 分类：

输入：这家餐厅太棒了。

标签：正面

输入：服务很差，再也不来了。

标签：负面

输入：味道不错，但价格太贵。

标签：

模型需要读取：

1. 示例中的输入文本。
2. 示例对应标签。
3. 当前输入。
4. 输入和标签的格式关系。

Attention 提供了这种 token-to-token 路由能力。

当前要预测标签的位置，可以关注前面示例中的：

1. 标签：正面。
2. 标签：负面。
3. 相似情感表达。
4. 分隔符和格式 token。

如果没有 attention 的上下文内显式读取能力，模型很难稳定利用 prompt 中任意位置的示例和证据。

一个 query 位置对上下文 token 的读取可以简化写成：

$$a_{\{qj\}} = \frac{\exp(q^{\text{top } k_j} / \sqrt{d})}{\sum_{\ell \in \mathcal{V}_q} \exp(q^{\text{top } k_{\ell}} / \sqrt{d})},$$

$$z_q = \sum_{j \in \mathcal{V}_q} a_{\{qj\}} v_j$$

这里 $\mathcal{V}_q$ 是 query 位置可见的 token 集合， d 是 head dimension， z_q 是读出的上下文信息。对 ICL 来说，关键不是 attention 公式本身多复杂，而是相关示例、标签、字段值和证据句能否在 $a_{\{qj\}}$ 中获得足够权重，并在后续层中被保留下来。

12.9 显式 Token 检索能力是什么意思

这里的“显式 token 检索能力”不是指外部向量数据库检索，而是指：

模型能在上下文 token 中定位、读取和使用相关 token。

例如：

1. 从前文找出变量定义。
2. 从 few-shot 示例中找相似输入和对应标签。
3. 从文档证据中找答案句。
4. 从工具返回 JSON 中找字段值。
5. 从代码上下文中找函数签名。
6. 从多轮对话中找用户最新约束。

这是一种内部检索能力。

它依赖：

1. attention score 是否把相关 token 权重拉高。
2. 位置编码是否支持距离关系。
3. 训练中是否见过类似格式。
4. 模型是否能抗干扰。
5. 后续层是否保留和加工检索到的信息。

显式 token 检索能力是 Transformer 的强项，也是长上下文任务的基础。

工程上可以把上下文内检索看成一个候选示例打分问题：

$$r_i = \lambda_s s(x, x_i) + \lambda_p p_i - \lambda_n n_i$$

其中 $s(x, x_i)$ 表示当前输入和第 i 个示例的相似度， p_i 表示位置带来的可利用性， n_i 表示噪声或冲突程度， $\lambda_s, \lambda_p, \lambda_n$ 是分析时的权重。真实 LLM 不会显式执行这条 toy 公式，但它有助于面试表达：ICL 质量同时受内容相关性、位置、噪声和格式影响。

12.10 示例到底提供了什么

Min et al. 的研究指出，demonstrations 中真实标签不总是必要。随机替换标签后，在一些分类和多选任务上性能下降没有想象中那么大。

这说明 few-shot 示例提供的不只是“正确样本”。它还提供：

1. Label space.
2. Input distribution.

3. Output format。
4. Task framing。
5. Token pattern。

例如：

评论：...

情感：正面/负面

即使示例标签被打乱，模型仍然知道：

这是情感分类任务。

候选标签是正面和负面。

输出应该是一个标签，而不是长篇解释。

当然，这不代表标签永远不重要。在需要学习新映射、特殊标签含义、复杂规则或结构化输出时，正确示例会非常重要。

更准确的结论是：

ICL 中示例同时提供格式、分布、标签空间和任务映射；不同任务对这些因素的依赖不同。

12.11 Induction Head 的直觉

Mechanistic interpretability 中常讨论 induction heads。

一个简化例子：

A -> B

...

A -> ?

如果模型在上下文前面看到 A 后面跟着 B，当后面再次看到 A 时，它可能预测 B。

这类似一种上下文内复制和归纳机制。

直觉上，induction head 做的事情像：

找到当前 token 在前文中出现的位置。

读取那个位置后面的 token。

把后继 token 用于当前预测。

这和 ICL 很相关，因为很多 few-shot 任务都可以看作：

在上下文中找模式 -> 把模式应用到当前输入

但要注意，真实模型中的 ICL 不可能只靠一个 induction head 完成。它涉及多层、多头、FFN、residual stream 和训练数据分布。

12.12 ICL 作为隐式优化器

另一种解释是把模型看成 meta-optimizer。

Dai et al. 的工作从理论和经验角度讨论：Transformer attention 与梯度下降存在某种 dual form，可以把 ICL 理解成隐式 fine-tuning。

直觉是：

示例相当于临时训练数据。

模型前向计算中产生某种“任务适配”的中间表示。

当前预测基于这些示例产生的临时适配结果。

这不是说模型真的执行了标准反向传播。

更稳妥的表达是：

ICL 的行为在某些任务上类似 fine-tuning，但实现机制是在固定参数的前向计算中完成的。

面试中不要把“ICL 等于梯度下降”说得过满。应该说：这是一个有启发的理论视角，可以解释部分现象，但 ICL 还有模式匹配、检索、先验记忆等多种机制共同参与。

12.13 上下文是临时程序

可以把 prompt 看成一种临时程序。

例如：

你是一个严格的 JSON 转换器。

把输入文本转换成 {"name": ..., "age": ...}。

输入: Alice is 20 years old.

输出: {"name": "Alice", "age": 20}

输入: Bob is 35 years old.

输出:

这里上下文提供了：

1. 角色约束。
2. 输出 schema。
3. 示例映射。
4. 当前输入。

模型不是永久学会了这个新 schema，而是在当前上下文里执行这个临时程序。

这也是为什么 agent、tool calling、RAG、workflow 都大量依赖 prompt：它们把运行时状态、工具说明、历史轨迹、候选文档和输出格式都编码成 token。

12.14 为什么 ICL 受示例顺序影响

ICL 对示例顺序敏感。

原因包括：

1. Causal LM 从左到右读取上下文。
2. 越靠近输出的位置通常更容易被利用。
3. 示例之间可能产生格式或语义干扰。
4. 模型可能对首尾位置有偏置。
5. 近期示例可能覆盖早期示例。

例如同样几个分类示例，顺序不同，最终预测可能不同。

这说明 ICL 不是理想化的统计学习算法，它受 Transformer 架构、位置编码、训练分布和 prompt 格式共同影响。

工程上常用策略包括：

1. 把最相关示例放在靠近 query 的位置。
2. 保持示例格式一致。
3. 避免标签分布严重偏斜。
4. 使用清晰分隔符。
5. 对候选 prompt 做验证集选择。

12.15 为什么 ICL 受干扰信息影响

上下文窗口不是无损数据库。

放入更多 token 可能带来更多证据，也可能带来更多噪声。

干扰信息会影响：

1. Attention 权重竞争。
2. 输出格式判断。
3. 任务边界识别。
4. 标签先验。
5. 多证据整合。
6. 幻觉和错误引用。

例如 RAG 中 top-k 过大，把很多无关 chunk 放进 prompt，模型可能反而回答更差。

所以长上下文不是越长越好。更合理的目标是：

把相关、完整、低噪声、格式清晰的信息放入上下文。

12.16 Lost in the Middle 与显式检索失败

Lost in the Middle 研究显示，相关信息放在上下文开头或结尾时，模型表现往往更好；放在中间时，性能可能明显下降。

这说明：

token 可见 != token 会被稳定使用

对 ICL 和显式 token 检索来说，这非常关键。

模型可能接收 100K token，但如果证据句在中间位置被忽略，那么长上下文能力并不可靠。

原因可能包括：

1. 训练中长距离精确检索样本不足。
2. 位置编码外推或位置偏置。
3. attention 权重被首尾或近邻信息吸引。
4. 中间信息在多层传播中被稀释。
5. 干扰文档太多。

后续第 13 章会专门展开长距离依赖、精确检索和 lost-in-the-middle。

12.17 ICL、RAG 和 Long Context 的组合

现代系统经常把三者结合。

RAG：从外部找文档。

Long context：把更多文档、工具结果、历史轨迹放进 prompt。

ICL：模型在 prompt 内读取、归纳、执行任务。

例如企业问答：

用户问题 -> 检索相关制度文档 -> 放入 prompt -> 模型根据文档回答并引用

这里模型需要：

1. 读取用户问题。
2. 在上下文文档中找证据。
3. 忽略无关 chunk。
4. 按要求输出答案和引用。
5. 在资料不足时拒答。

这不是单纯“语言生成”，而是上下文内任务执行。

12.18 为什么 ICL 是替代架构的重要考题

很多替代架构，例如 SSM、Mamba、RWKV、RetNet，都试图降低 Transformer 的长序列成本。

但它们要替代 Transformer，需要回答一个关键问题：

能否像 full attention 一样，在上下文中精确读取任意位置的示例、证据和绑定关系？

Transformer 的 full attention 有天然优势：

任意 token pair 可以直接交互。

状态空间或 recurrent 类模型通常更强调压缩历史到固定或有限状态。压缩可以提高效率，但可能损失精确 token-level retrieval。

因此比较架构时，不只看 perplexity 或吞吐，还要看：

1. few-shot ICL。
2. copy/retrieval。
3. passkey retrieval。
4. multi-document QA。
5. 代码变量引用。
6. 工具输出字段读取。
7. 长上下文中间证据利用。

这就是为什么 ICL 和显式 token 检索能力是下一代架构能否取代 Transformer 的关键评估项。

12.19 面向专家：ICL 的多机制解释

ICL 不是单一机制。

可以把它拆成多种成分：

1. 格式识别：识别输入输出模板。
2. 标签空间识别：知道候选标签有哪些。
3. 分布适配：理解当前样本来自什么分布。
4. 最近邻检索：找到相似示例。
5. 规则归纳：从示例中归纳映射。
6. 复制机制：把前文某些 token 复制到输出。
7. 先验调用：使用预训练中学到的任务知识。
8. 隐式优化：表现得像在上下文中做临时适配。

不同任务依赖不同机制。

例如：

情感分类：标签空间、格式和语义先验很重要。

人工构造映射：正确示例和绑定关系很重要。

代码补全：局部检索、变量绑定、语法模式很重要。

RAG 问答：证据检索、引用和抗干扰很重要。

所以讨论 ICL 时要避免单因解释。

12.20 面向专家：显式检索和隐式知识的分工

模型回答问题可能有两类来源。

第一，参数中的隐式知识：

预训练中学到的事实、语言模式、推理模板。

第二，上下文中的显式 token：

prompt、文档、示例、工具返回、用户约束。

很多失败来自二者冲突。

例如文档说：

退款期限是 14 天。

但模型参数中常见模式可能是：

退款期限是 30 天。

一个好的 RAG/ICL 模型应该优先使用上下文证据，而不是凭参数先验回答。

这就要求模型具备：

1. 上下文证据定位。
2. 参数知识抑制。
3. 冲突检测。
4. 引用约束遵守。
5. 资料不足拒答。

这也是企业 RAG 和长上下文问答中非常关键的能力。

12.21 ICL / Token Retrieval 审计指标与最小 demo

第二轮精修时，本章建议把 ICL 质量落到一组可审计指标：

1. `label_space_coverage`: prompt 中是否覆盖任务需要的候选标签或输出类型。
2. `format_consistency`: 示例是否使用一致的输入字段、输出字段和分隔符。
3. `context_evidence_use`: 模型预测是否真正使用上下文中的相关示例或证据，而不是只靠参数先验。
4. `retrieval_label_match`: 最相关示例或证据对应的标签是否支持最终答案。
5. `middle_evidence_risk`: 关键示例或证据是否落在长上下文中更容易被忽略的位置。
6. `conflict_rate`: 上下文中是否存在相似输入但标签或结论冲突的 bad case。
7. `icl_gate`: 把标签空间、格式、检索命中、位置风险和冲突证据组合成上线门禁。

标签空间覆盖可以写成：

$$C_{\{\mathrm{label}\}} = \frac{|\{\mathrm{prompt}\} \cap \{\mathrm{task}\}|}{|\{\mathrm{task}\}|}$$

其中 $\{\mathrm{prompt}\}$ 是 prompt 中实际出现的标签集合， $\{\mathrm{task}\}$ 是任务需要覆盖的标签集合。格式一致率可以写成：

$$C_{\{\mathrm{fmt}\}} = \frac{1}{m} \sum_{i=1}^m \mathbf{1}[f_i=1]$$

其中 $f_i=1$ 表示第 i 个示例具备完整字段和一致格式。一个简化门禁可以写成：

$$G_{\{\mathrm{icl}\}} = \mathbf{1} \left[\begin{aligned} &C_{\{\mathrm{label}\}} \geq \tau_{\{\mathrm{label}\}} \\ &\text{and } C_{\{\mathrm{fmt}\}} \geq \tau_{\{\mathrm{fmt}\}} \\ &\text{and } R_{\{\mathrm{mid}\}} \leq \tau_{\{\mathrm{mid}\}} \\ &\text{and } M_{\{\mathrm{ret}\}} \geq \tau_{\{\mathrm{ret}\}} \\ &\text{and } R_{\{\mathrm{conf}\}} \leq \tau_{\{\mathrm{conf}\}} \end{aligned} \right]$$

这里 R_{mid} 是关键证据落在中间位置且可能被忽略的比例， M_{ret} 是检索到的最相关示例支持正确标签的比例， R_{conf} 是相关但结论冲突的示例比例。它们可以用下面的 toy 口径理解：

$$M_{\{\mathrm{ret}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_{\{\mathrm{top}\}}, i = y_i^*]$$

$$R_{\mathrm{mid}} = \frac{|\{e_i: e_i \in E_{\mathrm{rel}}, p_i \in [p_{\mathrm{lo}}, p_{\mathrm{hi}}]\}|}{|E_{\mathrm{rel}}|}$$

$$R_{\mathrm{conf}} = \frac{|\{e_i: e_i \in E_{\mathrm{rel}}, y_i \neq y^{\star}\}|}{|E_{\mathrm{rel}}|}$$

其中 E_{rel} 是和当前 query 足够相似的上下文示例集合, p_i 是示例在 prompt 中的相对位置, y_i 是示例标签, y_{star} 是当前样本的正确标签。真实系统还应加入人工评估、模型输出日志、引用检查和分桶 bad case; 下面的 0 依赖 demo 只演示这些指标的直觉。

```
def token_set(text):
    return set(text.split())

def jaccard(a, b):
    a, b = token_set(a), token_set(b)
    return len(a & b) / len(a | b) if a or b else 0.0

def position_bonus(position):
    # Toy U-shape: beginning/end evidence is easier than middle evidence.
    return 0.20 * abs(position - 0.5) * 2 - 0.10

def ratio(count, total):
    return round(count / total, 3) if total else 0.0

def audit_prompt(name, examples, query, expected_label, task_labels):
    label_space = sorted({ex["label"] for ex in examples})
    label_coverage = ratio(len(set(label_space) & set(task_labels)), len(task_labels))
    format_consistency = ratio(
        sum("text" in ex and "label" in ex and "position" in ex for ex in examples),
        len(examples),
    )
    records = []
    for ex in examples:
        sim = jaccard(query, ex["text"])
        score = 2.0 * sim + position_bonus(ex["position"]) - 0.12 * ex.get("noise", 0)
        records.append({
            "id": ex["id"],
            "label": ex["label"],
            "sim": round(sim, 3),
            "pos": ex["position"],
            "score": round(score, 3),
        })
    top = max(records, key=lambda r: r["score"])
    predicted = top["label"]
    relevant = [r for r in records if r["sim"] >= 0.3]
    middle_relevant = [r["id"] for r in relevant if 0.35 <= r["pos"] <= 0.65]
    conflicts = [r["id"] for r in relevant if r["label"] != expected_label]
    retrieval_match = int(predicted == expected_label)
    middle_risk = ratio(len(middle_relevant), len(relevant))
    conflict_rate = ratio(len(conflicts), len(relevant))
    failed = []
```

```

if label_coverage < 1.0:
    failed.append("label_space_missing")
if format_consistency < 1.0:
    failed.append("format_inconsistent")
if middle_relevant:
    failed.append("middle_evidence_risk")
if not retrieval_match:
    failed.append("retrieval_label_mismatch")
if conflicts:
    failed.append("conflicting_relevant_example")
return {
    "name": name,
    "label_space_coverage": label_coverage,
    "format_consistency": format_consistency,
    "top_example": top["id"],
    "predicted": predicted,
    "retrieval_label_match": retrieval_match,
    "middle_relevant": middle_relevant,
    "conflicting_relevant": conflicts,
    "middle_evidence_risk": middle_risk,
    "conflict_rate": conflict_rate,
    "failed": failed,
    "gate_pass": not failed,
}

```

query = " 味道 不错 但 排队 太久"

expected = " 负面"

```

base_examples = [
    {"id": "pos_food", "text": " 这家 餐厅 味道 不错 服务 热情", "label": " 正面", "position": 0.10},
    {"id": "neg_wait", "text": " 味道 不错 但 排队 太久", "label": " 负面", "position": 0.90},
    {"id": "neu_price", "text": " 环境 一般 价格 合理", "label": " 中性", "position": 0.50, "noise": 1},
]
middle_examples = [dict(ex, position=0.50 if ex["id"] == "neg_wait" else ex["position"]) for ex in base_examples]
conflict_examples = [
    {"id": "pos_food", "text": " 这家 餐厅 味道 不错 服务 热情", "label": " 正面", "position": 0.10},
    {"id": "neg_wait", "text": " 味道 不错 但 排队 太久", "label": " 负面", "position": 0.50},
    {"id": "wrong_near_end", "text": " 味道 不错 但 排队 太久", "label": " 正面", "position": 0.95},
]

for case in [
    audit_prompt("relevant_near_end", base_examples, query, expected, [" 正面", " 负面"]),
    audit_prompt("relevant_middle", middle_examples, query, expected, [" 正面", " 负面"]),
    audit_prompt("conflicting_near_end", conflict_examples, query, expected, [" 正面", " 负面"]),
]:
    print(case)

```

示例输出:

```

{'name': 'relevant_near_end', 'label_space_coverage': 1.0, 'format_consistency': 1.0, 'top_example': 'neg_wait', 'predicted': '负面'}
{'name': 'relevant_middle', 'label_space_coverage': 1.0, 'format_consistency': 1.0, 'top_example': 'neg_wait', 'predicted': '负面'}
{'name': 'conflicting_near_end', 'label_space_coverage': 1.0, 'format_consistency': 1.0, 'top_example': 'wrong_near_end', 'predicted': '正面'}

```

这段 demo 的重点不是用 Jaccard 相似度替代 LLM attention, 而是把 ICL prompt 审计拆成可检查的问题: 标签空间是否足够、格式是否一致、相关证据是否处在高风险位置、最相似示例是否支持正确标签、冲突示例是否会覆盖正确证据。真实上线前, 应把这些 toy gate 换成任务验证集、长上下文位置分桶、引用校验和人工 bad case 复盘。

12.22 常见误区

误区 1: ICL 就是模型在推理时 fine-tuning

不准确。ICL 没有参数更新。它可能表现得像临时适配，但实现是在固定参数的前向计算中完成。

误区 2: Few-shot 示例越多越好

不一定。更多示例可能带来更多格式和分布信息，也可能引入干扰、增加成本、挤占证据空间。

误区 3: 示例标签错了，ICL 就一定完全失效

不一定。一些任务中 demonstrations 主要提供标签空间、输入分布和格式。但对新映射或复杂规则，正确标签仍然很重要。

误区 4: 长上下文模型一定能利用所有 token

不一定。Lost in the Middle 表明模型对中间位置证据的利用可能不稳定。

误区 5: RAG 检索到正确文档就一定回答正确

不一定。检索只是把文档放进上下文，模型还要在内部 attention 中正确读取、整合和引用。

误区 6: ICL 是纯 prompt 技巧，和架构无关

不对。ICL 强依赖 attention、位置编码、上下文长度、训练分布和模型规模。

12.23 面试高频问题

题 1: 什么是 In-Context Learning?

参考回答:

In-Context Learning 指模型在不更新参数的情况下，仅通过 prompt 中的任务说明、示例、标签或证据来临时适配任务并输出答案。

题 2: ICL 和 fine-tuning 有什么区别?

参考回答:

Fine-tuning 会用训练样本计算 loss 并通过反向传播更新模型参数，任务能力固化到权重里。ICL 不更新参数，而是把示例和任务说明放入 prompt。

题 3: 为什么 attention 对 ICL 很重要?

参考回答:

ICL 需要模型从 prompt 中读取相关示例、标签、格式和证据。Self-Attention 让当前预测位置可以直接关注上下文中任意 token，从 few-shot 示例中找到相似输入和对应输出，或从文档中找到证据句。因此 attention 提供了上下文内部的显式 token 检索和信息路由能力。

题 4: Few-shot 示例提供了什么信息?

参考回答:

Few-shot 示例不只提供正确答案，还提供任务格式、标签空间、输入分布、输出风格和输入输出映射。研究发现某些分类任务中随机抽取的 few-shot 示例往往比精心挑选的示例效果更好。

题 5: ICL 和 RAG 有什么关系?

参考回答:

RAG 是从外部知识库检索文档并放入 prompt，ICL 是模型在上下文中利用这些 token 完成任务。可以说 RAG 提供外部记忆，Transformer 提供推理能力。

题 6：什么是显式 token 检索能力？

参考回答：

显式 token 检索能力指模型能在上下文 token 中定位并使用相关信息，例如找到变量定义、few-shot 示例标签、文档证据句、工具定义等。

题 7：为什么长上下文不等于强 ICL？

参考回答：

长上下文只是让更多 token 可见，不保证模型能稳定利用所有 token。模型可能受位置偏置、干扰信息、训练分布和 attention 竞争影响。

题 8：为什么 ICL 是替代架构的重要评估项？

参考回答：

Transformer 的 full attention 让任意 token pair 可以直接交互，因此很擅长从上下文中读取示例、证据和绑定关系。很多替代架构如 RAG 和 LLM 推理引擎都缺乏这种能力，只能做 level retrieval。要替代 Transformer，不仅要看速度和 perplexity，还要验证 few-shot ICL、复制、passkey retrieval、多文档推理等。

12.24 小练习

1. 用自己的话解释 zero-shot、one-shot、few-shot 的区别。
2. 设计一个 few-shot prompt，让模型把自然语言转换成 JSON。
3. 比较 ICL、fine-tuning、RAG 的区别。
4. 举例说明 few-shot 示例中的 label space、input distribution、format 分别是什么。
5. 思考为什么随机标签在某些 ICL 任务中仍能保留部分效果。
6. 画出 RAG + ICL 的流程：外部检索、context construction、内部 attention、答案生成。
7. 设计一个测试显式 token 检索能力的小任务，例如从长文本中找 key-value。
8. 阅读 Lost in the Middle 摘要，解释为什么“可见”不等于“会用”。

12.25 本章总结

本章讲了 In-Context Learning 与显式 token 检索能力。

核心结论：

1. ICL 是模型在不更新参数的情况下，通过上下文中的说明、示例和证据临时适配任务。
2. Prompt 是运行时任务配置，context window 是模型的临时工作空间。
3. Attention 让模型可以显式读取上下文中的示例、标签、格式、证据和工具结果。
4. Few-shot 示例提供的不只是正确答案，还包括标签空间、输入分布、输出格式和任务 framing。
5. ICL 可能表现得像隐式优化，但它不是标准参数更新。
6. RAG 是外部检索，ICL/attention 是上下文内部检索和使用，两者可以组合。
7. 长上下文不等于强 ICL，模型可能受位置偏置、干扰信息和 lost-in-the-middle 影响。
8. 显式 token 检索能力是评估 Transformer 和替代架构的重要维度。

下一章会进入长距离依赖、精确检索与 lost-in-the-middle，进一步分析为什么模型能看到长上下文却不一定能稳定用好长上下文。